

Exact Fixed-Addend Differential and Linear Theory of Modular Addition with Linear-Time Algorithms

Carry-State and Transfer Recurrences

<https://github.com/Twilight-Dream-Of-Magic>

Independent Researcher

Abstract. We study the exact XOR-differential and linear properties of the fixed-addend modular-addition map

$$f_{\text{Constant}_{\text{Key}}}(X) = Y = X + \text{Constant}_{\text{Key}} \pmod{2^N}.$$

This unary map is the cryptanalytic object that appears when a modular-addition step is analyzed with one operand fixed by a subkey or constant. The main technical contributions are explicit and self-contained. First, we derive an exact differential description in terms of a 4-state carry-pair recurrence, together with equivalent scalar and matrix formulations. Second, we derive an exact linear description in terms of a 2-state carry-transfer recurrence that evaluates fixed-addend Walsh correlations. Third, we give explicit linear-time algorithms for both quantities, so the differential probability and the linear correlation of the fixed-addend map can be computed in $O(N)$ time once the visible masks and the addend are fixed. Fourth, we validate these formulas and algorithms by exhaustive small-width checks against brute force.

The fixed-addend problem is motivated by ARX designs, RC5-type key-injection steps, and the later one-addend-fixed literature. In this paper, however, that context is used only to situate the problem. The main contribution is a theorem-and-algorithm package for the fixed-addend ARX component itself: exact differential and linear recurrences, exact transfer matrices, and explicit linear-time evaluators that can be used directly in ARX analysis, implementation, and verification.

Keywords: modular addition by a constant · fixed-addend addition · exact differential probability · exact linear correlation · carry-state recurrence · linear-time algorithm · ARX cryptanalysis

Main Technical Contributions

The core contributions of this paper are the following.

- (M1) We give an exact 4-state carry-pair formulation for XOR differentials of the fixed-addend map $x \mapsto x + \text{Constant}_{\text{Key}} \pmod{2^n}$, together with equivalent scalar and matrix recurrences.
- (M2) We give an exact 2-state carry-transfer formulation for the corresponding fixed-addend linear correlations.
- (M3) We derive explicit linear-time evaluators for both the differential and the linear quantities, making the fixed-addend problem computationally closed at cost $O(n)$.
- (M4) We provide exhaustive small-space verification against brute force, so the recurrences and the algorithms are not only stated but also implementation-checked.

Contents

1	Introduction	4
	Part I: Exact Differential Theory of the Fixed-Addend Case	5

2	Exact Differential Core	5
2.1	Differential Problem, Carry-Pair State, and Roadmap	5
2.2	Fixed-Addend Differential Probability	5
2.3	Prefix Events and the Carry-Pair Parity Law	6
2.4	One-Bit Analysis from Level $i - 1$ to Level i	7
2.5	Exact Prefix Recurrence for the Differential Probability	8
3	Consequences, Compression, and Differential Algorithms	11
3.1	Immediate Consequences of the Local Recurrence	11
3.2	Consecutive-One Blocks and the Correct Run Law	11
3.3	Exact Finite Recursion and the Four-State Form	12
3.4	Worked Differential Micro-Example	13
3.5	Algorithmic Consequence: a Prefix Dynamic Program	14
3.6	Core Conclusion of Part I	15
	Part II: Exact Linear Theory of the Fixed-Addend Case	16
4	Exact Linear Core	16
4.1	Fixed-Addend Walsh Correlation and Carry State	16
4.2	Reduction to the Carry Word	16
4.3	Conditioned Partial Walsh Sums	17
4.4	One-Bit Weighted Recurrence	18
4.5	Exact Two-State Matrix Formula	19
5	Search, Interpretation, and Linear-Time Evaluation	19
5.1	Algorithmic Consequence: a Signed Prefix Transfer	19
5.2	Core Conclusion of Part II	20
5.3	Threshold Interpretation of the Carry into Bit i	21
5.4	A Worked Linear Micro-Example	21
	Part III: Supplementary Context, Extensions, and Downstream Views	23
6	Fixed-Addend Formalization as a Specialization of the Lipmaa–Moriai View	23
7	Azimi’s Bit-Vector Reinterpretation: Validity and Approximate Weight	23
7.1	Validity Predicate	23
7.2	Approximate Weight	24
8	Quasi-Hadamard Extensions and Two-Coordinate Lifts	24
9	Pseudo-Hadamard Transform as a Four-Projection Gate	25
10	Relation to the PHT Matrix Criterion	26
11	Relation to Wallen’s Carry-Function Classification	26
12	Schulte-Geers 2011: CCZ-Equivalent Quadratic Representation of Addition	27
13	One-Variable Constant-Addition Evidence	28
13.1	The spike-addend family studied by Guan–Ding	29
13.2	A Counting Corollary and Dyadic Values	31
13.3	From Probability Tables to an $O(n)$ Search Procedure	31
14	ADP/XOR State-Machine Analogies	32
15	Relation to Partitioning Theory	32

16 Chaskey and ChaCha as Downstream Uses of Fixed-Addend Carry Laws	33
17 The ADP/XOR Matrices as a Large-State Analogy	33
18 Relation to Maximum-ADP Recurrence Formulas	34
 Part IV: Supplementary Synthesis and Technical Corollaries	 35
19 The Eight Local Types as a Projected Automaton	35
20 Run Blocks as Matrix Powers	35
21 A Fixed-Addend Linear Specialization Corollary	36
22 Li et al. 2011: Carry-Free Booleanization of Addition	36
22.1 The full two-input carry-free recurrence	37
22.2 Walsh spectrum: why the coordinates are quasi-Bent	37
22.3 Correlation immunity of the full vectorial map	38
22.4 Fixed-addend specialization	38
23 Closed Forms for Constant Addend Blocks	39
24 Single-Bit Output Masks and Walsh Coefficients of Carry Intervals	39
25 Differential Side: Summary of the Supporting Evidence	40
26 Linear Side: Summary of the Supporting Evidence	41
27 Unified View: Local State $\rightarrow$ Block Matrix $\rightarrow$ Feasibility	41
28 Conclusion	44
 A Fixed-Subtractend Dependency as the Borrow Chain Skeleton	 45
A.1 Differential side: the borrow-pair skeleton	46
A.2 Linear side: the one-borrow signed transfer	47
B Full 4-State Differential Transition Matrices	48
C Full 2-State Linear Transfer Matrices	49
D Exhaustive C++ Verification and Reference Code	50
E Appendix D: The 8-State ADP/XOR Matrices	53
F Differential and Linear Roles of the Core Papers	54
 G Historical Reconstruction Notes	 56
G.1 The RC5-Period Fixed-Addend Subproblem	56
G.2 Why the “One Addend Fixed” Case Was a Natural Explosion Point	59
G.3 How the Origin Should Be Read	60
G.4 Reconstructing the Missing DP_K and LC_K Chains	72
G.5 Miyano 1998 OCR White-Box Reading and Clean Reconstruction	79
G.6 The Missing Middle, Derived by Hand	80
G.7 What Makes a Good Differential Characteristic in the Fixed-Addend Setting	82
G.8 What Makes a Good Linear Approximation in the Fixed-Addend Setting	84

1 Introduction

Modular addition with one addend fixed,

$$f_{\text{Constant}_{\text{Key}}}(X) = X + \text{Constant}_{\text{Key}} \pmod{2^n},$$

appears naturally inside ARX designs and key-injection steps such as the RC5 half-round setting [5, 6]. Once the addend is frozen, the relevant differential and linear quantities are no longer average two-input properties; they depend on the actual bit pattern of $\text{Constant}_{\text{Key}}$. This is the cryptanalytic subproblem that appears when the second operand is fixed by a subkey, round constant, or surrounding state.

The goal of this paper is to make that fixed-addend object fully explicit in a modern self-contained form. On the differential side, we derive an exact 4-state carry-pair recurrence. On the linear side, we derive an exact 2-state carry-transfer recurrence. In both cases, the resulting quantities admit explicit $O(n)$ evaluation algorithms once the visible masks and the addend are fixed, and these algorithms are checked exhaustively on small word sizes against brute force.

Later papers repeatedly point back to a one-addend-fixed branch and, on the differential side, to a linear-time procedure [7, 9, 11]. The main text of this draft focuses on the exact mathematics; the bibliographic discussion is moved to the final appendix.

The rest of the paper is organized as follows. Parts I and II contain the core theorem-and-algorithm package and can be read independently of the bibliographic discussion. Part I develops the exact differential theory of the fixed-addend map. Part II develops the exact linear theory. Parts III and IV collect supplementary technical context, extensions, and downstream views. The appendices collect auxiliary matrices, verifier material, and the literature notes.

A reference implementation accompanying this draft is available online [4].

Part I: Exact Differential Theory of the Fixed-Addend Case

2 Exact Differential Core

2.1 Differential Problem, Carry-Pair State, and Roadmap

We now turn directly to the differential object

$$f_{\text{Constant}_{\text{Key}}}(X) = X + \text{Constant}_{\text{Key}} \pmod{2^n},$$

with one variable input and one frozen addend. For fixed $\text{Constant}_{\text{Key}}$ and fixed visible difference pair (u, v) , the task is to compute exactly

$$\Pr_x [f_{\text{Constant}_{\text{Key}}}(x \oplus u) = f_{\text{Constant}_{\text{Key}}}(x) \oplus v].$$

Because one operand is frozen, this is not an averaged property of binary addition. The answer depends on the actual bit pattern of $\text{Constant}_{\text{Key}}$, and the dependence is mediated entirely by carry propagation.

The organizing principle is simple but decisive. Modular addition is locally affine once the incoming carry is known. In the differential setting, however, two executions are compared in parallel, on inputs x and $x \oplus u$. The future suffix therefore depends on two incoming carries rather than one. Once that pair is identified, the full probability becomes an exact low-to-high prefix recurrence.

Equivalently, after the visible affine part is peeled away, the only nonlinear hidden term that remains is

$$\text{Carry}_{\text{Constant}_{\text{Key}}}(x \oplus u) \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x),$$

so the real question is not whether carries matter, but how much of the two carry histories the suffix still needs. Since each execution remembers the processed prefix only through its current carry bit, the joint future depends only on the current pair (c_i, c'_i) . This is the conceptual reason the exact differential problem collapses to a 4-state system.

Part I therefore proceeds in three steps. First, it derives the exact carry-pair recurrence. Second, it compresses that recurrence into equivalent scalar and matrix forms. Third, it turns those forms into an explicit $O(n)$ evaluator for the fixed-addend differential probability.

2.2 Fixed-Addend Differential Probability

We now state the differential quantity to be computed. Fix a visible input difference u and a visible output difference v . The problem is to determine exactly which inputs x satisfy the relation

$$f_{\text{Constant}_{\text{Key}}}(x \oplus u) = f_{\text{Constant}_{\text{Key}}}(x) \oplus v,$$

and hence to compute the corresponding probability under uniform x . Unlike XOR, the output difference is not determined by u alone: the lower bits of the adder create a carry memory that propagates into higher positions.

The object is the fixed-key addition map

$$f_{\text{Constant}_{\text{Key}}}(x) = x + \text{Constant}_{\text{Key}} \pmod{2^n}.$$

To keep the notation aligned with the global convention, we write

$$f_{\text{Constant}_{\text{Key}}}(x) = x \boxplus \text{Constant}_{\text{Key}} = x + \text{Constant}_{\text{Key}} \pmod{2^n}.$$

For $u, v \in \mathbb{F}_2^n$, define the exact fixed-addend XOR differential probability

$$\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) := \Pr_x [f_{\text{Constant}_{\text{Key}}}(x \oplus u) = f_{\text{Constant}_{\text{Key}}}(x) \oplus v], \quad (1)$$

where x is uniform in $\mathbb{Z}_{2^n}$. This is the exact success probability of the visible differential pair (u, v) under the frozen adder. The difficulty is that the map is not bitwise independent: carries created in the low positions feed directly into the higher positions.

To see what must be remembered from the processed low bits, one compares two executions of the same unary map under the same frozen addend. The hidden term is

$$\text{Carry}_{\text{Constant}_{\text{Key}}}(x \oplus u) \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x),$$

so the task is no longer to guess *whether* carries matter, but to determine how much of those two carry histories must be kept in order to continue the computation exactly.

Write

$$x = \sum_{i=0}^{n-1} x_i 2^i, \quad \text{Constant}_{\text{Key}} = \sum_{i=0}^{n-1} a_i 2^i, \quad y = f_{\text{Constant}_{\text{Key}}}(x) = \sum_{i=0}^{n-1} y_i 2^i.$$

For the first execution one has

$$y_i = x_i \oplus a_i \oplus c_i, \tag{2}$$

$$c_0 = 0, \tag{3}$$

$$c_{i+1} = \text{Maj}(x_i, a_i, c_i) = (x_i \wedge a_i) \oplus (x_i \wedge c_i) \oplus (a_i \wedge c_i), \tag{4}$$

and for the second execution on input $x \oplus u$,

$$y'_i = x_i \oplus u_i \oplus a_i \oplus c'_i, \tag{5}$$

$$c'_0 = 0, \tag{6}$$

$$c'_{i+1} = \text{Maj}(x_i \oplus u_i, a_i, c'_i). \tag{7}$$

Hence the output difference satisfies

$$v_i = y_i \oplus y'_i = u_i \oplus c_i \oplus c'_i. \tag{8}$$

This identity is the central fixed-addend differential identity: the visible differential bit is controlled by the parity of two carry chains. Just as importantly, the next step of the computation uses the two carries separately through

$$c_{i+1} = \text{Maj}(x_i, a_i, c_i), \quad c'_{i+1} = \text{Maj}(x_i \oplus u_i, a_i, c'_i).$$

So one carry bit is too small a summary for exact differential continuation, because the two executions do not in general share the same carry. But the full processed prefix is also too large a summary, because once c_i and c'_i are fixed, the future updates no longer need the detailed lower history. The exact hidden object therefore sits strictly in between: the current carry pair (c_i, c'_i) . In RC5 notation this is exactly the local form of the fixed subkey additions $L_1 = L_0 + S_0$, $R_1 = R_0 + S_1$, and more generally $Z \mapsto Z + S_i$ inside the half-rounds. Kaliski and Yin study such steps by averaging the induced differential behaviour over the round key; the present analysis freezes the addend and asks how its actual bit pattern changes the exact probability [5].

2.3 Prefix Events and the Carry-Pair Parity Law

It remains to identify the exact prefix information retained by the suffix. If bits $0, 1, \dots, i-1$ already satisfy the target differential relation, then equation (8) shows that the visible slice at level i determines only the parity of the two carries, not the full carry pair. Thus each level leaves exactly two carry pairs compatible with the visible data, and the recurrence need only track how the surviving mass is split between them.

Define

$$d_i := u_i \oplus v_i.$$

For $0 \leq i \leq n$, let E_i be the event that the differential relation holds on bits $0, 1, \dots, i-1$, and put

$$\mathcal{A}_i := \Pr(E_i).$$

Then

$$\mathcal{A}_0 = 1, \quad \text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) = \mathcal{A}_n.$$

Lemma 1 (carry-pair parity law). *Under E_i , the carries at level i satisfy*

$$c_i \oplus c'_i = d_i = u_i \oplus v_i. \quad (9)$$

Consequently,

$$\begin{aligned} d_i = 0 &\implies (c_i, c'_i) \in \{(0, 0), (1, 1)\}, \\ d_i = 1 &\implies (c_i, c'_i) \in \{(0, 1), (1, 0)\}. \end{aligned}$$

Proof. Equation (8) rearranges to $c_i \oplus c'_i = u_i \oplus v_i$. $\square$

Corollary 1 (least significant bit condition). *If $u_0 \neq v_0$, then $\text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) = 0$.*

Proof. At level 0, both carries are zero, so equation (9) yields $u_0 \oplus v_0 = 0$. $\square$

By Lemma 1, only two exact states remain compatible at each level. It is therefore enough to track a single conditional mass. Define

$$\delta_i := \Pr(c_i = 1 \mid E_i). \quad (10)$$

Then under E_i one has

$$\Pr((c_i, c'_i) = (0, d_i) \mid E_i) = 1 - \delta_i, \quad \Pr((c_i, c'_i) = (1, 1 \oplus d_i) \mid E_i) = \delta_i. \quad (11)$$

Equivalently,

$$\mathcal{A}_i(1 - \delta_i) = \Pr(E_i \wedge (c_i, c'_i) = (0, d_i)), \quad \mathcal{A}_i\delta_i = \Pr(E_i \wedge (c_i, c'_i) = (1, 1 \oplus d_i)). \quad (12)$$

The first quantity $\mathcal{A}_i$ is the surviving prefix mass, and the second quantity δ_i records how that mass is split across the two parity-compatible carry pairs. Thus the whole fixed-addend differential problem has been reduced to two quantities:

$$\mathcal{A}_i \quad \text{and} \quad \delta_i.$$

2.4 One-Bit Analysis from Level $i-1$ to Level i

The recurrence follows from an exhaustive one-bit case analysis. At level i , the update is determined by the previous visible bits (u_{i-1}, v_{i-1}) , the parity requested at the next level $u_i \oplus v_i$, the addend bit a_{i-1} , and the fresh random bit x_{i-1} .

For $i \geq 1$, set

$$S_i := (u_{i-1}, v_{i-1}, u_i \oplus v_i).$$

The first two bits indicate which parity slice is visible at level $i-1$, and the third bit indicates which parity slice is requested at level i . Thus S_i is the complete visible local type, and the full local input for the recurrence is the pair (S_i, a_{i-1}) .

Let $j = i-1$ and $a = a_j$. The random bit for this local step is x_j . The exact local update is

$$(c_i, c'_i) = (\text{Maj}(x_j, a, c_{i-1}), \text{Maj}(x_j \oplus u_j, a, c'_{i-1})), \quad (13)$$

subject to the visible parity constraint

$$c_i \oplus c'_i = u_i \oplus v_i. \quad (14)$$

Direct substitution into equations (13)–(14) gives the complete one-bit branch table.

Proposition 1 (exact one-bit surviving branches). *For each local type S_i and addend bit a_{i-1} , the surviving values of x_{i-1} and the next carry pair (c_i, c'_i) are as follows.*

(B1) If $S_i = 000$, the live states at level $i - 1$ are 00 and 11.

- If $a_{i-1} = 0$, then $00 \xrightarrow{x=0,1} 00$ and $11 \xrightarrow{x=0} 00$, $11 \xrightarrow{x=1} 11$.
- If $a_{i-1} = 1$, then $00 \xrightarrow{x=0} 00$, $00 \xrightarrow{x=1} 11$, and $11 \xrightarrow{x=0,1} 11$.

(B2) If $S_i = 001$, no branch survives.

(B3) If $S_i = 010$, the live states are 01 and 10.

- If $a_{i-1} = 0$, then $01 \xrightarrow{x=0} 00$ and $10 \xrightarrow{x=0} 00$.
- If $a_{i-1} = 1$, then $01 \xrightarrow{x=1} 11$ and $10 \xrightarrow{x=1} 11$.

(B4) If $S_i = 011$, the live states are 01 and 10.

- If $a_{i-1} = 0$, then $01 \xrightarrow{x=1} 01$ and $10 \xrightarrow{x=1} 10$.
- If $a_{i-1} = 1$, then $01 \xrightarrow{x=0} 01$ and $10 \xrightarrow{x=0} 10$.

(B5) If $S_i = 100$, the live states are 01 and 10.

- If $a_{i-1} = 0$, then $01 \xrightarrow{x=1} 00$ and $10 \xrightarrow{x=0} 00$.
- If $a_{i-1} = 1$, then $01 \xrightarrow{x=1} 11$ and $10 \xrightarrow{x=0} 11$.

(B6) If $S_i = 101$, the live states are 01 and 10.

- If $a_{i-1} = 0$, then $01 \xrightarrow{x=0} 01$ and $10 \xrightarrow{x=1} 10$.
- If $a_{i-1} = 1$, then $01 \xrightarrow{x=0} 01$ and $10 \xrightarrow{x=1} 10$.

(B7) If $S_i = 110$, the live states are 00 and 11.

- If $a_{i-1} = 0$, then $00 \xrightarrow{x=0,1} 00$ and the state 11 dies.
- If $a_{i-1} = 1$, then the state 00 dies and $11 \xrightarrow{x=0,1} 11$.

(B8) If $S_i = 111$, the live states are 00 and 11.

- If $a_{i-1} = 0$, then 00 dies and $11 \xrightarrow{x=0} 01$, $11 \xrightarrow{x=1} 10$.
- If $a_{i-1} = 1$, then $00 \xrightarrow{x=0} 01$, $00 \xrightarrow{x=1} 10$, and 11 dies.

Proof. Each line follows by direct substitution into equations (4)–(7). For example, in type 000 the previous carries satisfy $(c_{i-1}, c'_{i-1}) \in \{00, 11\}$ and $u_{i-1} = 0$. If $a_{i-1} = 0$ and the previous state is 11, then

$$(c_i, c'_i) = (\text{Maj}(x_{i-1}, 0, 1), \text{Maj}(x_{i-1}, 0, 1)) = (x_{i-1}, x_{i-1}),$$

which gives 00 for $x_{i-1} = 0$ and 11 for $x_{i-1} = 1$. The other lines are obtained in the same way. $\square$

2.5 Exact Prefix Recurrence for the Differential Probability

Proposition 1 reduces the global law to the evolution of two scalars: the surviving prefix mass $\mathcal{A}_i$ and the conditional first-carry mass δ_i .

Set the sentinel values

$$u_{-1} = v_{-1} = a_{-1} = 0, \quad \delta_{-1} = 0.$$

Lemma 2 (local transport of surviving prefix mass). *Fix $i \geq 1$. Define*

$$p_i^{(0)} := \Pr((c_i, c'_i) = (0, d_i) \mid E_{i-1}), \quad p_i^{(1)} := \Pr((c_i, c'_i) = (1, 1 \oplus d_i) \mid E_{i-1}).$$

Then

$$\Pr(E_i \wedge (c_i, c'_i) = (0, d_i)) = \mathcal{A}_{i-1} p_i^{(0)}, \quad \Pr(E_i \wedge (c_i, c'_i) = (1, 1 \oplus d_i)) = \mathcal{A}_{i-1} p_i^{(1)}. \quad (15)$$

Consequently,

$$\mathcal{A}_i = \mathcal{A}_{i-1} (p_i^{(0)} + p_i^{(1)}), \quad \mathcal{A}_i (1 - \delta_i) = \mathcal{A}_{i-1} p_i^{(0)}, \quad \mathcal{A}_i \delta_i = \mathcal{A}_{i-1} p_i^{(1)}. \quad (16)$$

Proof. By Lemma 1, under E_i the only surviving carry pairs at level i are $(0, d_i)$ and $(1, 1 \oplus d_i)$. Therefore

$$E_i = \left(E_i \wedge (c_i, c'_i) = (0, d_i) \right) \dot{\cup} \left(E_i \wedge (c_i, c'_i) = (1, 1 \oplus d_i) \right),$$

where $\dot{\cup}$ denotes disjoint union. Multiplying the conditional probabilities $p_i^{(0)}$ and $p_i^{(1)}$ by $\Pr(E_{i-1}) = \mathcal{A}_{i-1}$ gives equation (15). Summing the two identities yields the formula for $\mathcal{A}_i$, and comparison with equation (12) gives the remaining two identities. $\square$

Theorem 1 (recursive determination of fixed-addend differential probability). *For each i there exists a local survival factor ϕ_i such that*

$$\mathcal{A}_{i+1} = \mathcal{A}_i \phi_i. \quad (17)$$

Hence

$$\text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) = \prod_{i=0}^{n-1} \phi_i. \quad (18)$$

Moreover, the conditional carry parameter evolves by

$$\delta_i = \begin{cases} \frac{a_{i-1} + \delta_{i-1}}{2}, & S_i = 000, \\ 0, & S_i = 001, \\ a_{i-1}, & S_i \in \{010, 100, 110\}, \\ \delta_{i-1}, & S_i \in \{011, 101\}, \\ \frac{1}{2}, & S_i = 111, \end{cases} \quad (19)$$

and

$$\phi_i = \begin{cases} 1, & S_i = 000, \\ 0, & S_i = 001, \\ \frac{1}{2}, & S_i \in \{010, 011, 100, 101\}, \\ 1 - (a_{i-1} + \delta_{i-1} - 2a_{i-1}\delta_{i-1}), & S_i = 110, \\ a_{i-1} + \delta_{i-1} - 2a_{i-1}\delta_{i-1}, & S_i = 111. \end{cases} \quad (20)$$

Proof. The initial step $i = 0$ is immediate from $c_0 = c'_0 = 0$, which yields the least-significant-bit condition through the sentinel type $S_0 = (0, 0, u_0 \oplus v_0)$. For every later step $i \geq 1$, condition on the preceding survival event. By equation (11), under E_{i-1} the current carry pair is

$$(c_{i-1}, c'_{i-1}) = (0, d_{i-1}) \quad \text{with probability } 1 - \delta_{i-1},$$

$$(c_{i-1}, c'_{i-1}) = (1, 1 \oplus d_{i-1}) \quad \text{with probability } \delta_{i-1}.$$

For the step indexed by S_i , let $p_i^{(0)}$ and $p_i^{(1)}$ be the quantities of Lemma 2. Then

$$\phi_i = p_i^{(0)} + p_i^{(1)}, \quad (21)$$

by equation (16), and whenever $\phi_i \neq 0$ one has

$$\delta_i = \frac{p_i^{(1)}}{p_i^{(0)} + p_i^{(1)}}$$

by dividing the last identity of equation (16) by the first one. If $\phi_i = 0$, then the total surviving mass already vanishes and the value of δ_i becomes immaterial for the final probability; the formula in equation (19) fixes a convenient representative value in those zero-mass cases. It therefore suffices to compute $(p_i^{(0)}, p_i^{(1)})$ from the local branch table.

If $S_i = 000$ and $a_{i-1} = 0$, the state 00 survives with both values of x_{i-1} and always lands in 00,

whereas the state 11 splits evenly between 00 and 11. Hence

$$p_i^{(0)} = (1 - \delta_{i-1}) + \frac{1}{2}\delta_{i-1}, \quad p_i^{(1)} = \frac{1}{2}\delta_{i-1}.$$

Therefore

$$\phi_i = 1, \quad \delta_i = \frac{\frac{1}{2}\delta_{i-1}}{1} = \frac{\delta_{i-1}}{2}.$$

If $S_i = 000$ and $a_{i-1} = 1$, the state 00 splits evenly between 00 and 11, whereas the state 11 survives twice and always lands in 11. Thus

$$p_i^{(0)} = \frac{1}{2}(1 - \delta_{i-1}), \quad p_i^{(1)} = \frac{1}{2}(1 - \delta_{i-1}) + \delta_{i-1},$$

so

$$\phi_i = 1, \quad \delta_i = \frac{\frac{1}{2}(1 - \delta_{i-1}) + \delta_{i-1}}{1} = \frac{1 + \delta_{i-1}}{2} = \frac{a_{i-1} + \delta_{i-1}}{2}.$$

If $S_i = 001$, Proposition 1 gives no surviving branch, and therefore

$$p_i^{(0)} = p_i^{(1)} = 0, \quad \phi_i = 0.$$

If $S_i \in \{010, 100\}$ and $a_{i-1} = 0$, each live state survives for exactly one value of x_{i-1} and lands in 00. Hence

$$p_i^{(0)} = \frac{1}{2}(1 - \delta_{i-1}) + \frac{1}{2}\delta_{i-1} = \frac{1}{2}, \quad p_i^{(1)} = 0,$$

which gives

$$\phi_i = \frac{1}{2}, \quad \delta_i = 0 = a_{i-1}.$$

If $S_i \in \{010, 100\}$ and $a_{i-1} = 1$, the same counting gives

$$p_i^{(0)} = 0, \quad p_i^{(1)} = \frac{1}{2}(1 - \delta_{i-1}) + \frac{1}{2}\delta_{i-1} = \frac{1}{2},$$

and therefore

$$\phi_i = \frac{1}{2}, \quad \delta_i = 1 = a_{i-1}.$$

If $S_i \in \{011, 101\}$, each live state again survives for exactly one value of x_{i-1} , but now the odd slice is preserved. Hence

$$p_i^{(0)} = \frac{1}{2}(1 - \delta_{i-1}), \quad p_i^{(1)} = \frac{1}{2}\delta_{i-1},$$

so

$$\phi_i = \frac{1}{2}, \quad \delta_i = \frac{\frac{1}{2}\delta_{i-1}}{\frac{1}{2}} = \delta_{i-1}.$$

If $S_i = 110$ and $a_{i-1} = 0$, only the state 00 survives and it survives twice, so

$$p_i^{(0)} = 1 - \delta_{i-1}, \quad p_i^{(1)} = 0.$$

If $S_i = 110$ and $a_{i-1} = 1$, only the state 11 survives and it survives twice, so

$$p_i^{(0)} = 0, \quad p_i^{(1)} = \delta_{i-1}.$$

Combining both cases,

$$\phi_i = \begin{cases} 1 - \delta_{i-1}, & a_{i-1} = 0, \\ \delta_{i-1}, & a_{i-1} = 1, \end{cases} \quad \delta_i = a_{i-1}.$$

This is exactly the $S_i = 110$ line of equations (19)–(20).

If $S_i = 111$ and $a_{i-1} = 0$, only the state 11 survives, and its two surviving branches split evenly between 01 and 10. Therefore

$$p_i^{(0)} = \frac{1}{2}\delta_{i-1}, \quad p_i^{(1)} = \frac{1}{2}\delta_{i-1},$$

so

$$\phi_i = \delta_{i-1}, \quad \delta_i = \frac{1}{2} \quad \text{whenever } \phi_i \neq 0.$$

If $S_i = 111$ and $a_{i-1} = 1$, the same reasoning with the state 00 gives

$$p_i^{(0)} = \frac{1}{2}(1 - \delta_{i-1}), \quad p_i^{(1)} = \frac{1}{2}(1 - \delta_{i-1}),$$

and hence

$$\phi_i = 1 - \delta_{i-1}, \quad \delta_i = \frac{1}{2} \quad \text{whenever } \phi_i \neq 0.$$

This is exactly the $S_i = 111$ line of equations (19)–(20).

This proves equations (19) and (20). Equation (17) is the definition of the local survival factor, and equation (18) follows by multiplying these factors along the scan. $\square$

3 Consequences, Compression, and Differential Algorithms

3.1 Immediate Consequences of the Local Recurrence

The recurrence yields the following immediate consequences.

Corollary 2 (forbidden pattern). *If $S_i = (0, 0, 1)$ for some $i \geq 1$, then $\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) = 0$.*

Proof. Equation (20) gives $\phi_i = 0$. $\square$

Corollary 3 (half steps). *If $S_i \in \{010, 011, 100, 101\}$, then $\phi_i = 1/2$. Therefore a run of t such local steps contributes exactly 2^{-t} to the total differential probability.*

Proof. This is immediate from equation (20). $\square$

Remark 1. These two consequences already explain the most basic global features of the differential distribution: abrupt zeros caused by forbidden local patterns, and short dyadic ladders $1, 1/2, 1/4, \dots$ caused by repeated half steps.

3.2 Consecutive-One Blocks and the Correct Run Law

The stronger claim that any nonzero block $u_i = v_i = 1$ forces the whole addend block $a_m, \dots, a_r$ to be constant is false when the block reaches the most significant position; Appendix C gives the counterexample $n = 2$, $\text{Constant}_{\text{Key}} = 10_2$, $u = v = 11_2$. The correct statement is the following.

Theorem 2 (correct fixed-addend run law). *Let $0 \leq m \leq r \leq n - 1$ and assume*

$$u_i = v_i = 1 \quad (m \leq i \leq r),$$

with $\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) \neq 0$.

(a) *If $r - m \geq 1$, then the interior addend bits satisfy*

$$a_m = a_{m+1} = \dots = a_{r-1}. \quad (22)$$

(b) *If moreover $r + 1 < n$, then the exit bit is forced by*

$$a_r = a_m \oplus (u_{r+1} \oplus v_{r+1}). \quad (23)$$

Proof. For every interior index i with $m < i \leq r$, one has $S_i = 110$. Since $\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) \neq 0$, every local factor on the block is nonzero. In particular, for each such i ,

$$\delta_i = a_{i-1}$$

by equation (19). If moreover $m < i < r$, then the next interior type is also $S_{i+1} = 110$, so equation (20) gives

$$\phi_{i+1} = \begin{cases} 1 - \delta_i, & a_i = 0, \\ \delta_i, & a_i = 1. \end{cases}$$

Since $\delta_i = a_{i-1} \in \{0, 1\}$ and $\phi_{i+1} \neq 0$, this forces

$$a_i = \delta_i = a_{i-1}.$$

Iterating over $i = m + 1, \dots, r - 1$ yields

$$a_m = a_{m+1} = \dots = a_{r-1}.$$

This proves equation (22).

If $r + 1 < n$, then the next local type is $S_{r+1} = (1, 1, u_{r+1} \oplus v_{r+1})$. If $u_{r+1} \oplus v_{r+1} = 0$, this is again type 110, and the nonzero condition forces $a_r = \delta_r = a_m$. If $u_{r+1} \oplus v_{r+1} = 1$, this is type 111, and the complementary mass must survive, so $a_r = 1 \oplus \delta_r = 1 \oplus a_m$. Together these give equation (23). $\square$

3.3 Exact Finite Recursion and the Four-State Form

The one-bit branch law of Proposition 1 can be encoded directly as a four-state transfer system. Fix the state order

$$00, 01, 10, 11.$$

For a fixed bit position and local data (a_i, u_i, v_i) , define the integer transition operator

$$M_{a_i, u_i, v_i}^D[r, s] := \#\{x \in \{0, 1\} : v_i = u_i \oplus r_0 \oplus r_1, s = (\text{Maj}(x, a_i, r_0), \text{Maj}(x \oplus u_i, a_i, r_1))\}, \quad (24)$$

where $r = (r_0, r_1)$ and $s = (s_0, s_1)$ denote exact carry-pair states.

Theorem 3 (exact four-state count). *Let $e_{00} = (1, 0, 0, 0)$ encode the initial state $(c_0, c'_0) = (0, 0)$ and let $\mathbf{1} = (1, 1, 1, 1)^\top$. Then*

$$2^n \text{DP}_{\text{ConstantKey}}^\boxplus(u \rightarrow v) = e_{00} \left(\prod_{i=0}^{n-1} M_{a_i, u_i, v_i}^D \right) \mathbf{1}. \quad (25)$$

Hence

$$\text{DP}_{\text{ConstantKey}}^\boxplus(u \rightarrow v) = 2^{-n} e_{00} \left(\prod_{i=0}^{n-1} M_{a_i, u_i, v_i}^D \right) \mathbf{1}.$$

Proof. By definition,

$$M_{a_i, u_i, v_i}^D[r, s] = \#\{x_i \in \{0, 1\} : \text{the local bit } x_i \text{ maps } r \text{ to } s \text{ and satisfies the visible constraint at } i\}.$$

Hence

$$e_{00} \left(\prod_{i=0}^{n-1} M_{a_i, u_i, v_i}^D \right) \mathbf{1} = \sum_{s_1, \dots, s_n} \prod_{i=0}^{n-1} M_{a_i, u_i, v_i}^D[s_i, s_{i+1}], \quad s_0 = 00,$$

which is exactly the number of assignments of $x_0, \dots, x_{n-1}$ whose successive carry pairs are

$$00 = s_0, s_1, \dots, s_n$$

and whose visible output difference equals v . Therefore the matrix product counts the surviving inputs x , and division by 2^n yields the probability. $\square$

Proposition 2 (normalized form of the four-state prefix count). *Let*

$$N_i = (N_i^{00}, N_i^{01}, N_i^{10}, N_i^{11})$$

be the exact prefix-count vector after processing bits $0, \dots, i-1$. Then

$$N_{i+1} = N_i M_{a_i, u_i, v_i}^D, \quad N_0 = e_{00}, \quad \mathcal{A}_i = 2^{-i} N_i \mathbf{1}.$$

Moreover, with $d_i = u_i \oplus v_i$,

$$N_i = \begin{cases} 2^i \mathcal{A}_i (1 - \delta_i, 0, 0, \delta_i), & d_i = 0, \\ 2^i \mathcal{A}_i (0, 1 - \delta_i, \delta_i, 0), & d_i = 1. \end{cases} \quad (26)$$

Proof. The recursion for N_i is the standard prefix-count update induced by the local transition matrices. Since each processed bit contributes a factor 2 to the sample-space size, the surviving prefix probability is

$$\mathcal{A}_i = \frac{N_i \mathbf{1}}{2^i}.$$

By Lemma 1, the vector N_i is supported on the parity slice determined by d_i . Equation (11) then gives the normalized mass split on that slice, and multiplying by the total surviving prefix count $2^i \mathcal{A}_i$ yields equation (26). $\square$

3.4 Worked Differential Micro-Example

Take $n = 6$, constant $\text{Constant}_{\text{Key}} = 101100_2$, input difference $u = 001110_2$, and target output difference $v = 001110_2$. Then

$$(a_0, a_1, a_2, a_3, a_4, a_5) = (0, 0, 1, 1, 0, 1),$$

and

$$(u_0, u_1, u_2, u_3, u_4, u_5) = (0, 1, 1, 1, 0, 0) = (v_0, v_1, v_2, v_3, v_4, v_5).$$

Hence

$$S_2 = (u_1, v_1, u_2 \oplus v_2) = (1, 1, 0) = 110,$$

$$S_3 = (u_2, v_2, u_3 \oplus v_3) = (1, 1, 0) = 110,$$

and

$$S_4 = (u_3, v_3, u_4 \oplus v_4) = (1, 1, 0) = 110.$$

Assume that $\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) \neq 0$. The first interior step is $S_2 = 110$, so equation (19) gives

$$\delta_2 = a_1 = 0.$$

At the next step one still has $S_3 = 110$. Therefore equation (20) yields

$$\phi_3 = \begin{cases} 1 - \delta_2, & a_2 = 0, \\ \delta_2, & a_2 = 1. \end{cases}$$

Since $a_2 = 1$ and $\delta_2 = 0$, this gives

$$\phi_3 = 0.$$

But then

$$\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) = \prod_{i=0}^5 \phi_i = 0,$$

contrary to the assumption of nonzero survival. Therefore

$$\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) = 0.$$

The contradiction occurs before any exhaustive evaluation: the local type sequence and the addend bits already force the differential to die at the third active step.

3.5 Algorithmic Consequence: a Prefix Dynamic Program

Proposition 2 gives the unnormalized recursion

$$N_{i+1} = N_i M_{a_i, u_i, v_i}^D, \quad N_0 = e_{00}, \quad \text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) = 2^{-n} N_n \mathbf{1},$$

while Theorem 1 gives its normalized quotient through $(\mathcal{A}_i, \delta_i)$. Algorithm 1 records that normalized form explicitly.

Algorithm 1 Exact fixed-addend differential probability in linear time (expanded scalar form)

Require: n -bit constant ConstantKey , input difference u , output difference v

Ensure: Exact value of $\text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v)$

```

1: Initialize the normalized prefix mass  $\mathcal{A} \leftarrow 1$ 
2: Initialize the scalar carry summary  $\delta \leftarrow 0$ 
3: if  $u_0 \neq v_0$  then
4:   return 0
5: end if
6: for  $i = 1$  to  $n - 1$  do
7:   Set  $\varepsilon_i \leftarrow u_i \oplus v_i$ 
8:   Read the previous visible bits  $u_{i-1}, v_{i-1}$  and the addend bit  $a_{i-1}$ 
9:   Initialize  $\phi_i \leftarrow 0$  and  $\delta_{\text{next}} \leftarrow 0$ 
10:  if  $(u_{i-1}, v_{i-1}, \varepsilon_i) = (0, 0, 0)$  then
11:     $\phi_i \leftarrow 1$ 
12:     $\delta_{\text{next}} \leftarrow \frac{a_{i-1} + \delta}{2}$ 
13:  else if  $(u_{i-1}, v_{i-1}, \varepsilon_i) = (0, 0, 1)$  then
14:    return 0
15:  else if  $(u_{i-1}, v_{i-1}, \varepsilon_i) \in \{(0, 1, 0), (1, 0, 0)\}$  then
16:     $\phi_i \leftarrow \frac{1}{2}$ 
17:     $\delta_{\text{next}} \leftarrow a_{i-1}$ 
18:  else if  $(u_{i-1}, v_{i-1}, \varepsilon_i) \in \{(0, 1, 1), (1, 0, 1)\}$  then
19:     $\phi_i \leftarrow \frac{1}{2}$ 
20:     $\delta_{\text{next}} \leftarrow \delta$ 
21:  else if  $(u_{i-1}, v_{i-1}, \varepsilon_i) = (1, 1, 0)$  then
22:    if  $a_{i-1} = 1$  then
23:       $\phi_i \leftarrow \delta$ 
24:    else
25:       $\phi_i \leftarrow 1 - \delta$ 
26:    end if
27:     $\delta_{\text{next}} \leftarrow a_{i-1}$ 
28:  else
29:    Here necessarily  $(u_{i-1}, v_{i-1}, \varepsilon_i) = (1, 1, 1)$ .
30:    if  $a_{i-1} = 1$  then
31:       $\phi_i \leftarrow 1 - \delta$ 
32:    else
33:       $\phi_i \leftarrow \delta$ 
34:    end if
35:     $\delta_{\text{next}} \leftarrow \frac{1}{2}$ 
36:  end if
37:  Update the prefix mass  $\mathcal{A} \leftarrow \mathcal{A} \phi_i$ 
38:  if  $\mathcal{A} = 0$  then
39:    return 0
40:  end if
41:  Update  $\delta \leftarrow \delta_{\text{next}}$ 
42: end for
```

43: **return** $\mathcal{A}$

3.6 Core Conclusion of Part I

Corollary 4 (exact linear-time differential evaluation). *For every fixed $\text{Constant}_{\text{Key}} \in \mathbb{Z}_{2^n}$ and every $u, v \in \mathbb{F}_2^n$, Algorithm 1 computes $\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v)$ exactly in time $O(n)$ using $O(1)$ working memory. Equivalently, the exact 4-state count update computes the same value in one low-to-high scan.*

Proof. Theorem 1 gives correctness. Each iteration reads the local data, selects one of finitely many cases, and updates $(\mathcal{A}, \delta)$ in constant time. The algorithm stores only $\mathcal{A}$, δ , and the current local bits, so the working memory is $O(1)$. Since the scan touches each bit once, the runtime is $O(n)$. The equivalent 4-state statement follows from Theorem 3. $\square$

Part II: Exact Linear Theory of the Fixed-Addend Case

4 Exact Linear Core

4.1 Fixed-Addend Walsh Correlation and Carry State

Part II develops the linear analogue of Part I for the same fixed-addend map. The observable is now a Walsh correlation rather than a survival probability, so the hidden state changes accordingly.

Only one execution is observed on the linear side, so the hidden memory will not be a carry pair. However, one undifferentiated running total is still too small an exact state: prefixes ending with carry 0 and prefixes ending with carry 1 do not acquire the same future phases, so merging them destroys exact cancellation information. The exact state is therefore a pair of conditioned signed masses, one for each current carry value.

This part follows the same overall pattern as Part I. First, it removes the visible affine contribution and isolates the carry-dependent part of the phase. Second, it identifies the exact two-component prefix state. Third, it turns the resulting one-bit signed transfer law into an explicit $O(n)$ evaluator for fixed-addend Walsh correlations.

We again freeze the key material and study

$$f_{\text{Constant}_{\text{Key}}}(x) = x + \text{Constant}_{\text{Key}} \pmod{2^n}.$$

For masks $\alpha, \beta \in \mathbb{F}_2^n$, define the exact fixed-addend correlation

$$\text{CORRELATION}(\alpha \rightarrow \beta) = 2^{-n} \sum_{x \in \mathbb{F}_2^n} (-1)^{\langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle}. \quad (27)$$

This is the exact fixed-addend Walsh correlation. In ARX language, it is the signed bias induced by a frozen-addend addition step under the visible mask pair (α, β) ; in particular, the value can depend sharply on the bit pattern of $\text{Constant}_{\text{Key}}$ [5].

4.2 Reduction to the Carry Word

The first structural step is to separate the visible affine part of the phase from the carry-dependent part.

Let c_i be the carry into bit i in the addition $x \boxplus \text{Constant}_{\text{Key}}$, so that

$$c_0 = 0, \quad c_{i+1} = \text{Maj}(x_i, a_i, c_i),$$

and

$$f_{\text{Constant}_{\text{Key}}}(x) = x \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x).$$

Therefore

$$\langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle = \langle \alpha \oplus \beta, x \rangle \oplus \langle \beta, \text{Constant}_{\text{Key}} \rangle \oplus \langle \beta, \text{Carry}_{\text{Constant}_{\text{Key}}}(x) \rangle. \quad (28)$$

Hence

$$\text{CORRELATION}(\alpha \rightarrow \beta) = (-1)^{\langle \beta, \text{Constant}_{\text{Key}} \rangle} 2^{-n} \sum_x (-1)^{\langle \alpha \oplus \beta, x \rangle \oplus \langle \beta, \text{Carry}_{\text{Constant}_{\text{Key}}}(x) \rangle}. \quad (29)$$

Equation (29) is the linear counterpart of the differential carry reduction. Once the visible affine contribution has been peeled away, all nontrivial dependence is pushed into one carry chain and not two. A whole carry word still looks global, but under a low-to-high scan that word is generated recursively from the current carry bit alone. So the entire fixed-addend linear problem is governed by a single carry chain, and the next task is to preserve exactly enough signed information to continue that one-carry recursion without collapsing future cancellation.

4.3 Conditioned Partial Walsh Sums

The suffix does not need the full processed prefix. It does, however, need to distinguish prefixes ending with carry 0 from prefixes ending with carry 1, because those two classes acquire different phases under the next bit. The minimal exact online state is therefore a pair of conditioned Walsh sums, one for each current carry value.

This is the precise linear analogue of the carry-pair reduction in Part I. There, two executions force a pair state (c_i, c'_i) . Here, one execution leaves a single carry bit, but the signed observable forces us to remember how much current mass sits on carry 0 and how much sits on carry 1.

Define

$$\mu_i := \alpha_i \oplus \beta_i, \quad \nu_i := \beta_i.$$

Then the phase in equation (27) decomposes bitwise as

$$\langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle = \bigoplus_{j=0}^{n-1} (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j). \quad (30)$$

Accordingly, for each prefix length i , the accumulated prefix phase is

$$\bigoplus_{j=0}^{i-1} (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j).$$

For $0 \leq i \leq n$ and $r \in \{0, 1\}$, define the partial Walsh sums conditioned on the current carry by

$$L_i(r) := \sum_{x_0, \dots, x_{i-1}} (-1)^{\sum_{j=0}^{i-1} (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j)} \mathbf{1}_{\{c_i=r\}}. \quad (31)$$

Then

$$L_0(0) = 1, \quad L_0(1) = 0,$$

and the final correlation is

$$\text{CORRELATION}(\alpha \rightarrow \beta) = 2^{-n} (L_n(0) + L_n(1)). \quad (32)$$

Lemma 3 (prefix-suffix factorization through the current carry). *Fix $i \in \{0, \dots, n\}$. For each $r \in \{0, 1\}$, let the suffix carry chain be defined by*

$$c_i^{(r)} = r, \quad c_{j+1}^{(r)} = \text{Maj}(x_j, a_j, c_j^{(r)}) \quad (i \leq j \leq n-1),$$

and set

$$R_i(r) := \sum_{x_i, \dots, x_{n-1}} (-1)^{\sum_{j=i}^{n-1} (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j^{(r)})}.$$

Then

$$2^n \text{CORRELATION}(\alpha \rightarrow \beta) = \sum_{r \in \{0,1\}} L_i(r) R_i(r). \quad (33)$$

Proof. Using equation (30), the full Walsh sum is

$$\sum_{x_0, \dots, x_{n-1}} (-1)^{\sum_{j=0}^{n-1} (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j)}.$$

Partition this sum according to the current carry value $c_i \in \{0, 1\}$. For each fixed value r , the phase splits as

$$\sum_{j=0}^{n-1} (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j) = \sum_{j=0}^{i-1} (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j) \oplus \sum_{j=i}^{n-1} (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j^{(r)}),$$

because once $c_i = r$ is fixed, the later carries are exactly the suffix chain $c_j^{(r)}$. The prefix contribution is $L_i(r)$ by equation (31), and the suffix contribution is $R_i(r)$ by definition. Summing over $r \in \{0, 1\}$ gives equation (33). Dividing by 2^n recovers the correlation normalization. $\square$

4.4 One-Bit Weighted Recurrence

Fix the addend bit a_i and the local control bits (μ_i, ν_i) , enumerate the two values of x_i , and record the induced sign and next carry. This yields the exact one-bit signed transfer law.

Let

$$P_i := L_i(0), \quad Q_i := L_i(1).$$

For a current carry r and next carry s , define the one-bit weighted transfer

$$T_i(r, s) := \sum_{x \in \{0, 1\} : \text{Maj}(x, a_i, r) = s} (-1)^{\mu_i x \oplus \nu_i a_i \oplus \nu_i r}. \quad (34)$$

Proposition 3 (exact one-bit linear carry transfer). *For every bit position i and every $s \in \{0, 1\}$,*

$$L_{i+1}(s) = \sum_{r \in \{0, 1\}} L_i(r) T_i(r, s). \quad (35)$$

In row-vector form,

$$(L_{i+1}(0), L_{i+1}(1)) = (L_i(0), L_i(1)) M_{a_i, \mu_i, \nu_i}^L, \quad (36)$$

where $M_{a_i, \mu_i, \nu_i}^L = (T_i(r, s))_{r, s \in \{0, 1\}}$.

Proof. Starting from equation (31) at level $i + 1$,

$$\begin{aligned} L_{i+1}(s) &= \sum_{x_0, \dots, x_i} (-1)^{\sum_{j=0}^i (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j)} \mathbf{1}_{\{c_{i+1}=s\}} \\ &= \sum_{r \in \{0, 1\}} \sum_{x_0, \dots, x_{i-1}} (-1)^{\sum_{j=0}^{i-1} (\mu_j x_j \oplus \nu_j a_j \oplus \nu_j c_j)} \mathbf{1}_{\{c_i=r\}} \sum_{\substack{x_i \in \{0, 1\} \\ \text{Maj}(x_i, a_i, r) = s}} (-1)^{\mu_i x_i \oplus \nu_i a_i \oplus \nu_i r}. \end{aligned}$$

The inner sum is exactly $T_i(r, s)$ by equation (34), while the outer prefix sum is exactly $L_i(r)$. Therefore

$$L_{i+1}(s) = \sum_{r \in \{0, 1\}} L_i(r) T_i(r, s),$$

which proves equations (35) and (36). $\square$

Proposition 4 (explicit evaluation of the one-bit linear transfer). *For each bit position i , the local transfer matrix admits the closed forms*

$$M_{0, \mu_i, \nu_i}^L = \begin{pmatrix} 1 + (-1)^{\mu_i} & 0 \\ (-1)^{\nu_i} & (-1)^{\mu_i \oplus \nu_i} \end{pmatrix}, \quad M_{1, \mu_i, \nu_i}^L = \begin{pmatrix} (-1)^{\nu_i} & (-1)^{\mu_i \oplus \nu_i} \\ 0 & 1 + (-1)^{\mu_i} \end{pmatrix}. \quad (37)$$

Proof. Suppose first that $a_i = 0$. Then

$$\text{Maj}(x, 0, 0) = 0, \quad \text{Maj}(x, 0, 1) = x.$$

Therefore

$$T_i(0, 0) = \sum_{x \in \{0, 1\}} (-1)^{\mu_i x} = 1 + (-1)^{\mu_i}, \quad T_i(0, 1) = 0,$$

and, since only $x = 0$ contributes to $s = 0$ while only $x = 1$ contributes to $s = 1$ when the current carry is 1,

$$T_i(1, 0) = (-1)^{\nu_i}, \quad T_i(1, 1) = (-1)^{\mu_i \oplus \nu_i}.$$

This gives the first matrix in equation (37).

Suppose next that $a_i = 1$. Then

$$\text{Maj}(x, 1, 0) = x, \quad \text{Maj}(x, 1, 1) = 1.$$

Hence

$$T_i(0, 0) = (-1)^{\nu_i}, \quad T_i(0, 1) = (-1)^{\mu_i \oplus \nu_i},$$

while

$$T_i(1, 0) = 0, \quad T_i(1, 1) = \sum_{x \in \{0,1\}} (-1)^{\mu_i x} = 1 + (-1)^{\mu_i}.$$

This gives the second matrix in equation (37). $\square$

Specializing equation (37) to the four possible pairs (μ_i, ν_i) yields the explicit local updates. For $a_i = 0$ one has

$$(P_{i+1}, Q_{i+1}) = \begin{cases} (2P_i + Q_i, Q_i), & (\mu_i, \nu_i) = (0, 0), \\ (Q_i, -Q_i), & (\mu_i, \nu_i) = (1, 0), \\ (2P_i - Q_i, -Q_i), & (\mu_i, \nu_i) = (0, 1), \\ (-Q_i, Q_i), & (\mu_i, \nu_i) = (1, 1), \end{cases} \quad (38)$$

and for $a_i = 1$ one has

$$(P_{i+1}, Q_{i+1}) = \begin{cases} (P_i, P_i + 2Q_i), & (\mu_i, \nu_i) = (0, 0), \\ (P_i, -P_i), & (\mu_i, \nu_i) = (1, 0), \\ (-P_i, -P_i + 2Q_i), & (\mu_i, \nu_i) = (0, 1), \\ (-P_i, P_i), & (\mu_i, \nu_i) = (1, 1). \end{cases} \quad (39)$$

4.5 Exact Two-State Matrix Formula

Iterating equation (36) gives the exact transfer product.

Theorem 4 (exact fixed-addend linear transfer formula). *Let $e_0^\top = (1, 0)$ encode the initial carry $c_0 = 0$ and let $\mathbf{1}^\top = (1, 1)^\top$. Then*

$$\text{CORRELATION}(\alpha \rightarrow \beta) = 2^{-n} e_0^\top \left(\prod_{i=0}^{n-1} M_{a_i, \mu_i, \nu_i}^L \right) \mathbf{1}^\top. \quad (40)$$

Proof. Iterating equation (36) gives

$$(L_n(0), L_n(1)) = (L_0(0), L_0(1)) \left(\prod_{i=0}^{n-1} M_{a_i, \mu_i, \nu_i}^L \right) = e_0^\top \left(\prod_{i=0}^{n-1} M_{a_i, \mu_i, \nu_i}^L \right).$$

Using equation (32),

$$\begin{aligned} \text{CORRELATION}(\alpha \rightarrow \beta) &= 2^{-n} (L_n(0) + L_n(1)) \\ &= 2^{-n} (L_n(0), L_n(1)) \mathbf{1}^\top \\ &= 2^{-n} e_0^\top \left(\prod_{i=0}^{n-1} M_{a_i, \mu_i, \nu_i}^L \right) \mathbf{1}^\top. \end{aligned}$$

$\square$

5 Search, Interpretation, and Linear-Time Evaluation

5.1 Algorithmic Consequence: a Signed Prefix Transfer

Algorithm 2 executes equations (35), (38), and (39).

Algorithm 2 Exact fixed-addend linear correlation in linear time (expanded signed-transfer form)**Require:** n -bit constant $\text{Constant}_{\text{Key}}$, input mask α , output mask β **Ensure:** Exact value of $\text{CORRELATION}(\alpha \rightarrow \beta)$

```

1: Initialize the signed carry-conditioned Walsh state  $L \leftarrow (1, 0)$ 
2: for  $i = 0$  to  $n - 1$  do
3:   Read the addend bit  $a_i$ 
4:   Set the local control bits  $\mu_i \leftarrow \alpha_i \oplus \beta_i$  and  $\nu_i \leftarrow \beta_i$ 
5:   Set  $P \leftarrow L(0)$  and  $Q \leftarrow L(1)$ 
6:   if  $(a_i, \mu_i, \nu_i) = (0, 0, 0)$  then
7:      $L \leftarrow (2P + Q, Q)$ 
8:   else if  $(a_i, \mu_i, \nu_i) = (0, 0, 1)$  then
9:      $L \leftarrow (2P - Q, -Q)$ 
10:  else if  $(a_i, \mu_i, \nu_i) = (0, 1, 0)$  then
11:     $L \leftarrow (Q, -Q)$ 
12:  else if  $(a_i, \mu_i, \nu_i) = (0, 1, 1)$  then
13:     $L \leftarrow (-Q, Q)$ 
14:  else if  $(a_i, \mu_i, \nu_i) = (1, 0, 0)$  then
15:     $L \leftarrow (P, P + 2Q)$ 
16:  else if  $(a_i, \mu_i, \nu_i) = (1, 0, 1)$  then
17:     $L \leftarrow (-P, -P + 2Q)$ 
18:  else if  $(a_i, \mu_i, \nu_i) = (1, 1, 0)$  then
19:     $L \leftarrow (P, -P)$ 
20:  else
21:    (necessarily  $(a_i, \mu_i, \nu_i) = (1, 1, 1)$ )
22:     $L \leftarrow (-P, P)$ 
23:  end if
24: end for
25: return  $2^{-n}(L(0) + L(1))$ 

```

5.2 Core Conclusion of Part II

Corollary 5 (exact linear-time linear evaluation). *For every fixed $\text{Constant}_{\text{Key}} \in \mathbb{Z}_{2^n}$ and every mask pair $\alpha, \beta \in \mathbb{F}_2^n$, Algorithm 2 computes $\text{CORRELATION}(\alpha \rightarrow \beta)$ exactly in time $O(n)$ using $O(1)$ working memory.*

Proof. Theorem 4 gives correctness. At each bit position, the algorithm computes (μ_i, ν_i) , selects one of finitely many local kernels, and updates the two-component vector L . Each step therefore uses constant work and constant memory. Since the scan runs once over the n bit positions, the runtime is $O(n)$ and the working memory is $O(1)$. $\square$

Proposition 5 (exact optimization form of the fixed-addend linear search problem). *For fixed $\text{Constant}_{\text{Key}} \in \mathbb{Z}_{2^n}$,*

$$\max_{\alpha, \beta \in \mathbb{F}_2^n} |\text{CORRELATION}(\alpha \rightarrow \beta)| = 2^{-n} \max_{(\mu_i, \nu_i) \in \{0,1\}^2} \left| e_0^\top \left(\prod_{i=0}^{n-1} M_{a_i, \mu_i, \nu_i}^L \right) \mathbf{1}^\top \right|. \quad (41)$$

The mask bits are recovered uniquely from the control bits by

$$\beta_i = \nu_i, \quad \alpha_i = \mu_i \oplus \nu_i.$$

Proof. The change of variables

$$(\alpha_i, \beta_i) \longleftrightarrow (\mu_i, \nu_i) = (\alpha_i \oplus \beta_i, \beta_i)$$

is bijective at each bit position, with inverse

$$(\mu_i, \nu_i) \longmapsto (\alpha_i, \beta_i) = (\mu_i \oplus \nu_i, \nu_i).$$

Applying this bitwise bijection to equation (40) gives

$$\text{CORRELATION}(\alpha \rightarrow \beta) = 2^{-n} e_0^\top \left(\prod_{i=0}^{n-1} M_{a_i, \mu_i, \nu_i}^L \right) \mathbf{1}^\top.$$

Taking absolute values and maximizing over all masks is therefore equivalent to maximizing over all control sequences $(\mu_i, \nu_i)_{i=0}^{n-1}$, which proves equation (41). $\square$

5.3 Threshold Interpretation of the Carry into Bit i

There is also a second exact description of the carry state, namely a threshold form. Let

$$(\text{Constant}_{\text{Key}})_{<i} := \text{Constant}_{\text{Key}} \bmod 2^i.$$

Proposition 6 (threshold characterization of the carry into bit i). *For every fixed addend $\text{Constant}_{\text{Key}}$ and every $i \in \{0, \dots, n-1\}$, the carry into bit i is the threshold event*

$$c_i(x) = 1 \iff (x \bmod 2^i) \geq 2^i - (\text{Constant}_{\text{Key}})_{<i}, \quad ((\text{Constant}_{\text{Key}})_{<i} \neq 0). \quad (42)$$

If $(\text{Constant}_{\text{Key}})_{<i} = 0$, then $c_i(x) = 0$ identically.

Proof. Write

$$x = q2^i + t, \quad 0 \leq t < 2^i, \quad t = x \bmod 2^i.$$

Also write

$$\text{Constant}_{\text{Key}} = (\text{Constant}_{\text{Key}})_{<i} + 2^i h$$

for some integer h . Then

$$x + \text{Constant}_{\text{Key}} = (q + h)2^i + (t + (\text{Constant}_{\text{Key}})_{<i}).$$

The carry into bit i is 1 exactly when the low-order sum $t + (\text{Constant}_{\text{Key}})_{<i}$ contributes one extra multiple of 2^i , namely when

$$t + (\text{Constant}_{\text{Key}})_{<i} \geq 2^i.$$

Substituting $t = x \bmod 2^i$ and rearranging gives

$$c_i(x) = 1 \iff (x \bmod 2^i) \geq 2^i - (\text{Constant}_{\text{Key}})_{<i},$$

which is equation (42). If $(\text{Constant}_{\text{Key}})_{<i} = 0$, then

$$0 \leq x \bmod 2^i \leq 2^i - 1,$$

so the inequality can never hold and $c_i(x) = 0$ identically. $\square$

5.4 A Worked Linear Micro-Example

Take $n = 4$, constant $\text{Constant}_{\text{Key}} = 0110_2$, input mask $\alpha = 0110_2$, and output mask $\beta = 0100_2$. Then

$$\mu = \alpha \oplus \beta = 0010_2, \quad \nu = \beta = 0100_2.$$

Reading the bits from low to high gives

$$(a_0, a_1, a_2, a_3) = (0, 1, 1, 0),$$

$$(\mu_0, \mu_1, \mu_2, \mu_3) = (0, 1, 0, 0), \quad (\nu_0, \nu_1, \nu_2, \nu_3) = (0, 0, 1, 0).$$

Therefore the four local kernels are

$$M_{a_0, \mu_0, \nu_0}^L = M_{0,0,0}^L = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix},$$

$$M_{a_1, \mu_1, \nu_1}^L = M_{1,1,0}^L = \begin{pmatrix} 1 & -1 \\ 0 & 0 \end{pmatrix},$$

$$M_{a_2, \mu_2, \nu_2}^L = M_{1,0,1}^L = \begin{pmatrix} -1 & -1 \\ 0 & 2 \end{pmatrix},$$

$$M_{a_3, \mu_3, \nu_3}^L = M_{0,0,0}^L = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}.$$

Starting from

$$L_0 = e_0^\top = (1, 0),$$

the exact prefix states are

$$L_1 = L_0 M_{0,0,0}^L = (2, 0),$$

$$L_2 = L_1 M_{1,1,0}^L = (2, -2),$$

$$L_3 = L_2 M_{1,0,1}^L = (-2, -6),$$

$$L_4 = L_3 M_{0,0,0}^L = (-10, -6).$$

Hence

$$L_4 \mathbf{1}^\top = -16,$$

and equation (32) gives the exact correlation

$$\text{CORRELATION}(\alpha \rightarrow \beta) = 2^{-4}(-16) = -1.$$

Thus this mask pair is a perfect negative approximation for the chosen addend. The example shows concretely how the local signed transfers accumulate: the second step creates antisymmetry, the third step amplifies it, and the final step preserves the full magnitude.

Part III: Supplementary Context, Extensions, and Downstream Views

The core theorem-and-algorithm package is complete at the end of Part II. Part III is supplementary: it records how the same carry-governed structure relates to later formalizations, two-variable lifts, and downstream uses.

6 Fixed-Addend Formalization as a Specialization of the Lipmaa–Moriai View

Lipmaa–Moriai express the two-input differential probability through the XOR of two carry strings. In their notation,

$$\text{DP}^+(\alpha, \beta \rightarrow \gamma) = \Pr_{x,y} [\text{Carry}(x, y) \oplus \text{Carry}(x \oplus \alpha, y \oplus \beta) = \alpha \oplus \beta \oplus \gamma].$$

This is the freeze-of-one-input bridge of Table 6. At the arithmetic level, freezing $y = \text{Constant}_{\text{Key}}$ is an exact specialization of the binary map $(x, y) \mapsto x + y$ to the unary family $x \mapsto x + \text{Constant}_{\text{Key}}$. What fails is only the probabilistic shortcut that would substitute $y = \text{Constant}_{\text{Key}}$ into the two-input averaged formula and pretend that the averaging measure were unchanged. In the two-input setting one averages over all y , but in the fixed-addend setting the defining feature is that one specific $\text{Constant}_{\text{Key}}$ matters. Thus the specialization to one fixed operand is not a trivial corollary at the probability level; it is a different probabilistic object whose hidden-state chain remembers the actual bits of the constant. This explains why the fixed-addend case is naturally treated as a separate object and why Azimi later observes that the general bit-vector model does not automatically solve the fixed-constant problem.

7 Azimi’s Bit-Vector Reinterpretation: Validity and Approximate Weight

Azimi et al. convert the fixed-addend differential theory into solver-grade bit-vector formulas. In the terminology of Table 6, this is a bit-vector-compilation bridge: the underlying fixed-addend object is unchanged, but its validity and weight logic are rewritten as efficient symbolic constraints. This is mathematically informative for reverse engineering because it identifies exactly which pieces of the fixed-addend theory had to be made machine-readable.

7.1 Validity Predicate

They define two structural masks

$$s_{001} = \neg(u \ll 1) \wedge \neg(v \ll 1) \wedge (u \oplus v), \quad s_{11*} = (u \ll 1) \wedge (v \ll 1),$$

and an auxiliary bit-vector g detecting the invalid 11* positions by analysing whether a whole carry-chain block violates the run constraint. Their final validity theorem says that

$$\text{valid}_{\boxplus_a}(u, v) = \text{Equals}(s_{001} \vee (s_{11*} \wedge g), 0)$$

is true if and only if the differential is valid. In other words, the forbidden local pattern 001 and the forbidden run pattern inside 11*-chains are precisely the two obstructions to validity. This does not introduce a different probabilistic object; it compiles the same stepwise obstruction structure of the fixed-addend recurrence into an SMT-friendly bit-vector predicate.

7.2 Approximate Weight

For a valid differential, Azimi write the exact weight as

$$\text{wt}_{\boxplus_a}(u, v) = -\log_2 \text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) = -\sum_{i=0}^{n-1} \log_2 \phi_i.$$

The irrationality of some local factors means that no exact fixed-width bit-vector can represent the weight in general. They therefore separate the easy positions, where $\phi_i \in \{1, 1/2\}$, from the long 11*-chains where

$$\phi_i = \frac{p_i}{2^{\ell_i-1}}$$

for an integer numerator p_i , and approximate $\log_2 p_i$. Their bit-vector function **BvWeight** computes a close approximation to the weight with an explicit one-sided error bound

$$-0.029n \leq \text{wt}_{\boxplus_a}(u, v) - \text{apxwt}_{\boxplus_a}(u, v) \leq 0.$$

Thus the bit-vector approximation slightly *overestimates* the exact weight, but by a quantitatively controlled amount. For the present paper, the point is simple: Azimi's model is only possible because the old fixed-addend differential theory had already exposed exactly which local blocks contribute half-weights, which blocks are impossible, and which blocks require run-aware numerators.

8 Quasi-Hadamard Extensions and Two-Coordinate Lifts

The Pseudo-Hadamard Transform paper by Lipmaa matters here not because it is one of the three Miyano source nodes, but because it reveals what the post-2001 mathematics does *after* the local addition law has been understood. In the terminology of Table 6, this is a coupled-lift bridge. The relevant class is no longer a single map $x \mapsto x \boxplus y$, but a coupled two-coordinate mapping

$$F(x_1, x_2) = (2^{\kappa_{11}}x_1 \boxplus_{\sigma} 2^{\kappa_{12}}x_2, 2^{\kappa_{21}}x_1 \boxplus_{\sigma} 2^{\kappa_{22}}x_2),$$

where $\boxplus_{\sigma}$ means either modular addition ($\sigma = 1$) or subtraction ($\sigma = 0$). Lipmaa calls this class the *Quasi-Hadamard Transforms*. The key point is that the coordinatewise addition/subtraction law is unchanged; what changes is merely that there are now two output coordinates and hence two carry chains coupled inside one larger criterion.

To make the structure explicit, write

$$z_{j1} = 2^{\kappa_{j1}}x_1, \quad z_{j2} = 2^{\kappa_{j2}}x_2, \quad y_j = z_{j1} \boxplus_{\sigma} z_{j2}$$

for $j \in \{1, 2\}$. For a differential $\delta = (\Delta x \rightarrow \Delta y)$ with $\Delta x, \Delta y \in \mathbb{F}_2^{2n}$, define

$$\omega_j = J(\sigma \cdot (\Delta z_{j1} \oplus \Delta y_j) \oplus \Delta z_{j1} \oplus 1 \oplus \text{eq}(\Delta z_{j1}, \Delta z_{j2}, \Delta y_j)) \oplus \text{xor}(\Delta z_{j1}, \Delta z_{j2}, \Delta y_j).$$

Then the coupled system can be written as a block matrix equation

$$M(\delta)x = a(\delta, x), \tag{43}$$

with

$$M(\delta) = \begin{pmatrix} J[(\Delta z_{11} \oplus \Delta y_1)]J^{\kappa_{11}} & J[(\Delta z_{12} \oplus \Delta y_1)]J^{\kappa_{12}} \\ J[(\Delta z_{21} \oplus \Delta y_2)]J^{\kappa_{21}} & J[(\Delta z_{22} \oplus \Delta y_2)]J^{\kappa_{22}} \end{pmatrix},$$

and

$$a(\delta, x) = \begin{pmatrix} \omega_1 \oplus J((\Delta z_{11} \oplus \Delta z_{12}) \cdot c_1^{\sigma}) \\ \omega_2 \oplus J((\Delta z_{21} \oplus \Delta z_{22}) \cdot c_2^{\sigma}) \end{pmatrix},$$

where c_j^{σ} is the carry or borrow chain of the j th coordinate addition. In other words, the later linear-algebraic framework does not invent a new local phenomenon; it stacks two old ones and keeps track of their compatibility.

The next lemma is a restatement of Lipmaa's Lemma 2 in the notation of the present draft [8].

Lemma 4 (Lipmaa's linear-algebraic lifting). *For every Quasi-Hadamard Transform F of the form above, the differential probability satisfies*

$$\text{DP}_F(\delta) = \Pr_x[M(\delta)x = a(\delta, x)].$$

When the two coordinates collapse to a single L_1 -type map $F(x_1, x_2) = 2^{\kappa_1}x_1 \boxplus_\sigma 2^{\kappa_2}x_2$ with $\kappa_1 \leq \kappa_2 + 1$, the carry term becomes eliminable and one recovers the closed-form and log-time world of ordinary addition and subtraction.

This cited lemma is structurally informative. It shows that once the local addition law has been isolated, the passage to a larger coupled mapping is explicit: one writes down the same bit-level constraints for each coordinate, packages them into a block matrix, and then studies when the carry terms can still be neutralised. That is why the PHT paper functions as a bridge citation in the present discussion.

9 Pseudo-Hadamard Transform as a Four-Projection Gate

The most important invertible member of the Quasi-Hadamard family is

$$\text{PHT}(x_1, x_2) = (2x_1 + x_2, x_1 + x_2),$$

with inverse

$$\text{PHT}^{-1}(y_1, y_2) = (y_1 - y_2, 2y_2 - y_1).$$

The following theorem is the PHT possibility criterion proved as Theorem 3 in [8]. Let

$$\text{PHT}_1(x_1, x_2) = 2x_1 + x_2, \quad \text{PHT}_2(x_1, x_2) = x_1 + x_2,$$

and

$$\text{PHT}_1^{-1}(y_1, y_2) = y_1 - y_2, \quad \text{PHT}_2^{-1}(y_1, y_2) = 2y_2 - y_1.$$

Theorem 5 (PHT possibility criterion). *For $\delta = (\Delta x_1, \Delta x_2 \rightarrow \Delta y_1, \Delta y_2)$, the differential δ is PHT-possible if and only if all four projected differentials are possible:*

$$\text{DP}_{\text{PHT}_1}(\Delta x_1, \Delta x_2 \rightarrow \Delta y_1) > 0,$$

$$\text{DP}_{\text{PHT}_2}(\Delta x_1, \Delta x_2 \rightarrow \Delta y_2) > 0,$$

$$\text{DP}_{\text{PHT}_1^{-1}}(\Delta y_1, \Delta y_2 \rightarrow \Delta x_1) > 0,$$

$$\text{DP}_{\text{PHT}_2^{-1}}(\Delta y_2, \Delta y_1 \rightarrow \Delta x_2) > 0.$$

Equivalently, the forward coordinate laws and the inverse coordinate laws must all be simultaneously satisfiable. The order in the fourth test is intentional: the second inverse coordinate is the scalar map $(y_2, y_1) \mapsto 2y_2 - y_1$, so its two-input differential is read with $(\Delta y_2, \Delta y_1)$ in that order.

Remark 2 (Relevance to the present framework). The point for the present draft is structural rather than bibliographic. The PHT criterion uses the addition law four times: two projected checks come from the forward coordinates and two from the inverse coordinates. Since each coordinate is still just a shifted addition or subtraction, the whole coupled transform remains downstream of the local carry law. That is why the fixed-addend core remains relevant when later work moves from one coordinate to two.

A small exhaustive example makes the criterion concrete. Take $n = 4$ and

$$(\Delta x_1, \Delta x_2 \rightarrow \Delta y_1, \Delta y_2) = (0000, 0001 \rightarrow 0001, 0001).$$

A direct enumeration over all $2^{2n} = 256$ inputs gives

$$\text{DP}_{\text{PHT}}(0000, 0001 \rightarrow 0001, 0001) = \frac{128}{256} = \frac{1}{2}.$$

The four projected probabilities are

$$\begin{aligned} \text{DP}_{\text{PHT}_1}(0000, 0001 \rightarrow 0001) &= 1, & \text{DP}_{\text{PHT}_2}(0000, 0001 \rightarrow 0001) &= \frac{1}{2}, \\ \text{DP}_{\text{PHT}_1^{-1}}(0001, 0001 \rightarrow 0000) &= \frac{1}{2}, & \text{DP}_{\text{PHT}_2^{-1}}(0001, 0001 \rightarrow 0001) &= \frac{1}{2}, \end{aligned}$$

so the criterion predicts possibility and the brute-force count agrees.

To make sure this was not an isolated example, the accompanying C++ checker exhaustively tests all differentials when $n \leq 4$. It evaluates the direct PHT differential probability, the four projected probabilities, and compares the zero/nonzero verdicts. No mismatches were found across 69,904 quadruples $(\Delta x_1, \Delta x_2, \Delta y_1, \Delta y_2)$. That does not prove Lipmaa’s theorem, of course, but it is a useful consistency check against transcription or indexing mistakes.

10 Relation to the PHT Matrix Criterion

This bridge paper contributes three mathematically concrete points to the present framework.

First, it confirms that the post-LM01 line continues to work with carry-state mathematics, but expresses it in a larger matrix language. Second, it shows that the same addition law can be imported into coupled mappings such as PHT and then into concrete cipher components such as the Twofish partial round. Third, it clarifies the technical comparison used here: the local carry laws and the later matrix systems are mathematically linked rather than disconnected.

So the PHT paper is not the origin. It is a mathematically downstream coupled-lift bridge from “local carry law” to “structured coupled transform”, and it helps explain how coordinatewise addition constraints later reappear inside larger constructions.

11 Relation to Wallen’s Carry-Function Classification

Wallen works in the general two-input setting and proves that the correlation of a linear approximation of the carry function can be computed recursively using common-prefix masks. In his notation, the Fourier transform of the carry components satisfies

$$\widehat{C}_0(v, w) = \delta(v, w), \quad \widehat{C}_{i+1}(v, w) = \frac{1}{2} \left(\delta(v + e_i, w) + \delta(v, w + e_i) + \widehat{C}_i(v, w) - \widehat{C}_i(v + e_i, w + e_i) \right),$$

and the final correlation of a carry approximation is expressed via a recursively computed common-prefix mask $z = \text{cpm}(u, \text{eq}(v, w))$. The resulting theorem is that

$$C(u^{\text{Carry}} \leftarrow v, w) = 0 \quad \text{if } vz \neq 0 \text{ or } wz \neq 0,$$

and otherwise

$$C(u^{\text{Carry}} \leftarrow v, w) = (-1)^{\text{HW}(vw)} 2^{-\text{HW}(z)}.$$

It is important not to collapse the roles of the present draft and Wallen’s work. Wallen studies the general two-input linear/carry setting, whereas the present draft isolates the lower one-variable fixed-addend branch obtained after freezing one input. The relevance of Wallen here is therefore twofold. First, he provides the general carry-function backbone from which the fixed-addend linear side should be understood as a specialization. Second, the full 2003 report explicitly acknowledges Miyano 1998 as the predecessor on the one-addend-fixed linear side, which helps fix the literature relation between the two levels.

This is the mathematical backbone of the general linear-addition theory. The bridge to the present draft is the freeze-of-one-input relation from Table 6: the fixed-addend problem is simpler because one of the two inputs of the carry function is no longer random but fixed. Our transfer formula (40) is therefore not claimed to be Wallen’s theorem itself, but the exact one-variable fixed-constant specialization of the same carry-function viewpoint.

The present draft should therefore not be read as replacing Wallen’s work. Rather, Wallen systematizes

the outer two-input carry-linear theory, while the present paper isolates the lower fixed-addend one-variable branch. The newly recovered full report is especially valuable because it explicitly connects these layers by identifying Miyano 1998 as the predecessor on the one-addend-fixed linear side.

12 Schulte-Geers 2011: CCZ-Equivalent Quadratic Representation of Addition

A later paper that is highly useful for the present paper, especially on the linear side, is Schulte-Geers' 2011 note on CCZ-equivalence of addition modulo 2^n . In the terminology of Table 6, this is a graph-transport bridge. Its role here is specific: it does not replace the fixed-addend analysis, but gives a very clean global Boolean reformulation showing that the same carry mathematics can be transported to a quadratic vectorial function whose Walsh and differential behaviour become unusually transparent.

Let $c(x, y)$ be the carry word of $x \boxplus y$ and define

$$q(x, y) = M(x \star y),$$

where M is the right-shifted partial-sums map. Schulte-Geers proves that the map

$$\beta(x, y) = (x \oplus c(x, y), y \oplus c(x, y))$$

is a bijection with inverse

$$\alpha(x, y) = (x \oplus q(x, y), y \oplus q(x, y)),$$

and that the linear map

$$T(x, y, z) = (x \oplus y \oplus z, x \oplus y, x \oplus z)$$

maps the graph of q onto the graph of modular addition. In other words, addition modulo 2^n is CCZ-equivalent to a quadratic vectorial Boolean function. This does not erase the carry; it repackages it.

The linear payoff is immediate. Writing $\phi_2(u, v, w)$ for the correlation of the two-input addition approximation with output mask u and input masks v, w , Schulte-Geers obtains the explicit formula

$$\phi_2(u, v, w) = \phi_q(u \oplus v \oplus w, u \oplus v, u \oplus w),$$

and if we put

$$z = M^t(u \oplus v \oplus w),$$

then

$$\phi_2(u, v, w) = \mathbf{1}_{u \oplus v \leq z} \mathbf{1}_{u \oplus w \leq z} (-1)^{(u \oplus v) \cdot (u \oplus w)} 2^{-\text{HW}(z)}, \quad (44)$$

where $\leq$ denotes the bitwise partial order. This is mathematically important for the present draft because it shows that the support mask governing nonzero correlations, obtained recursively in Wallen's presentation, admits the explicit closed form $z = M^t(u \oplus v \oplus w)$. So the line "Miyano $\rightarrow$ Wallen" can now be extended to the three-step chain

$$\begin{aligned} & \text{Miyano fixed-addend carry kernel} \\ & \longrightarrow \text{Wallen recursive carry classification} \\ & \longrightarrow \text{Schulte-Geers explicit CCZ/Boolean form.} \end{aligned}$$

The mathematical point is that the underlying two-input addition graph is preserved up to graph transport: by moving to a CCZ-equivalent quadratic shadow one obtains a more explicit closed form without changing the transported local mechanism.

This also clarifies why the present paper insists on the fixed-addend 2-state transfer system rather than jumping directly to equation (44). Schulte-Geers studies the full two-input graph of addition and uses Fourier analysis on a quadratic function q . Miyano's 1996 claim, by contrast, is explicitly about

$$f_{\text{Constant}_{\text{Key}}}(X) = Y = X + \text{Constant}_{\text{Key}} \pmod{2^N},$$

with one addend fixed. The fixed-addend problem therefore lives one level lower: before the global graph transformation, at the stage where a single carry bit already controls the exact correlation. In that sense Schulte-Geers is not the core fixed-addend analysis, but a later unifier that independently supports the linear half of the present formulation.

There is a differential counterpart as well. The same CCZ lens converts differential equations of addition into simple linear equations and, as Schulte-Geers states, gives a simple re-derivation of the Lipmaa–Moriai explicit probability formula. This matters because it shows that both the differential and linear two-input theories admit the same global Boolean reformulation. The fixed-addend viewpoint used here remains carry-first; the Schulte-Geers paper shows a later two-input graph-transport reformulation of the carry-governed addition law.

Proposition 7 (one-bit sparse-mask law and partitioned refinement). *One of the best-known local approximations of addition in later ARX papers is*

$$(\alpha \oplus (\alpha \gg 1), \alpha) \xrightarrow{\boxplus} \alpha, \quad (45)$$

for sparse masks α . In the one-bit case $\alpha = 2^i$ with $i > 0$, let $s = u \boxplus v$ be two-input addition and let c_i be the carry into bit i . Then

$$u_i \oplus u_{i-1} \oplus v_i \oplus s_i = u_{i-1} \oplus c_i.$$

Consequently:

- (a) for every fixed incoming carry c_{i-1} , exactly three of the four pairs (u_{i-1}, v_{i-1}) satisfy $c_i = u_{i-1}$, so the approximation

$$\Pr[u_i \oplus u_{i-1} \oplus v_i \oplus s_i = 0] = \frac{3}{4}$$

and therefore has imbalance $1/2$;

- (b) conditioned on $u_{i-1} = v_{i-1}$, one has $c_i = u_{i-1}$ identically, so the same approximation becomes exact on that subset.

For fixed-addend addition $y = x \boxplus \text{Constant}_{\text{Key}}$, the corresponding identity becomes

$$x_i \oplus x_{i-1} \oplus y_i = x_{i-1} \oplus a_i \oplus c_i.$$

Hence in the fixed-addend setting the sign and magnitude are controlled exactly by the distribution of the single carry bit c_i , which is precisely what the 2-state transfer matrices compute.

Proof. The displayed identity follows from $s_i = u_i \oplus v_i \oplus c_i$. For part (a), fix $c_{i-1} \in \{0, 1\}$ and write

$$c_i = \text{Maj}(u_{i-1}, v_{i-1}, c_{i-1}).$$

A direct inspection of the four possible pairs (u_{i-1}, v_{i-1}) shows that exactly three satisfy $c_i = u_{i-1}$. Therefore $\Pr(c_i = u_{i-1}) = 3/4$, which is equivalent to the stated probability for the approximation. For part (b), if $u_{i-1} = v_{i-1}$ then

$$c_i = \text{Maj}(u_{i-1}, u_{i-1}, c_{i-1}) = u_{i-1},$$

so the approximation is exact on that subset. The fixed-addend identity is the same computation with v_i replaced by the constant bit a_i and s_i replaced by y_i . $\square$

13 One-Variable Constant-Addition Evidence

The paper of Guan and Ding studies the side observable

$$\Delta_{\text{Constant}_{\text{Key}}}(x) := (x \boxplus \text{Constant}_{\text{Key}}) \oplus x \oplus \text{Constant}_{\text{Key}}, \quad (46)$$

for fixed constants $\text{Constant}_{\text{Key}}$ and uniform x [16]. In the terminology of Table 6, this is an observable-projection bridge: the hidden carry word itself becomes the measured variable. At first sight this looks

like a side issue. In fact it isolates a one-variable fixed-addend observable whose distribution is governed directly by carry structure.

Indeed, by the standard carry decomposition

$$x \boxplus \text{Constant}_{\text{Key}} = x \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x),$$

one gets the identity

$$\Delta_{\text{Constant}_{\text{Key}}}(x) = \text{Carry}_{\text{Constant}_{\text{Key}}}(x). \quad (47)$$

So Guan–Ding’s “equivalent problem” is literally a problem about the visible carry word of fixed-addend addition. It is the same carry object that drives the present differential and linear analyses, except that here the carry word itself is promoted from hidden state to observable.

13.1 The spike-addend family studied by Guan–Ding

Guan–Ding solve the distribution problem completely for the special family

$$A_i := \{2^i, 2^{n-1} + 2^i\}, \quad 0 \leq i \leq n-2. \quad (48)$$

The extra most-significant spike 2^{n-1} does not affect the lower carry chain started at bit i , so both constants in A_i induce the same carry-run structure below the top boundary. Their key combinatorial device is the “succession weight”, i.e. the length of the consecutive run of ones starting at a prescribed bit position.

Definition 1 (run length from level i). For $0 \leq i \leq n-1$ and $x \in \{0,1\}^n$, define $\rho_i(x)$ to be the largest integer $r \geq 0$ such that

$$x_i = x_{i+1} = \cdots = x_{i+r-1} = 1,$$

with the convention that $\rho_i(x) = 0$ when $x_i = 0$, and with truncation at the word boundary when the run reaches bit $n-1$.

This is the same quantity that Guan–Ding call the succession weight of the suffix X_i [16]. In modern notation it is simply the carry-run length generated when one adds a single spike at bit i .

Proposition 8 (exact local carry law for a spike addend). *Let $\text{Constant}_{\text{Key}} \in A_i$, and let $c_j(x)$ be the carry into bit j of $x \boxplus \text{Constant}_{\text{Key}}$. Then*

$$c_0(x) = 0, \quad c_1(x) = \cdots = c_i(x) = 0, \quad c_{i+1}(x) = x_i, \quad (49)$$

and for every $i+1 \leq j \leq n-2$,

$$c_{j+1}(x) = x_j c_j(x). \quad (50)$$

Consequently,

$$\Delta_{\text{Constant}_{\text{Key}}}(x) = \sum_{j=i+1}^{n-1} c_j(x) 2^j. \quad (51)$$

Proof. For $\text{Constant}_{\text{Key}} \in A_i$, all addend bits below position i are zero, so no carry can be created before bit i and hence $c_1(x) = \cdots = c_i(x) = 0$. At bit i one has $a_i = 1$ and $c_i(x) = 0$, so

$$c_{i+1}(x) = \text{Maj}(x_i, 1, 0) = x_i.$$

For every higher bit j with $i+1 \leq j \leq n-2$, the addend bit is again $a_j = 0$, hence

$$c_{j+1}(x) = \text{Maj}(x_j, 0, c_j(x)) = x_j c_j(x).$$

Equation (51) is then just the identity $\Delta_{\text{Constant}_{\text{Key}}}(x) = \text{Carry}_{\text{Constant}_{\text{Key}}}(x)$. If $\text{Constant}_{\text{Key}} = 2^{n-1} + 2^i$, the extra top bit affects only the most significant output position and possible overflow, not the lower recursion (49)–(50). $\square$

Theorem 6 (Guan–Ding carry-run law for A_i). *Let $\text{Constant}_{\text{Key}} \in A_i$. For uniform $x \in \{0, 1\}^n$, put*

$$\Delta_{\text{Constant}_{\text{Key}}}(x) = (x \boxplus \text{Constant}_{\text{Key}}) \oplus x \oplus \text{Constant}_{\text{Key}}.$$

Then $\Delta_{\text{Constant}_{\text{Key}}}(x)$ is completely determined by $\rho_i(x)$:

$$\Delta_{\text{Constant}_{\text{Key}}}(x) = \sum_{j=i+1}^{i+\rho_i(x)} 2^j. \quad (52)$$

Equivalently, $\Delta_{\text{Constant}_{\text{Key}}}(x)$ is either zero or a contiguous block of ones beginning at bit $i + 1$.

Moreover, for $0 \leq t < n - i - 1$,

$$\Pr[\rho_i(x) = t] = 2^{-(t+1)}, \quad (53)$$

and at the top boundary,

$$\Pr[\rho_i(x) = n - i - 1] = 2^{-(n-i-1)}. \quad (54)$$

Hence the nonzero values of $\Delta_{\text{Constant}_{\text{Key}}}(x)$ occur with a truncated geometric law, and all admissible outputs are carry blocks of the form

$$0, \quad 2^{i+1}, \quad 2^{i+1} + 2^{i+2}, \quad \dots, \quad \sum_{j=i+1}^{n-1} 2^j.$$

Proof. By Proposition 8, the carry is born at bit $i + 1$ iff $x_i = 1$, and once born it propagates by the simple law $c_{j+1}(x) = x_j c_j(x)$. Therefore the carry survives from bit $i + 1$ through bit $i + t$ exactly when

$$x_i = x_{i+1} = \dots = x_{i+t-1} = 1,$$

and it stops at the first position where a zero occurs. Hence the carry word is precisely the contiguous block in (52). Since the bits of x are uniform and independent, the run length distribution is geometric until the top boundary, giving (53) and (54). Adding the extra term 2^{n-1} affects only the most significant position and possible overflow, hence the lower carry run started at bit i is unchanged for $\text{Constant}_{\text{Key}} = 2^{n-1} + 2^i$. $\square$

Remark 3 (unpacked case split of the carry-run law). Theorem 6 can be read in three concrete cases:

(C1) If $x_i = 0$, then no carry is created, so $\rho_i(x) = 0$ and $\Delta_{\text{Constant}_{\text{Key}}}(x) = 0$ with probability $1/2$.

(C2) If

$$x_i = x_{i+1} = \dots = x_{i+t-1} = 1, \quad x_{i+t} = 0, \quad 1 \leq t \leq n - i - 2,$$

then the carry block has exact length t ,

$$\Delta_{\text{Constant}_{\text{Key}}}(x) = \sum_{j=i+1}^{i+t} 2^j,$$

and this event has probability $2^{-(t+1)}$.

(C3) If

$$x_i = x_{i+1} = \dots = x_{n-2} = 1,$$

then the carry chain reaches the top boundary,

$$\Delta_{\text{Constant}_{\text{Key}}}(x) = \sum_{j=i+1}^{n-1} 2^j,$$

and the probability is $2^{-(n-i-1)}$.

This is the full mathematical meaning of Guan–Ding’s succession-weight language: a run of ones in the input suffix is the carry block, seen directly as an output observable.

The significance of Theorem 6 for the present draft is not merely that it gives another exact formula. It shows that even when one does *not* ask for a differential probability or a Walsh correlation, the same one-variable fixed-addend object collapses to a carry-run law. In that sense Guan–Ding strongly supports the claim that carry-run structure is the correct hidden object for fixed-addend one-variable observables.

13.2 A Counting Corollary and Dyadic Values

The previous theorem immediately gives the counting law behind Guan–Ding’s discussion of “structure and count”.

Corollary 6 (count of attainable carry blocks in the spike family). *Fix $0 \leq i \leq n-2$ and $\text{Constant}_{\text{Key}} \in A_i$. For each integer j with $1 \leq j \leq n-i$, there is exactly one carry block of Hamming weight j attainable as $\Delta_{\text{Constant}_{\text{Key}}}(x)$, namely*

$$B_{i,j} := \sum_{m=i+1}^{i+j} 2^m$$

(with the obvious truncation when $i+j = n$), and its probability is

$$\Pr[\Delta_{\text{Constant}_{\text{Key}}}(x) = B_{i,j}] = \begin{cases} 2^{-j}, & 1 \leq j < n-i, \\ 2^{-(n-i-1)}, & j = n-i. \end{cases} \quad (55)$$

In particular, all probabilities in this family are dyadic and decay exponentially with the carry-block length.

Proof. The block $B_{i,j}$ corresponds to the event $\rho_i(x) = j-1$. Therefore the claim follows directly from (53) and (54). $\square$

This is one of the reasons the paper is worth integrating into the present framework. It gives an explicit example of a fixed-addend quantity whose entire distribution is dictated by low-level carry runs. That is the same style of local-to-global mechanism on which the fixed-addend differential and linear theories rely.

13.3 From Probability Tables to an $O(n)$ Search Procedure

Guan–Ding do not stop at the probability table. They explicitly turn it into an optimal search order for the equation

$$(x \boxplus \text{Constant}_{\text{Key}}) = x \oplus \text{Constant}_{\text{Key}} \oplus \eta, \quad (56)$$

where η is the guessed carry word [16]. Their principle is simple: enumerate pairs $(\text{Constant}_{\text{Key}}, \eta)$ in decreasing probability order. For the spike family A_i , this means trying shorter carry blocks first, because Theorem 6 and Corollary 6 show that these have the largest probabilities.

Proposition 9 (probability ordering for the Guan–Ding equivalent problem). *Among all deterministic guess orders for the candidate pairs $(\text{Constant}_{\text{Key}}, \eta)$ in equation (56), an order of nonincreasing probabilities minimizes the expected number of guesses. For the spike family A_i , this means that one should guess shorter carry blocks before longer carry blocks; ties inside the same probability layer may be broken arbitrarily.*

Proof. This is the standard exchange argument for optimal guessing. If two adjacent candidates with probabilities $p < q$ are guessed in the order (p, q) , swapping them to (q, p) lowers the expected guess count by exactly $q - p > 0$. Repeating this swap until no inversion remains yields the unique optimal shape: probabilities must be nonincreasing. The spike-family conclusion then follows from Corollary 6, which shows that the probability of a nonzero carry block depends only on its length and decays dyadically with that length. $\square$

Remark 4 (what Guan–Ding’s Algorithm 1 contributes exactly). Guan–Ding’s Algorithm 1 is precisely the probability ordering of Proposition 9. For their restricted equivalent problem, they report success probability 1, time complexity $O(n)$, and expected guess count

$$\frac{1}{2n} \left(3n^2 - 7n + 12 - 3 \cdot 2^{-n+3} \right). \quad (57)$$

The important point for the present draft is not the polynomial itself, but the mechanism that produces it: once the visible carry word has been reduced to a run-length law, the candidate space acquires an exact probability ranking and therefore an exact optimal guess order.

Remark 5 (what this contributes to the fixed-addend picture). Guan–Ding does *not* give us Miyano’s missing core differential or linear recurrences. What it gives us is a precise downstream one-variable carry-run analysis:

- (G1) the one-variable fixed-addend setting remains mathematically natural long after the RC5 era;
- (G2) carry-run structure alone is enough to produce exact probability tables and an $O(n)$ search strategy for a nontrivial fixed-addend observable; and
- (G3) the phrase “linear-time algorithm” in the later descriptions of Miyano is therefore mathematically compatible with the present paper: this style of low-bit carry recursion really does lead to linear-time procedures in one-variable constant-addition problems.

So this paper should be read as side evidence for the *tractability* of fixed-addend carry mathematics, not as a replacement for Miyano’s differential/linear core itself.

14 ADP/XOR State-Machine Analogies

The ADP/XOR line of Lipmaa, Wallen and Dumas studies a dual object: the additive differential probability of XOR when differences are expressed using addition modulo 2^N . Their key theorem is that the function is a rational series with an 8-dimensional linear representation,

$$\text{adp}_{\oplus}(\alpha, \beta \rightarrow \gamma) = LA_{\omega_{N-1}} \cdots A_{\omega_0} C, \quad \omega_i = 4\alpha_i + 2\beta_i + \gamma_i. \quad (58)$$

Later work proves the maximality law

$$\max_{\alpha, \beta} \text{adp}_{\oplus}(\alpha, \beta \rightarrow \gamma) = \text{adp}_{\oplus}(0, \gamma \rightarrow \gamma),$$

and derives recurrence formulas for adjoining leading bits. This line is not about Miyano’s fixed-addend addition itself, but the analogy is mathematically deep:

- fixed-addend differentials of addition are governed by a 4-state hidden carry-pair automaton;
- fixed-addend linear approximations are governed by a 2-state hidden carry automaton;
- additive differentials of XOR are governed by an 8-state octal automaton.

All three are finite-state probability machines built from local carry/borrow compatibility. This is one reason the ADP/XOR papers matter for the present paper: once the hidden state is identified, the global quantity admits a linear finite-state representation and can be analyzed by matrix propagation.

15 Relation to Partitioning Theory

The partitioning papers by Leurent and later Chaskey work are not core source papers, but they are direct consumers of the kind of mathematics developed here [20]. In the terminology of Table 6, this is a conditioning-consumer bridge: the local addition law is not replaced, only conditioned on low carry information. Their linear starting point is the sparse-mask law

$$(\alpha \oplus (\alpha \gg 1), \alpha) \xrightarrow{\boxplus} \alpha,$$

which for a single active bit has imbalance $1/2$ by Proposition 7. Proposition 7 also explains the basic partitioning move: once one conditions on the relevant low carry context, the local approximation can become exact on the selected subset. Their differential starting points are the elementary fixed-weight patterns

$$(2^i, 2^i) \xrightarrow{\boxplus} 0, \quad (2^i, 0) \xrightarrow{\boxplus} 2^i,$$

each with probability $1/2$ in the basic unpartitioned setting. They then observe that if one partitions the data according to carefully chosen low bits, the effective probabilities or imbalances inside each subset can increase to 1 or to a larger effective bias. Rewriting Leurent’s cost ratios in reciprocal *gain* notation, the improvement factors take the form

$$G_{\text{lin}} = \mu \frac{\hat{\varepsilon}^2}{\varepsilon^2}, \quad G_{\text{diff}} = \mu \frac{T\hat{p}}{2p}, \quad G_{\text{diff-2}} = \mu \frac{T\hat{p}^2}{2p^2},$$

where μ is the retained data fraction, T is the structure size, and $\hat{p}$ or $\hat{\varepsilon}$ are the improved subset values. In exact mathematical terms, partitioning is a conditioning step on low carry information: one guesses or filters a low-bit context so that the local addition law becomes more deterministic on the retained subset.

16 Chaskey and ChaCha as Downstream Uses of Fixed-Addend Carry Laws

Differential-linear attacks on Chaskey and ChaCha still lean on local carry laws. In the ChaCha line one finds the explicit carry approximations

$$\Pr(\text{Carry}(x, y)[i] = y[i - 1]) = \frac{1}{2} \left(1 + \frac{1}{2}\right) = \frac{3}{4}, \quad i > 0,$$

and

$$\Pr(\text{Carry}(x, y)[i] \oplus \text{Carry}(x, y)[i - 1] = 0) = \frac{1}{2} \left(1 + \frac{1}{2}\right) = \frac{3}{4}, \quad i > 0.$$

These formulas are explicit local carry laws inside downstream differential-linear analyses. Likewise, differential-linear complexity estimates of the form

$$O(1/(p^2\varepsilon^4))$$

for a differential part of probability p and a linear part of imbalance ε show how local addition mathematics feeds directly into full-cipher attack complexity.

Writing the carry skeleton explicitly makes this reuse easy to track: later ARX attacks are not invoking unrelated tricks, but reusing local addition laws inside larger cipher analyses.

17 The ADP/XOR Matrices as a Large-State Analogy

To make the finite-state analogy completely explicit, Appendix D lists the eight 8×8 matrices of the ADP/XOR representation. The mathematical message is that once the right local state is chosen, a probability object that initially looks intractable becomes a matrix product. The same structural move occurs in the present paper:

Object	Hidden state size	canonical representation
fixed-addend differential of addition	4	carry-pair matrices $M_{a,u,v}^D$
fixed-addend linear approximation of addition	2	carry matrices $M_{a,\mu,\nu}^L$
additive differential probability of XOR	8	octal matrices $A_0, \dots, A_7$

This is the cleanest way to extract the method contribution of the ADP/XOR papers without conflating them with the core fixed-addend line.

18 Relation to Maximum-ADP Recurrence Formulas

The maximum-ADP paper by Mouha et al. is also methodologically relevant because it shows how once an automaton exists, one can derive clean leading-bit recurrences. For example, they prove recurrences of the form

$$\text{adp}_{\oplus}(\alpha 0, \beta 0 \rightarrow \gamma 0) = \text{adp}_{\oplus}(\alpha, \beta \rightarrow \gamma)$$

and four-term recurrences for words with leading 1 bits. This is mathematically parallel to our use of the local matrices $M_{a,u,v}^D$ and $M_{a,\mu,\nu}^L$: one bit is appended, one local transfer is applied, and the global probability updates by a finite-state rule. Once the hidden state is identified, finite-state recurrences follow systematically.

Part IV: Supplementary Synthesis and Technical Corollaries

Part III surveyed downstream shells, two-variable lifts, and later reformulations that preserve the same carry-governed local law in different exact senses. Part IV records supplementary synthesis and technical corollaries. The task here is to place the derived core and its alternative surfaces side by side, extract the invariant mechanism, and summarize the common technical picture.

19 The Eight Local Types as a Projected Automaton

The scalar recurrence of Theorem 1 becomes transparent once one projects the 4-state automaton onto the parity classes $d_i = 0$ and $d_i = 1$. The eight local types are best understood as follows.

- $S_i = 000$: the previous and current parities are both 0. The hidden state lies in $\{00, 11\}$ at both levels. Both exact states remain feasible, and the new conditional carry probability is the arithmetic mean of the deterministic branch induced by a_{i-1} and the previous hidden mass. Hence $\phi_i = 1$ and $\delta_i = (a_{i-1} + \delta_{i-1})/2$.
- $S_i = 001$: previous parity 0, current parity 1. This is the forbidden configuration; every exact branch dies. Hence $\phi_i = 0$.
- $S_i \in \{010, 100\}$: parity changes but one of the visible bits is active. Exactly one of the two hidden parity-1 states is reachable from each compatible parity-0 state, producing a factor $1/2$ and pinning the new carry to the addend bit a_{i-1} .
- $S_i \in \{011, 101\}$: one remains in parity 1 with one visible active bit. The mass is merely transported inside the parity-1 class, so $\delta_i = \delta_{i-1}$ while the local factor remains $1/2$.
- $S_i = 110$: this is the interior of a $u = v = 1$ block. The state stays in parity 0, but survival forces the previous hidden carry probability to agree with the addend bit. This is exactly where the addend-run theorem comes from.
- $S_i = 111$: this is the terminal branch of a $u = v = 1$ block where the next XOR parity flips. The local factor is the complement of the 110 factor, and therefore expresses the final-bit twist of the run theorem.

This projected picture makes the differential mechanism fully explicit. The whole differential half is a finite-state machine together with a parity projection.

20 Run Blocks as Matrix Powers

There is also a useful matrix-power viewpoint on long differential blocks. Consider the normalized local matrices $P_{a,u,v}^D = 2^{-1}M_{a,u,v}^D$. When one remains inside a parity-0 block with $u_i = v_i = 1$, the relevant exact matrices are

$$P_{0,1,1}^D = \frac{1}{2} \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}, \quad P_{1,1,1}^D = \frac{1}{2} \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}.$$

A chain of identical addend bits therefore corresponds to repeated multiplication by the same sparse matrix. The reason the chain theorem collapses to a run-constraint is that these matrices have rank one on the live subspace: after one successful step the hidden mass is forced into a one-dimensional direction. This is the automata-theoretic meaning of “the interior addend bits must be equal”.

A similar phenomenon appears for parity-1 transport. The matrices

$$P_{0,0,1}^D = P_{0,1,0}^D = \frac{1}{2} \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad P_{1,0,1}^D = P_{1,1,0}^D = \frac{1}{2} \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

transport mass inside the odd-parity class and explain why the local probability contribution is exactly 1/2 in four separate cases.

21 A Fixed-Addend Linear Specialization Corollary

We can now isolate the linear consequences that are rigorously immediate from the one-carry transfer formula.

Corollary 7 (fixed-addend linear specialization). *For every fixed addend $\text{Constant}_{\text{Key}} \in \mathbb{F}_2^n$ and masks $\alpha, \beta \in \mathbb{F}_2^n$, the correlation of the approximation $\alpha \rightarrow \beta$ is given exactly by equation (40). In particular:*

- (L1) *the correlation depends explicitly on the actual addend bits $a_0, \dots, a_{n-1}$;*
- (L2) *if for some bit position i the current prefix vector L_i satisfies $L_i M_{a_i, \mu_i, \nu_i}^L = 0$, then the final correlation is zero;*
- (L3) *if, from some position onward, the recurrence remains on a one-dimensional signed subspace and each remaining local action is multiplication by a factor in $\{\pm 1, \pm 2\}$, then the terminal correlation is a signed dyadic rational.*

Proof. Property (L1) is immediate from equation (40). For (L2), once a prefix vector becomes zero, every later prefix vector remains zero under further matrix multiplication, so the terminal sum also vanishes. For (L3), the assumed one-dimensional transport shows that the remaining terminal sum is obtained from the current nonzero scalar by multiplying by finitely many factors in $\{\pm 1, \pm 2\}$ and then normalizing by 2^n . $\square$

This corollary is the fixed-addend linear specialization forced by the 1996 linear abstract once the one-carry recurrence is made explicit.

22 Li et al. 2011: Carry-Free Booleanization of Addition

The 2011 Boolean-function paper of Li et al. provides an additional mathematical viewpoint. In the terminology of Table 6, this is a state-elimination bridge. It studies the full two-input map

$$G(X, Y) := X + Y \pmod{2^n}$$

as an n -dimensional vectorial Boolean function on $2n$ input bits [17]. Its role in the present discussion is more specific than a generic citation. The paper isolates three exact mathematical facts:

- (B1) the explicit carry state can be eliminated and absorbed into a recurrence on the coordinate functions themselves;
- (B2) the higher coordinate functions have a highly restricted Walsh spectrum, namely a quasi-Bent law with amplitudes 0 or $\pm 2^{-i}$; and
- (B3) the whole vectorial map is first-order but not second-order correlation immune.

Only the first item descends literally to the fixed-addend setting used in this draft, because freezing one input changes the ambient function class. The full two-input results are still worth stating explicitly, however, because they show what the same carry skeleton becomes after Booleanization.

22.1 The full two-input carry-free recurrence

Write

$$X = \sum_{i=0}^{n-1} x_i 2^i, \quad Y = \sum_{i=0}^{n-1} y_i 2^i, \quad G(X, Y) = \sum_{i=0}^{n-1} g_i(X, Y) 2^i.$$

Let c_i be the carry into bit i , so

$$c_0 = 0, \quad g_i(X, Y) = x_i \oplus y_i \oplus c_i, \quad c_{i+1} = x_i y_i \oplus x_i c_i \oplus y_i c_i.$$

Li et al.'s key algebraic step is to eliminate c_i in favour of the previous observable coordinate g_i .

Theorem 7 (Li et al. carry-free coordinate recurrence). *For the full two-input addition map $G(X, Y) = X + Y \bmod 2^n$, the coordinate functions satisfy*

$$g_0(X, Y) = x_0 \oplus y_0, \tag{59}$$

and for $0 \leq i \leq n-2$,

$$g_{i+1}(X, Y) = x_{i+1} \oplus y_{i+1} \oplus x_i y_i \oplus (x_i \oplus y_i)(g_i(X, Y) \oplus x_i \oplus y_i). \tag{60}$$

Proof. Since $g_i = x_i \oplus y_i \oplus c_i$, one has

$$c_i = g_i(X, Y) \oplus x_i \oplus y_i.$$

Substituting this into the carry recursion gives

$$c_{i+1} = x_i y_i \oplus x_i(g_i \oplus x_i \oplus y_i) \oplus y_i(g_i \oplus x_i \oplus y_i) = x_i y_i \oplus (x_i \oplus y_i)(g_i \oplus x_i \oplus y_i),$$

and then $g_{i+1} = x_{i+1} \oplus y_{i+1} \oplus c_{i+1}$ yields equation (60). Equation (59) is the base case $c_0 = 0$. $\square$

Two structural consequences are immediate. First, g_i depends only on the prefix variables $(x_0, \dots, x_i, y_0, \dots, y_i)$. Second, the hidden carry state has been absorbed into the previous observable coordinate, so the addition law can now be analysed directly as a recursion in Boolean-function space rather than as an external state machine.

22.2 Walsh spectrum: why the coordinates are quasi-Bent

For a Boolean function h on m bits, write its normalized Walsh transform as

$$S_h(w) := 2^{-m} \sum_{z \in \mathbb{F}_2^m} (-1)^{h(z) \oplus \langle w, z \rangle}.$$

A k -order quasi-Bent function is one whose Walsh coefficients take only the values 0 or $\pm 2^{-(m-k)/2}$.

Theorem 8 (Li et al. quasi-Bent coordinate law). *Let $G = (g_{n-1}, \dots, g_0)$ be the full two-input addition map. Then g_0 is linear, and for every $1 \leq i \leq n-1$ the coordinate function g_i is a $(2n-2i)$ -order quasi-Bent Boolean function on $2n$ variables. Equivalently,*

$$S_{g_i}(w) \in \{0, \pm 2^{-i}\}, \quad w \in \mathbb{F}_2^{2n}, \quad 1 \leq i \leq n-1. \tag{61}$$

Interpretive proof sketch. The base case is

$$g_1 = x_1 \oplus y_1 \oplus x_0 y_0,$$

whose Walsh coefficients are 0 or $\pm 1/2$. For the induction step, equation (60) shows that passing from g_i to g_{i+1} adds one new bit pair (x_i, y_i) and splits the Walsh sum into four local branches. On those four branches, the previous coefficient is either copied, sign-flipped, or cancelled, and every surviving coefficient acquires one extra factor $1/2$. Hence every nonzero Walsh amplitude of g_{i+1} has magnitude $2^{-(i+1)}$. Because g_i ignores the top $2n-2(i+1)$ variables, the Walsh support is confined to a prefix

subspace; together with the amplitude law, this matches the quasi-Bent order $2n - 2i$ via the defining magnitude formula. $\square$

The important point for the present draft is not merely the label “quasi-Bent”. It is the mechanism behind it: every increase in bit position adds one more carry-conditioned local split, and that split halves the magnitude of every surviving Walsh coefficient. The spectrum is therefore structured by the carry recursion rather than by an unstructured growth of algebraic complexity.

22.3 Correlation immunity of the full vectorial map

Li et al. also analyse the full vectorial Boolean function

$$G(X, Y) = (g_{n-1}(X, Y), \dots, g_0(X, Y)).$$

Recall that a vectorial Boolean function $H : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^r$ is correlation immune of order t if for every nonzero $\alpha \in \mathbb{F}_2^r$ and every input mask w with $1 \leq \text{HW}(w) \leq t$, one has

$$S_{\alpha \cdot H}(w) = 0, \quad \alpha \cdot H := \bigoplus_{j=0}^{r-1} \alpha_j H_j.$$

Theorem 9 (Li et al. correlation-immunity law). *The full two-input addition map G is correlation immune of order 1 but not of order 2. Equivalently, for every nonzero $\alpha \in \mathbb{F}_2^n$ and every mask $w \in \mathbb{F}_2^{2n}$ with $\text{HW}(w) = 1$,*

$$S_{\alpha \cdot G}(w) = 0,$$

whereas some mask w with $\text{HW}(w) = 2$ yields

$$S_{\alpha \cdot G}(w) \neq 0.$$

Interpretive proof sketch. Single-bit observations cannot linearly expose the carry interaction, so first-order Walsh coefficients vanish for every nonzero output combination. But the second coordinate already contains the bilinear term

$$g_1 = x_1 \oplus y_1 \oplus x_0 y_0,$$

so the two-bit interaction (x_0, y_0) leaves a nonzero second-order trace. In other words, the first visible nonlinearity of addition appears not at order 1 but at order 2, precisely when the smallest carry interaction $x_0 y_0$ becomes observable. $\square$

Remark 6 (what survives after freezing one addend). For the present fixed-addend setting, Theorem 7 descends exactly by replacing Y with the constant word $\text{Constant}_{\text{Key}}$. Theorem 8 and Theorem 9, however, are statements about the full $2n$ -variable vectorial map and should not be transplanted verbatim to $x \mapsto x \boxplus \text{Constant}_{\text{Key}}$. What survives literally is the carry-elimination mechanism itself.

22.4 Fixed-addend specialization

The same idea, specialized to fixed addend addition, gives a useful alternative representation.

Proposition 10 (fixed-addend carry-free coordinate recurrence). *Let $f_i(x)$ denote the i -th output bit of $x \boxplus \text{Constant}_{\text{Key}}$. Then*

$$f_0(x) = x_0 \oplus a_0, \quad f_{i+1}(x) = x_{i+1} \oplus a_{i+1} \oplus x_i \oplus a_i \oplus x_i a_i \oplus (x_i \oplus a_i) f_i(x). \quad (62)$$

for $0 \leq i \leq n - 2$.

Proof. Let c_i be the carry into bit i . Since

$$c_i = f_i(x) \oplus x_i \oplus a_i,$$

and

$$c_{i+1} = x_i a_i \oplus x_i c_i \oplus a_i c_i,$$

substitution gives

$$c_{i+1} = x_i a_i \oplus x_i (f_i(x) \oplus x_i \oplus a_i) \oplus a_i (f_i(x) \oplus x_i \oplus a_i) = x_i a_i \oplus x_i \oplus a_i \oplus (x_i \oplus a_i) f_i(x).$$

Using $f_{i+1}(x) = x_{i+1} \oplus a_{i+1} \oplus c_{i+1}$ yields equation (62). The base case $f_0(x) = x_0 \oplus a_0$ follows from $c_0 = 0$. $\square$

This recurrence encodes the same addition as the carry automaton but with the state absorbed into the evolving output function itself. In the full two-input setting, Theorems 7, 8, and 9 show the broader Boolean consequences of that elimination. In the fixed-addend setting, Proposition 10 is the exact one-variable descendant needed in the present draft.

23 Closed Forms for Constant Addend Blocks

The 2-state matrices admit exact powers on constant runs of addend bits. Let

$$B_0 = M_{0,0,0}^L = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}, \quad B_1 = M_{1,0,0}^L = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}.$$

Proposition 11 (exact transfer through constant-addend inactive blocks). *For every integer $r \geq 0$,*

$$B_0^r = \begin{pmatrix} 2^r & 0 \\ 2^r - 1 & 1 \end{pmatrix}, \quad B_1^r = \begin{pmatrix} 1 & 2^r - 1 \\ 0 & 2^r \end{pmatrix}. \quad (63)$$

Consequently, if on a block of r consecutive bit positions one has $(\mu_j, \nu_j) = (0, 0)$ and all addend bits on that block are equal to $a \in \{0, 1\}$, then the transfer through the entire block is exactly B_a^r .

Proof. The case $r = 0$ is the identity matrix. If the formula holds for r , then

$$B_0^{r+1} = \begin{pmatrix} 2^r & 0 \\ 2^r - 1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 2^{r+1} & 0 \\ 2^{r+1} - 1 & 1 \end{pmatrix},$$

and similarly

$$B_1^{r+1} = \begin{pmatrix} 1 & 2^r - 1 \\ 0 & 2^r \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 2^{r+1} - 1 \\ 0 & 2^{r+1} \end{pmatrix}.$$

This proves equation (63). The final statement is immediate because on such a block each local transfer matrix equals the same B_a . $\square$

These formulas explain why long low-order blocks of equal addend bits behave so rigidly. They are the linear-side counterpart of the differential run theorem, and they replace bit-by-bit simulation on inactive blocks by an exact closed form.

24 Single-Bit Output Masks and Walsh Coefficients of Carry Intervals

Proposition 12 (single-bit output masks as Walsh coefficients of carry intervals). *Fix i with $0 \leq i \leq n-1$, put $\beta = 2^i$, and let α be arbitrary. Then*

$$\text{CORRELATION}(\alpha \rightarrow 2^i) = 2^{-n} \sum_x (-1)^{\langle \alpha \oplus 2^i, x \rangle \oplus a_i \oplus c_i(x)}. \quad (64)$$

If $(\text{Constant}_{\text{Key}})_{<i} = 0$, then $c_i(x) \equiv 0$ and the correlation reduces to a Walsh coefficient of the linear form $\langle \alpha \oplus 2^i, x \rangle$. If $(\text{Constant}_{\text{Key}})_{<i} \neq 0$, define

$$\tau_i := 2^i - (\text{Constant}_{\text{Key}})_{<i}.$$

Then

$$c_i(x) = \mathbf{1}_{\{x \bmod 2^i \geq \tau_i\}},$$

so equation (64) is a Walsh coefficient of the interval indicator $\mathbf{1}_{\{t \geq \tau_i\}}$ on $\{0, \dots, 2^i - 1\}$.

Proof. Equation (64) is equation (27) specialized to $\beta = 2^i$, so that $\langle \beta, \text{Constant}_{\text{Key}} \rangle = a_i$ and $\langle \beta, \text{Carry}_{\text{Constant}_{\text{Key}}}(x) \rangle = c_i(x)$. If $(\text{Constant}_{\text{Key}})_{<i} = 0$, then equation (42) gives $c_i(x) \equiv 0$. If $(\text{Constant}_{\text{Key}})_{<i} \neq 0$, the same threshold formula yields

$$c_i(x) = 1 \iff (x \bmod 2^i) \geq 2^i - (\text{Constant}_{\text{Key}})_{<i} \iff (x \bmod 2^i) \geq \tau_i.$$

Hence the carry bit is exactly the interval indicator claimed above. $\square$

This gives a fixed-addend one-variable analogue of the later explicit Walsh formulas of the CCZ/Boolean line: the 2-state carry transfer and the interval-Walsh viewpoint are exact rewritings of the same correlation.

25 Differential Side: Summary of the Supporting Evidence

At this point the differential half rests on explicit mathematics. The 4-state automaton gives an exact count formula. The scalar recurrence of Machado is fully compatible with it. The addend-run theorem is a direct consequence of the 110/111 cases. Azimi's validity predicate is the same obstruction structure compiled into bit-vectors. Therefore the fixed-addend differential half now rests on explicit mathematics rather than on citation alone.

26 Linear Side: Summary of the Supporting Evidence

The linear half is less directly exposed by later citations than the differential half, but mathematically it is on strong ground. We have an exact finite-state formula, a direct reduction to the carry function, a clear structural alignment with Wallen’s general classification, and three exact auxiliary surfaces: the carry-free coordinate recurrence of Proposition 10, the constant-block transfer law of Proposition 11, and the interval-Walsh representation of Proposition 12. Therefore the fixed-addend linear half is no longer an informal analogy; it is an explicit formulation that explains why later papers could build on it.

27 Unified View: Local State $\rightarrow$ Block Matrix $\rightarrow$ Feasibility

The deepest unifying point of the current synthesis is that the cited descendants of Miyano are not a pile of unrelated tricks. In the precise bridge senses catalogued in Table 6, they repeatedly expose three surfaces of one carry-governed scheme:

- (U1) a *local hidden state* governed by carries or carry-pairs;
- (U2) a *matrix propagation law* obtained by chaining local transitions;
- (U3) a *feasibility or possibility criterion* obtained by asking whether the propagated mass or constraint system remains non-zero.

This is the exact compression used in the sequel. Otherwise the 4-state automaton, Machado’s recurrence, Wallen’s carry recursion, and Lipmaa’s L_2 matrix equation can look unrelated even though they are connected by quotient, freeze, or coupled-lift relations built on the same carry-driven local law.

Surface 1: exact local state

For the fixed-addend XOR differential of $x \mapsto x \boxplus \text{Constant}_{\text{Key}}$, the exact hidden state is the carry pair

$$s_i = (c_i, c'_i) \in \{00, 01, 10, 11\},$$

so the local evolution is a 4-state machine. For the fixed-addend linear problem, the exact hidden state is just the single carry

$$c_i \in \{0, 1\},$$

so the local evolution is a 2-state machine. In both cases the visible bit relation only tells us a parity-level fact, while the exact internal state remains hidden and must be propagated.

Surface 2: matrix propagation

The matrix form is the exact globalisation of those local states:

$$N_{i+1} = N_i M_{a_i, u_i, v_i}^D$$

for the differential side and

$$L_{i+1} = L_i M_{a_i, \mu_i, \nu_i}^L$$

for the linear side. Once this is written down, the whole word-level probability object is a chained product of small local operators.

The Lipmaa L_2 / PHT line is the coupled-lift version of the same move. One no longer propagates a tiny probability vector directly; instead one packages the coordinatewise addition constraints into a block matrix equation

$$M(\delta)x = a(\delta, x).$$

But that is still a matrix-propagation statement. The only difference is that the state is now lifted from one coordinate to a coupled two-coordinate mapping.

Surface 3: feasibility as survival of support

The third surface is where the various papers start to look stylistically different while staying mathematically the same.

For the differential 4-state automaton, feasibility means that the propagated mass does not vanish:

$$e_{00} \left(\prod_{i=0}^{n-1} M_{a_i, u_i, v_i}^D \right) \mathbf{1} > 0.$$

For the linear side, non-zero correlation means that the signed carry propagation does not cancel completely:

$$e_0^\top \left(\prod_{i=0}^{n-1} M_{a_i, \mu_i, \nu_i}^L \right) \mathbf{1}^\top \neq 0.$$

For the PHT / L_2 side, feasibility means that the block matrix equation has at least one solution. In each case, the final question is the same: *did the locally propagated structure survive globally or not?*

Machado as a rank-1 compression of the 4-state machine

The relation between the exact 4-state automaton and Machado's scalar recurrence can be written as an exact quotient statement.

Proposition 13 (Machado's scalar recurrence as a quotient of the exact four-state machine). *Let $N_i = (N_i^{00}, N_i^{01}, N_i^{10}, N_i^{11})$ be the exact prefix-count vector after processing bits $0, \dots, i-1$, let*

$$\Lambda_i := N_i \mathbf{1}, \quad d_i := u_i \oplus v_i,$$

and assume $\Lambda_i > 0$. Then:

- (i) N_i is supported on the parity slice determined by d_i :

$$d_i = 0 \implies N_i \in \mathcal{V}_0, \quad d_i = 1 \implies N_i \in \mathcal{V}_1,$$

where

$$\mathcal{V}_0 = \{(\lambda_0, 0, 0, \lambda_1)\}, \quad \mathcal{V}_1 = \{(0, \lambda_0, \lambda_1, 0)\}.$$

- (ii) *There exists a unique $\delta_i \in [0, 1]$ such that*

$$N_i = \Lambda_i \mathcal{V}_{d_i}(\delta_i),$$

with

$$\mathcal{V}_0(\delta) = (1 - \delta, 0, 0, \delta), \quad \mathcal{V}_1(\delta) = (0, 1 - \delta, \delta, 0).$$

- (iii) *If*

$$N_{i+1} = N_i M_{a_i, u_i, v_i}^D,$$

then the induced evolution of (Λ_i, δ_i) is exactly the recurrence of Theorem 1, and

$$\phi_i = \frac{\Lambda_{i+1}}{2\Lambda_i}.$$

Proof. Lemma 1 shows that every surviving prefix at level i satisfies $c_i \oplus c'_i = d_i$. Since N_i counts surviving prefixes by exact carry pair, this forces the support of N_i to lie in $\mathcal{V}_0$ when $d_i = 0$ and in $\mathcal{V}_1$ when $d_i = 1$, proving (i).

Because $\Lambda_i > 0$, the normalized vector $\Lambda_i^{-1} N_i$ is a probability distribution on the corresponding slice. By the definition of $\delta_i = \Pr(c_i = 1 \mid E_i)$ and equation (11), this distribution is exactly $\mathcal{V}_{d_i}(\delta_i)$. This proves (ii), and uniqueness of δ_i is immediate from the displayed coordinates.

Finally, equation (24) gives the exact one-bit update $N_{i+1} = N_i M_{a_i, u_i, v_i}^D$. Each live prefix has exactly two possible extensions of the next input bit, so the local survival factor is the ratio of surviving extensions

to all extensions, namely $\phi_i = \Lambda_{i+1}/(2\Lambda_i)$. Applying the local matrix to the slice vector $\mathcal{V}_{d_i}(\delta_i)$ and renormalizing gives exactly the same update formulas for δ_i and ϕ_i as in Theorem 1; equivalently, one may compare with the branch table of Proposition 1. This proves (iii). $\square$

Thus the scalar theory is not a separate theory. It is the one-parameter normalization of the exact four-state machine on the live parity slice.

Placement of the PHT bridge in the same skeleton

The PHT paper therefore fits naturally into the same skeleton. More precisely, by the coupled-lift relation of Table 6, it does *not* introduce a new local non-linearity. Instead, it lifts the same addition law into a two-coordinate environment and then asks for simultaneous satisfiability of the forward and inverse coordinate projections. That is why the PHT criterion can be stated as four projected possibility checks: each projected check is still downstream of the same local carry law, merely viewed through a coordinate lens.

This also explains why the phrase “local state $\rightarrow$ block matrix $\rightarrow$ feasibility” is not decorative prose. It is the actual genealogy:

$$\begin{aligned} &\text{Miyano local fixed-addend carry law} \\ \implies &\text{Machado / Wallen finite-state recurrences} \\ \implies &\text{Lipmaa block-matrix lifting and PHT criteria.} \end{aligned}$$

Once written this way, the citation graph becomes a single mathematical lineage rather than a collection of disconnected branches.

C++ sanity suite for the unified skeleton

To test this unification against implementation errors, the accompanying C++ sanity suite performs five families of checks:

- (C1) exhaustive comparison of brute-force differential probabilities against the exact 4-state matrix product for all $n \leq 6$;
- (C2) exhaustive comparison of brute-force linear correlations against the exact 2-state transfer product for all $n \leq 6$;
- (C3) exhaustive verification for all $n \leq 4$ that direct PHT possibility agrees with the four-projection criterion;
- (C4) exhaustive verification for all $n \leq 8$ of the threshold-carry characterization (42) and of the carry-free coordinate recurrence of Proposition 10;
- (C5) exhaustive verification for all $n \leq 6$ of the single-bit interval Walsh formula of Proposition 12.

All five check families returned zero mismatches in the current draft. In particular, items (C1) and (C2) are the small-space validation of the two recovered $O(n)$ evaluators: for every fixed addend and every admissible visible tuple with $n \leq 6$, Algorithm 1 agrees exactly with brute-force differential counting and Algorithm 2 agrees exactly with brute-force Walsh summation. The same verifier also reproduces the counterexample to the discarded stronger run-law claim discussed above. This does not replace proof, but it does provide an implementation-level consistency check against transcription or indexing mistakes.

The payoff is a sharper technical summary. The paper is no longer merely saying “many later papers cited Miyano.” It is saying something stronger and much more useful: *the descendants keep reusing the same carry-governed hidden-state mechanism, first locally, then as a recurrence, then as a block-matrix feasibility problem.*

28 Conclusion

This paper turns the fixed-addend map $x \mapsto x + \text{Constant}_{\text{Key}} \pmod{2^n}$ into an explicit modern object with exact differential and linear formulas. The main results are concrete: an exact 4-state carry-pair recurrence for XOR differentials, an exact 2-state carry-transfer recurrence for fixed-addend linear correlations, and two explicit $O(n)$ evaluators that compute these quantities once the addend and the visible masks are fixed. The accompanying verifier checks both evaluators exhaustively on all small spaces with $n \leq 6$, where Algorithm 1 matches brute-force differential counting and Algorithm 2 matches brute-force Walsh summation with zero mismatches.

The paper does not rely on recovering any literal lost notation. The principal cryptologic contribution is the exact evaluation of addend-dependent differential probabilities and linear correlations for the fixed-addend modular-addition map, together with proofs, algorithms, and implementation checks. If later work wants implementations, searches, solver models, or ARX design heuristics, it can build directly on these explicit recurrences and evaluators.

A Fixed-Subtractend Dependency as the Borrow Chain Skeleton

In the main text, the fixed-addend unary object is written as

$$f_{\text{Constant}_{\text{Key}}} : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_{2^n}, \quad f_{\text{Constant}_{\text{Key}}}(x) = x + \text{Constant}_{\text{Key}} \pmod{2^n}.$$

Its inverse-side presentation is

$$f_{\text{Constant}_{\text{Key}}}^{-1} : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_{2^n}, \quad f_{\text{Constant}_{\text{Key}}}^{-1}(y) = y - \text{Constant}_{\text{Key}} \pmod{2^n}.$$

The purpose of this appendix is to expose the reverse-side hidden memory explicitly. On the forward side, the hidden object is the carry chain. On the inverse side, the corresponding hidden object is the borrow chain. We show that the fixed-subtractend dependency is governed by the same local memory skeleton, now read through reverse borrow propagation.

Bitwise borrow recursion. Write

$$y = \sum_{i=0}^{n-1} y_i 2^i, \quad \text{Constant}_{\text{Key}} = \sum_{i=0}^{n-1} k_i 2^i, \quad x = f_{\text{Constant}_{\text{Key}}}^{-1}(y) = \sum_{i=0}^{n-1} x_i 2^i.$$

Let b_i denote the borrow entering bit position i , with

$$b_0 = 0.$$

Then subtraction satisfies

$$x_i = y_i \oplus k_i \oplus b_i$$

and the next borrow is given by

$$b_{i+1} = \text{Borrow}(y_i, k_i, b_i),$$

where

$$\text{Borrow}(y_i, k_i, b_i) = (\overline{y_i} \wedge k_i) \oplus (\overline{y_i} \wedge b_i) \oplus (k_i \wedge b_i).$$

Equivalently, $b_{i+1} = 1$ iff $y_i < k_i + b_i$ in ordinary integer arithmetic.

Define the borrow word

$$\text{Borrow}_{\text{Constant}_{\text{Key}}}(y) := \sum_{i=0}^{n-1} b_i 2^i.$$

Proposition 14 (Exact borrow correction identity). *For every fixed $\text{Constant}_{\text{Key}} \in \mathbb{Z}_{2^n}$ and every $y \in \mathbb{Z}_{2^n}$,*

$$f_{\text{Constant}_{\text{Key}}}^{-1}(y) = y \oplus \text{Constant}_{\text{Key}} \oplus \text{Borrow}_{\text{Constant}_{\text{Key}}}(y).$$

Proof. By the bitwise subtraction rule,

$$x_i = y_i \oplus k_i \oplus b_i$$

for every bit position i . Reassembling the bits yields

$$x = y \oplus \text{Constant}_{\text{Key}} \oplus \text{Borrow}_{\text{Constant}_{\text{Key}}}(y).$$

□

Carry and borrow are the same hidden chain seen from opposite directions. The inverse-side borrow chain is not a new phenomenon. It is the same local memory already present in the forward addend-side execution.

Proposition 15 (Matched carry–borrow identity). *Suppose*

$$y = f_{\text{Constant}_{\text{Key}}}(x) = x + \text{Constant}_{\text{Key}} \pmod{2^n}, \quad x = f_{\text{Constant}_{\text{Key}}}^{-1}(y) = y - \text{Constant}_{\text{Key}} \pmod{2^n}.$$

Let c_i be the carry entering bit i in the forward addition $x + \text{Constant}_{\text{Key}}$, and let b_i be the borrow entering bit i in the reverse subtraction $y - \text{Constant}_{\text{Key}}$. Then

$$b_i = c_i \quad \text{for all } 0 \leq i \leq n.$$

Proof. Let $x_{<i}, \text{Constant}_{\text{Key}<i}, y_{<i}$ denote the truncations to the lower i bits. By definition of the forward carry,

$$x_{<i} + \text{Constant}_{\text{Key}<i} = y_{<i} + c_i 2^i.$$

By definition of the reverse borrow in $y - \text{Constant}_{\text{Key}}$,

$$x_{<i} + \text{Constant}_{\text{Key}<i} = y_{<i} + b_i 2^i.$$

Hence $b_i = c_i$. □

Therefore, fixed-subtractend dependency is not governed by a different hidden memory. It is governed by the same chain, rewritten in inverse coordinates.

A.1 Differential side: the borrow-pair skeleton

For fixed $\text{Constant}_{\text{Key}}$, define the reverse-side XOR-differential probability by

$$\text{DP}_{\text{Constant}_{\text{Key}}}^{\Box}(V \rightarrow U) := \Pr_y \left[f_{\text{Constant}_{\text{Key}}}^{-1}(y \oplus V) \oplus f_{\text{Constant}_{\text{Key}}}^{-1}(y) = U \right].$$

Using Proposition 14,

$$\begin{aligned} f_{\text{Constant}_{\text{Key}}}^{-1}(y \oplus V) \oplus f_{\text{Constant}_{\text{Key}}}^{-1}(y) &= ((y \oplus V) \oplus \text{Constant}_{\text{Key}} \oplus \text{Borrow}_{\text{Constant}_{\text{Key}}}(y \oplus V)) \oplus (y \oplus \text{Constant}_{\text{Key}} \oplus \text{Borrow}_{\text{Constant}_{\text{Key}}}(y)) \\ &= V \oplus \text{Borrow}_{\text{Constant}_{\text{Key}}}(y \oplus V) \oplus \text{Borrow}_{\text{Constant}_{\text{Key}}}(y). \end{aligned}$$

Hence the visible differential event is entirely controlled by the difference of two borrow chains.

Proposition 16 (Reverse differential bridge). *For all $U, V \in \mathbb{F}_2^n$,*

$$\text{DP}_{\text{Constant}_{\text{Key}}}^{\Box}(V \rightarrow U) = \Pr_y [\text{Borrow}_{\text{Constant}_{\text{Key}}}(y \oplus V) \oplus \text{Borrow}_{\text{Constant}_{\text{Key}}}(y) = U \oplus V].$$

Now consider two reverse executions:

- one on y , with borrow chain (b_i) ;
- one on $y \oplus V$, with borrow chain (b'_i) .

At bit position i , the two outputs are

$$x_i = y_i \oplus k_i \oplus b_i, \quad x'_i = y_i \oplus v_i \oplus k_i \oplus b'_i.$$

Therefore

$$u_i = x_i \oplus x'_i = v_i \oplus b_i \oplus b'_i.$$

Equivalently,

$$b_i \oplus b'_i = u_i \oplus v_i.$$

This is the reverse-side analogue of the carry-pair parity law from the fixed-addend differential core. Thus the exact hidden state is again a pair state

$$(b_i, b'_i) \in \{00, 01, 10, 11\},$$

with the parity slice forced by $u_i \oplus v_i$.

The one-bit update is determined by

$$b_{i+1} = \text{Borrow}(y_i, k_i, b_i), \quad b'_{i+1} = \text{Borrow}(y_i \oplus v_i, k_i, b'_i).$$

Given k_i , v_i , and the current pair state (b_i, b'_i) , the next pair state is obtained by enumerating the single free bit $y_i \in \{0, 1\}$. Hence the differential evaluation is again a forced low-to-high prefix process.

Theorem 10 (Fixed-subtractend differential exact prefix law). *For fixed $\text{Constant}_{\text{Key}}$, the exact XOR-differential probability of the reverse map $f_{\text{Constant}_{\text{Key}}}^{-1}(y) = y - \text{Constant}_{\text{Key}}$ is governed by a four-state borrow-pair prefix system. The local state is (b_i, b'_i) , the parity slice is determined by*

$$b_i \oplus b'_i = u_i \oplus v_i,$$

and the one-bit transition is induced by

$$b_{i+1} = \text{Borrow}(y_i, k_i, b_i), \quad b'_{i+1} = \text{Borrow}(y_i \oplus v_i, k_i, b'_i).$$

Proof. The bridge identity in Proposition 16 shows that the differential event depends only on the difference of two borrow chains. The relation

$$b_i \oplus b'_i = u_i \oplus v_i$$

constrains the admissible pair states at each bit. The local update rule is first-order and is completely determined by the two equations above. Therefore the full exact probability is obtained by a four-state prefix propagation. $\square$

As a consequence, the reverse-side fixed-subtractend differential problem does not introduce a new state complexity. It is the same order-4 hidden-state law as in the fixed-addend differential core, now written in borrow coordinates.

A.2 Linear side: the one-borrow signed transfer

We next define the reverse-side linear correlation

$$\text{CORRELATION}_{\text{Constant}_{\text{Key}}}^{\square}(\beta \rightarrow \alpha) := 2^{-n} \sum_{y \in \mathbb{F}_2^n} (-1)^{\langle \beta, y \rangle \oplus \langle \alpha, f_{\text{Constant}_{\text{Key}}}^{-1}(y) \rangle}.$$

Using Proposition 14,

$$\begin{aligned} \langle \beta, y \rangle \oplus \langle \alpha, f_{\text{Constant}_{\text{Key}}}^{-1}(y) \rangle &= \langle \beta, y \rangle \oplus \langle \alpha, y \oplus \text{Constant}_{\text{Key}} \oplus \text{Borrow}_{\text{Constant}_{\text{Key}}}(y) \rangle \\ &= \langle \alpha \oplus \beta, y \rangle \oplus \langle \alpha, \text{Constant}_{\text{Key}} \rangle \oplus \langle \alpha, \text{Borrow}_{\text{Constant}_{\text{Key}}}(y) \rangle. \end{aligned}$$

Hence the only nonlinear hidden object is again the borrow chain.

Proposition 17 (Reverse linear bridge). *For all masks $\alpha, \beta \in \mathbb{F}_2^n$,*

$$\text{CORRELATION}_{\text{Constant}_{\text{Key}}}^{\square}(\beta \rightarrow \alpha) = (-1)^{\langle \alpha, \text{Constant}_{\text{Key}} \rangle} 2^{-n} \sum_{y \in \mathbb{F}_2^n} (-1)^{\langle \alpha \oplus \beta, y \rangle \oplus \langle \alpha, \text{Borrow}_{\text{Constant}_{\text{Key}}}(y) \rangle}.$$

Set

$$\tilde{\mu}_i := \alpha_i \oplus \beta_i, \quad \tilde{\nu}_i := \alpha_i.$$

For $0 \leq i \leq n$ and $r \in \{0, 1\}$, define the conditioned partial Walsh sums

$$\tilde{L}_i(r) := \sum_{y_0, \dots, y_{i-1}} (-1)^{\sum_{j=0}^{i-1} (\tilde{\mu}_j y_j \oplus \tilde{\nu}_j k_j \oplus \tilde{\nu}_j b_j)} \cdot 1_{\{b_i=r\}}.$$

The initial condition is

$$\tilde{L}_0(0) = 1, \quad \tilde{L}_0(1) = 0.$$

The final correlation is

$$\text{CORRELATION}_{\text{Constant}_{\text{Key}}}^{\square}(\beta \rightarrow \alpha) = 2^{-n} (\tilde{L}_n(0) + \tilde{L}_n(1)).$$

Fix a current borrow r and next borrow s . The local signed transfer is

$$\tilde{T}_i(r, s) := \sum_{\substack{y_i \in \{0,1\} \\ \text{Borrow}(y_i, k_i, r) = s}} (-1)^{\tilde{\mu}_i y_i \oplus \tilde{\nu}_i k_i \oplus \tilde{\nu}_i r}.$$

Define the 2×2 transfer matrix

$$\tilde{M}_i = (\tilde{T}_i(r, s))_{r, s \in \{0,1\}}.$$

Then

$$(\tilde{L}_{i+1}(0), \tilde{L}_{i+1}(1)) = (\tilde{L}_i(0), \tilde{L}_i(1)) \tilde{M}_i.$$

Theorem 11 (Fixed-subtractend linear exact prefix law). *For fixed $\text{Constant}_{\text{Key}}$, the exact reverse-side linear correlation of $f_{\text{Constant}_{\text{Key}}}^{-1}(y) = y - \text{Constant}_{\text{Key}}$ is governed by a two-state one-borrow signed prefix transfer system. The hidden state is the current borrow $b_i \in \{0, 1\}$, and the full correlation is obtained by multiplying the local matrices $\tilde{M}_i$ from low to high bits.*

Proof. By Proposition 17, the correlation depends only on the visible input mask, the fixed constant, and the borrow word. Since the borrow recursion is first-order, conditioning on the current borrow bit b_i is sufficient. The quantity $\tilde{T}_i(r, s)$ gives the exact local signed contribution of bit i , and the matrix recursion above gives the complete low-to-high propagation. Thus the exact linear evaluation is a two-state transfer process. $\square$

Interpretation. The differential side of the reverse map is governed by a *borrow pair*, hence a four-state exact prefix law. The linear side is governed by a *single borrow*, hence a two-state exact signed transfer law. Therefore, fixed-subtractend dependency does not create a new hidden-state order. It exposes the same local memory skeleton already reconstructed for fixed-addend dependency, but through the reverse borrow coordinates of the inverse unary object.

Conceptual consequence. This shows that the fixed-subtractend side is not merely a corollary in the weak sense of replacing $\text{Constant}_{\text{Key}}$ by $-\text{Constant}_{\text{Key}}$. Rather, it possesses its own explicit hidden-object formulation: on the reverse side, the exact low-to-high structure is naturally expressed by borrow propagation. In this sense, the fixed-subtractend dependency is the reverse borrow realization of the same unary analysis core.

B Full 4-State Differential Transition Matrices

State order is (00, 01, 10, 11). Each matrix counts the number of choices of the current random bit $x_i \in \{0, 1\}$ that realise the local transition under the specified triple (a_i, u_i, v_i) .

$$M_{000}^D$$

$$M_{000}^D = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}.$$

$$M_{001}^D$$

$$M_{001}^D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

$$M_{010}^D$$

$$M_{010}^D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

$$M_{011}^D$$

$$M_{011}^D = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}.$$

$$M_{100}^D$$

$$M_{100}^D = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}.$$

$$M_{101}^D$$

$$M_{101}^D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

$$M_{110}^D$$

$$M_{110}^D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

$$M_{111}^D$$

$$M_{111}^D = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}.$$

C Full 2-State Linear Transfer Matrices

State order is $(c_i = 0, c_i = 1)$. Each entry is a signed sum of local Walsh contributions over the current random bit x_i .

$$M_{000}^L$$

$$M_{000}^L = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}.$$

$$M_{001}^L$$

$$M_{001}^L = \begin{pmatrix} 2 & 0 \\ -1 & -1 \end{pmatrix}.$$

$$M_{010}^L$$

$$M_{010}^L = \begin{pmatrix} 0 & 0 \\ 1 & -1 \end{pmatrix}.$$

$$M_{011}^L$$

$$M_{011}^L = \begin{pmatrix} 0 & 0 \\ -1 & 1 \end{pmatrix}.$$

$$M_{100}^L$$

$$M_{100}^L = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}.$$

M_{101}^L

$$M_{101}^L = \begin{pmatrix} -1 & -1 \\ 0 & 2 \end{pmatrix}.$$

 M_{110}^L

$$M_{110}^L = \begin{pmatrix} 1 & -1 \\ 0 & 0 \end{pmatrix}.$$

 M_{111}^L

$$M_{111}^L = \begin{pmatrix} -1 & 1 \\ 0 & 0 \end{pmatrix}.$$

D Exhaustive C++ Verification and Reference Code

To make the verification engineering-grade rather than merely verbal, the formulas were checked against brute force with the standalone C++20 verifier `fixed_addend_reference_verifier.cpp`. The additional companion file `fixed_addend_dpk_lck_reference.cpp` isolates the names DP_K and LC_K used in Appendix G.4. These files are deliberately released as readable reference code rather than as benchmark-tuned search engines. Its job is to make the four-state differential transfer, the two-state signed linear transfer, and the brute-force cross-checks visible in one place.

The current companion verifier performs the following checks.

- (V1) For width $n = 8$ and addends $00, 01, 3c, a5, ff$, it compares the exact four-state differential count against brute force for every pair (u, v) .
- (V2) For the same width and addends, it compares the exact two-state Walsh numerator against brute force for every mask pair (α, β) .
- (V3) For width $n = 16$ and addends $0000, 0001, 1234, beef, ffff$, it performs seeded random brute-force checks on both the differential and linear sides.

A successful run prints:

Listing 1: Expected output of the companion verifier.

```
example C_K(alpha,beta) = -16/16
all fixed-addend differential and linear checks passed
```

The build command is intentionally plain:

Listing 2: Build command for the reference verifier.

```
g++ -O2 -std=c++20 -Wall -Wextra -pedantic fixed_addend_reference_verifier.cpp -o fixed_addend_reference_verifier
g++ -O2 -std=c++20 -Wall -Wextra -pedantic fixed_addend_dpk_lck_reference.cpp -o fixed_addend_dpk_lck_reference
```

The core of the released implementation is shown below. The full companion file also contains the brute-force routines and the test driver.

Listing 3: Core exact fixed-addend evaluators.

```
static unsigned majority(unsigned x, unsigned a, unsigned carry)
{
    return (x & a) ^ (x & carry) ^ (a & carry);
}

static std::uint64_t exact_differential_count(
    unsigned n,
    std::uint64_t constant,
    std::uint64_t input_difference,
    std::uint64_t output_difference)
{
    std::array<std::uint64_t, 4> state_count{1, 0, 0, 0};

    for (unsigned i = 0; i < n; ++i)
    {
        std::array<std::uint64_t, 4> next_count{0, 0, 0, 0};
        const unsigned addend_bit = (constant >> i) & 1u;
```

```

const unsigned input_diff_bit = (input_difference >> i) & 1u;
const unsigned output_diff_bit = (output_difference >> i) & 1u;

for (unsigned state = 0; state < 4; ++state)
{
    const auto count = state_count[state];
    if (count == 0) continue;

    const unsigned carry0 = state >> 1;
    const unsigned carry1 = state & 1u;

    for (unsigned x_bit = 0; x_bit <= 1; ++x_bit)
    {
        const unsigned visible_difference = input_diff_bit ^ carry0 ^ carry1;
        if (visible_difference != output_diff_bit) continue;

        const unsigned next_carry0 = majority(x_bit, addend_bit, carry0);
        const unsigned next_carry1 = majority(x_bit ^ input_diff_bit,
                                                addend_bit,
                                                carry1);
        const unsigned next_state = (next_carry0 << 1) | next_carry1;
        next_count[next_state] += count;
    }

    state_count = next_count;
}

return state_count[0] + state_count[1] + state_count[2] + state_count[3];
}

static std::int64_t exact_linear_numerator(
    unsigned n,
    std::uint64_t constant,
    std::uint64_t input_mask,
    std::uint64_t output_mask)
{
    std::array<std::int64_t, 2> signed_mass{1, 0};

    for (unsigned i = 0; i < n; ++i)
    {
        std::array<std::int64_t, 2> next_mass{0, 0};
        const unsigned addend_bit = (constant >> i) & 1u;
        const unsigned input_mask_bit = (input_mask >> i) & 1u;
        const unsigned output_mask_bit = (output_mask >> i) & 1u;

        for (unsigned carry = 0; carry <= 1; ++carry)
        {
            const auto mass = signed_mass[carry];
            if (mass == 0) continue;

            for (unsigned x_bit = 0; x_bit <= 1; ++x_bit)
            {
                const unsigned y_bit = x_bit ^ addend_bit ^ carry;
                const unsigned exponent_bit =
                    (input_mask_bit & x_bit) ^ (output_mask_bit & y_bit);
                const std::int64_t sign = exponent_bit ? -1 : 1;
                const unsigned next_carry = majority(x_bit, addend_bit, carry);
                next_mass[next_carry] += sign * mass;
            }
        }

        signed_mass = next_mass;
    }

    return signed_mass[0] + signed_mass[1];
}

```

Differential side. For every $1 \leq n \leq 6$, every addend $\text{Constant}_{\text{Key}} \in \mathbb{F}_2^n$, and every pair $(u, v) \in \mathbb{F}_2^n \times \mathbb{F}_2^n$, the verifier compares three quantities:

- (D1) the brute-force count of x satisfying $(x \oplus u) + \text{Constant}_{\text{Key}} = (x + \text{Constant}_{\text{Key}}) \oplus v$,
- (D2) the exact four-state count from equation (25),
- (D3) the scalar recurrence count from Theorem 1.

All three agree exhaustively for 299,592 instances, corresponding to 17,895,696 explicit x -evaluations.

Run-law correction. The same verifier was used to test the block theorem. It finds that the stronger statement

$$a_m = a_{m+1} = \dots = a_r$$

is false in general: the first counterexample is $n = 2$, $\text{Constant}_{\text{Key}} = 10_2$, $u = v = 11_2$, where the block reaches the most significant bit. After correcting the theorem to the form stated in Theorem 2, the verifier checks the corrected law exhaustively for all nonzero differentials with $1 \leq n \leq 6$ and finds no counterexample.

Linear side. For every $1 \leq n \leq 6$, every addend $\text{Constant}_{\text{Key}} \in \mathbb{F}_2^n$, and every mask pair $(\alpha, \beta) \in \mathbb{F}_2^n \times \mathbb{F}_2^n$, the verifier compares:

(L1) the brute-force Walsh sum from equation (27),

(L2) the two-state carry-kernel sum from equation (40).

These agree exhaustively for 299,592 instances, again corresponding to 17,895,696 explicit x -evaluations. Taken together with the differential paragraph above, this means that both recovered $O(n)$ evaluators are exhaustively correct on all small spaces with $n \leq 6$: Algorithm 1 matches brute-force differential counting and Algorithm 2 matches brute-force Walsh summation, with zero mismatches in the current verifier.

PHT possibility side. For every $1 \leq n \leq 4$ and every quadruple $(\Delta x_1, \Delta x_2, \Delta y_1, \Delta y_2) \in (\mathbb{F}_2^n)^4$, the verifier compares:

(P1) the direct brute-force verdict that the PHT differential $(\Delta x_1, \Delta x_2 \rightarrow \Delta y_1, \Delta y_2)$ is possible,

(P2) the four-projection verdict from Theorem 5, namely positivity of the projected differentials for PHT_1 , PHT_2 , PHT_1^{-1} , and PHT_2^{-1} (with the fourth test read in the order $(\Delta y_2, \Delta y_1 \rightarrow \Delta x_2)$).

These agree exhaustively for 69,904 instances. This is only a consistency check for the stated criterion, not a substitute for Lipmaa's proof.

Derived one-variable identities. The current verifier also checks three algebraic identities that are derived in the present draft rather than quoted as ready-made formulas from the later literature. First, it verifies exhaustively for all $1 \leq n \leq 8$, all addends $\text{Constant}_{\text{Key}}$, all inputs x , and all bit positions i that the threshold characterization

$$c_i(x) = 1 \iff x \bmod 2^i \geq 2^i - (\text{Constant}_{\text{Key}})_{<i}$$

of equation (42) agrees with the actual carry bit into position i . Second, it verifies exhaustively for the same range that Proposition 10 reproduces every output bit of $x \mapsto x \boxplus \text{Constant}_{\text{Key}}$. Third, it verifies exhaustively for all $1 \leq n \leq 6$, all addends $\text{Constant}_{\text{Key}}$, all masks α , and all output coordinates i that the direct Walsh sum for $(\alpha \rightarrow 2^i)$ agrees with the interval formula of Proposition 12. No mismatch was found in any of these three derived checks.

Extra spot checks. Beyond exhaustive testing up to $n = 6$, the verifier also performs 20,000 random spot checks at $n = 8$ on each side. No mismatch was found.

The point of this appendix is simple: the formulas in Parts II and III are checked against brute force, and the one place where a tempting stronger statement failed was corrected in the main text.

E Appendix D: The 8-State ADP/XOR Matrices

The ADP/XOR line shows that neighboring ARX probability problems can also be expressed as explicit matrix products. Let

$$A_0 = \frac{1}{4} \begin{pmatrix} 4 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix},$$

and for $k \in \{1, \dots, 7\}$ define A_k by XOR-permuting rows and columns, i.e. $(A_k)_{i,j} = (A_0)_{i \oplus k, j \oplus k}$. The full matrices are listed below.

A_0

$$A_0 = \frac{1}{4} \begin{pmatrix} 4 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

A_1

$$A_1 = \frac{1}{4} \begin{pmatrix} 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 4 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

A_2

$$A_2 = \frac{1}{4} \begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 4 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

A_3

$$A_3 = \frac{1}{4} \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 4 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}.$$

A_4

$$A_4 = \frac{1}{4} \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 4 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

 A_5

$$A_5 = \frac{1}{4} \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

 A_6

$$A_6 = \frac{1}{4} \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 4 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

 A_7

$$A_7 = \frac{1}{4} \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 4 \end{pmatrix}.$$

The probability is then

$$\text{adp}_{\oplus}(\alpha, \beta \rightarrow \gamma) = LA_{\omega_{n-1}} \cdots A_{\omega_0} C, \quad L = (1, 1, 1, 1, 1, 1, 1, 1), \quad C = e_0.$$

F Differential and Linear Roles of the Core Papers

To make the role split fully explicit, we record here how each paper contributes to the reconstruction.

- (P1) **Miyano 1996 differential technical report.** Source node for the fixed-addend differential claim; recovered here through later citations and by the fact that its abstract already states dependence on the addend.
- (P2) **Miyano 1996 linear technical report.** Source node for the fixed-addend linear claim; reconstructed here through the abstract pair and Wallen’s predecessor statement.
- (P3) **Miyano 1998 merged letter.** Compresses the two 1996 reports into one conceptual statement.

- (P4) **Lipmaa–Moriai 2001.** Forces the existence of a fixed-addend differential algorithmic skeleton and provides the general two-input carry-difference viewpoint.
- (P5) **Machado 2001.** Exposes the scalar recurrence and the addend-run constraints in the differential half.
- (P6) **Wallen 2003.** Supplies the carry-function classification that acts as the mathematical backbone for the linear half. The full research report also explicitly identifies Miyano 1998 as the predecessor for the one-addend-fixed linear case, which makes it not only a mathematical backbone but also a historical bridge-strengthening node.
- (P7) **Azimi 2020 conference version.** First solver-oriented bit-vector model of fixed-addend differential validity and approximate weight.
- (P8) **Azimi 2022 full version.** Expands the conference version with detailed error analysis and stronger engineering interpretation.
- (P9) **Guan–Ding 2010.** Makes the visible-carry observable $\Delta_{\text{Constant}_{\text{Key}}}(x)$ explicit, solves its spike-addend run law exactly, and converts the resulting dyadic block distribution into an optimal $O(n)$ guess order.
- (P10) **Li et al. 2011.** Eliminates explicit carries from the full two-input coordinate functions, proves the quasi-Bent Walsh law $S_{g_i}(w) \in \{0, \pm 2^{-i}\}$, and shows first-order but not second-order correlation immunity of vectorial addition.
- (P11) **Schulte-Geers 2011.** Supplies a later CCZ/Boolean reformulation in which addition is equivalent to a quadratic shadow, the Lipmaa–Moriai differential formula is re-derived, and the linear support mask becomes explicit via $z = M^t(u \oplus v \oplus w)$.
- (P12) **Lipmaa–Wallen–Dumas 2004 and Mouha 2021.** Show that neighboring ARX probability problems become finite-state automata once the right state representation is found.
- (P13) **Leurent/partitioning, Chaskey, and ChaCha papers.** Downstream consumers showing that local addition laws are attack fuel rather than isolated curiosities.

Supplementary bridges beyond the core papers. Lipmaa’s 2002 PHT paper is not needed to identify the original Miyano source nodes, but it is useful as a bridge citation. It shows that the local addition law can be lifted to Quasi-Hadamard transforms, expressed as block matrix equations, and then applied to PHT and Twofish. In the terminology of Table 6, it demonstrates a coupled-lift extension beyond the one-coordinate case.

Schulte-Geers’ 2011 CCZ-equivalence paper plays a different but equally valuable role. It does not recover Miyano historically, yet it gives a later global reformulation in which addition modulo 2^n is CCZ-equivalent to a quadratic vectorial Boolean function, the differential side collapses to linear equations, and the linear side acquires a completely explicit Walsh formula. For this draft it is therefore best understood as a graph-transport / Booleanized reformulation of the two-input carry-governed addition graph rather than as a new source node.

G Historical Reconstruction Notes

This appendix section collects the longer historical argument that was previously placed in the front half of the paper. Its role is motivational and documentary: it explains why the fixed-addend problem emerges naturally from the RC5 period, how later citations constrain a plausible origin-level reading of the Miyano line, and why the present exact carry-based kernels are historically well motivated. The main theorem statements in the body of the paper do not depend on recovering any literal lost 1996 notation; the appendix is included so that the historical chain remains available without competing with the main technical contribution in the opening pages. We relate these exact kernels to the Miyano–Lipmaa–Machado–Wallen line, but this historical positioning is auxiliary; the paper’s main contribution is the explicit theorem-and-algorithm package above.

We can now compress the historical picture into one sentence, with the important caveat that it is a historical summary rather than a recoverable quotation from 1996. What follows should therefore be read explicitly as a mathematical reverse-engineering exercise: it reconstructs the smallest fixed-addend differential and linear package consistent with the surviving citations and with the exact results proved in the body. It does not claim recovery of Miyano’s literal notation, page order, or documentary wording.

Accordingly, this appendix deliberately carries two origin-level viewpoints that the main text does not need to foreground. The 1996 viewpoint is procedural: fixed-subkey RC5 additions, small-word tables, low-bit carry bookkeeping, and the pressure to discover a reusable prefix recurrence. The 1998 viewpoint is compressive: the differential and linear branches are already close enough to be summarized together as addend dependency of differential/linear probability of addition. That is exactly the burden assigned to the appendix, so that Parts I and II can remain a modern theorem-and-algorithm presentation while the historical chain still appears in full.

Remark 7 (integrated historical summary). The most economical origin-level package consistent with the cited later literature and with the exact results proved in this draft consists of:

- (F1) a four-state carry-pair description for XOR differentials of $x \mapsto x \boxplus \text{Constant}_{\text{Key}}$, equivalently the scalar recurrence of Theorem 1 and the matrix formula of Theorem 3;
- (F2) a two-state carry-conditioned transfer description for fixed-addend linear approximations, equivalently the matrix formula of Theorem 4;
- (F3) explicit dependence of both halves on the actual addend bits, so that the fixed-addend problem cannot be replaced by an average-case two-input formula without losing the object under study.

This is a claim about equivalence class under exact normalization, not a claim about the literal notation of the lost 1996 reports.

G.1 The RC5-Period Fixed-Addend Subproblem

The historical starting point is not a later solver model and not a retrospective abstraction. It is the concrete half-round arithmetic already present in RC5. Kaliski and Yin rewrite RC5 into half-round form,

$$L_1 = L_0 + S_0, \quad R_1 = R_0 + S_1, \quad R_i = ((L_{i-1} \oplus R_{i-1}) \lll R_{i-1}) + S_i,$$

so that each half-round ends with the addition of a *fixed round subkey* [5]. Therefore the local arithmetic object

$$X \mapsto X + S_i \pmod{2^n}$$

is already present inside the cipher before anybody isolates it as a standalone mathematical primitive.

This point matters on both sides. On the differential side, Kaliski and Yin study half-round characteristics while averaging over round keys, so the fixed-addend map is present but not yet separated from the surrounding attack logic [5]. On the linear side, they explicitly state that non-perfect half-round approximations can have key-dependent bias and give a concrete example: for the half-round addition $R_i = Y + S_i$, the approximation $R_i[s] = Y[s] \oplus S_i[s]$ has bias ranging from 0 to $1/2$ and averaging $1/4$ for $s \geq 1$ [5]. Thus addend dependence is already visible in the RC5-period attack literature, even though the fixed-addend map is not yet isolated as a standalone object.

Knudsen and Meier sharpen the same pressure point from the differential side. They explicitly comment that the addition of key words in each half-round can only moderately contribute to the avalanche effect of bit differences, and they exploit this fact to obtain stronger attacks and weak-key phenomena [6]. This does not yet produce an independent theory of fixed-addend addition, but it does show that by 1995–1996 one could not honestly analyse RC5 without facing the arithmetic behaviour of

$$X \mapsto X + \text{Constant}_{\text{Key}} \pmod{2^n}$$

for a fixed key-dependent addend.

G.1.1 How the RC5 Word Object Is Extracted

The key reverse-engineering move is not a literary metaphor. It is an exact factorization step. The RC5 half-round does not present the analyst with a naked unary function at first glance; it presents a larger word transformation ending in addition by a fixed subkey. So the first real technical question is:

what is the exact 32-bit function object sitting at the end of that half-round, once the surrounding wiring is peeled away?

For the two initialization half-rounds, the answer is immediate:

$$L_1 = L_0 + S_0, \quad R_1 = R_0 + S_1.$$

For the later half-round,

$$R_i = ((L_{i-1} \oplus R_{i-1}) \lll R_{i-1}) + S_i,$$

the answer is only one line less obvious: the XOR-and-rotation wrapper first manufactures a word, and the terminal arithmetic step then adds the fixed round subkey to that word.

Proposition 18 (exact extraction of the RC5 32-bit word object). *For each half-round of RC5 there exists a feeder map G_i and a fixed-subkey unary map f_{S_i} such that the updated word is exactly*

$$W_i = f_{S_i}(G_i(\text{state})), \quad f_{S_i}(X) := X + S_i \pmod{2^n}.$$

More precisely:

- (a) *for the first left update one may take $G_0(L_0) = L_0$, so $L_1 = f_{S_0}(L_0)$;*
- (b) *for the first right update one may take $G_1(R_0) = R_0$, so $R_1 = f_{S_1}(R_0)$;*
- (c) *for every later half-round one may take*

$$G_i(L_{i-1}, R_{i-1}) := ((L_{i-1} \oplus R_{i-1}) \lll R_{i-1}),$$

so that

$$R_i = f_{S_i}(G_i(L_{i-1}, R_{i-1})).$$

Consequently, all key-bit dependence of the terminal modular-addition step is concentrated in the unary word map f_{S_i} .

Proof. The first two identities are immediate from

$$L_1 = L_0 + S_0, \quad R_1 = R_0 + S_1.$$

For every later half-round, define

$$G_i(L_{i-1}, R_{i-1}) := ((L_{i-1} \oplus R_{i-1}) \lll R_{i-1}).$$

Substituting this definition into the RC5 update equation gives

$$R_i = G_i(L_{i-1}, R_{i-1}) + S_i = f_{S_i}(G_i(L_{i-1}, R_{i-1})).$$

Thus the half-round is factored exactly into an upstream feeder map and a terminal addition-by-subkey map. Since the feeder map supplies only the variable input word while the fixed subkey enters only through the addition rule, all key-bit dependence of the terminal arithmetic step is concentrated in f_{S_i} . $\square$

This proposition is the precise place where the function object is born. It is not born by abstraction in the vague sense of “let us ignore the rest of RC5”. It is born because the half-round literally factors through a unary modular-addition map on one word.

Once that extraction has been made, the notation of this draft simply renames the concrete round subkey word S_i by the generic symbol $\text{Constant}_{\text{Key}}$, and writes the extracted object uniformly as

$$f_{\text{Constant}_{\text{Key}}}(X) = Y = X + \text{Constant}_{\text{Key}} \pmod{2^N}.$$

Remark 8 (why this extraction is mathematically honest). This extraction does *not* claim that the whole RC5 half-round has been solved. It claims something narrower and exact: the terminal arithmetic component is a standalone unary word map with a fixed addend. In modern language one may treat this as a 32-bit local function object of the RC5 half-round. It is not an S-box in the classical lookup-table sense, but it plays the same object-level role: it is the compact nonlinear component whose exact differential and linear behavior depends on the concrete subkey bits. Reverse-engineering Miyano therefore means reverse-engineering how this extracted word object was analyzed, not pretending that the rest of RC5 disappeared.

Corollary 8 (the exact local RC5 questions that Miyano had to isolate). *Once the RC5 word object has been extracted, the local fixed-key questions become exactly:*

$$\Pr_X[f_{S_i}(X \oplus U) \oplus f_{S_i}(X) = V]$$

on the differential side, and

$$2^{-n} \sum_X (-1)^{\langle \alpha, X \rangle \oplus \langle \beta, f_{S_i}(X) \rangle}$$

on the linear side.

Proof. These are precisely the XOR-differential probability and Walsh correlation of the extracted unary map f_{S_i} . They ask how the terminal addition-by-subkey component behaves when the variable input word X ranges over the word space. $\square$

Remark 9 (why two separate 1996 reports are historically plausible). Once the RC5 word object has been extracted, the road immediately forks. The differential question compares two executions of the same unary map under the same fixed subkey, so the hidden bookkeeping object is a pair of carries. The linear question studies one execution through a signed observable, so the hidden bookkeeping object is a single carry together with cancellation of signs. The arithmetic source is the same, but the working proof machinery is no longer the same. From an exact formulation viewpoint, this is exactly why two parallel reports of moderate length are historically plausible: after the common extraction step, the differential and linear workflows rapidly diverge into two different low-to-high calculi.

G.1.2 The Three Miyano Nodes

The reconstruction therefore begins from three Miyano nodes rather than one isolated citation. The first is the 1996 differential technical report, *Differential Characteristics of Addition upon the Addend Dependency*. The second is the 1996 linear technical report, *Linear Approximation Probability of Addition upon the Addend Dependency*. The third is the 1998 IEICE letter *Addend Dependency of Differential/Linear Probability of Addition*. The right reading posture here is that of a reverse-engineering report, not of a bibliography. We are not asking what notation Miyano literally wrote on the page. We are asking what problem a 1996 analyst actually had on the desk, what elementary tools were realistically available, and what minimal chain of ideas could turn that desk problem into two 15-page technical reports.

In that 1996 setting, one should not imagine a ready-made ARX theory, a generic state-machine package, or a solver model. One should imagine small-width exhaustive tables, hand-written bit recurrences,

elementary conditional probabilities, and repeated low-to-high case splits. The natural first move is to freeze the subkey addend, enumerate the behaviour for small word sizes, and ask a blunt engineering question: *what information about the fixed bit pattern actually changes the probability?* The three Miyano nodes are exactly the documentary trace one would expect if that experiment gradually escaped the RC5 context and became its own object.

The two 1996 abstracts are nearly parallel: the object is the same fixed-addend map

$$f_{\text{Constant}_{\text{Key}}}(X) = Y = X + \text{Constant}_{\text{Key}} \pmod{2^N},$$

the application setting is the same fixed key material used in ciphers such as RC5, and the claim is the same in form—the relevant probability is a function of the addend.

This parallelism already supports a structural reading. The smallest mathematically adequate reading is not that Miyano merely listed empirical key dependencies, but that he isolated one concrete primitive from the RC5 environment and treated it as its own object of analysis. The shortest plausible workflow is: fix the addend, make small tables, observe addend dependence, discover that the low bits decide what the high bits are still allowed to do, and then compress that observation into a tiny running summary. On the differential side, any exact low-to-high evaluation must involve two executions and hence a pair of carries. On the linear side, one execution is approximated and the hidden recursion must involve one carry. The two 1996 titles therefore point to two parallel carry-governed theories attached to the same fixed-addend map.

The later references tighten this constraint. Lipmaa and Moriai state that Miyano studied the simpler case with one addend fixed and derived a linear-time algorithm for the corresponding differential probability [7]. Machado is even more explicit: he writes that [1] “presents several unproved theorems”, leaves their use for calculating the probability to the reader, and gives no algorithm [9]. On the linear side, Wallen’s full 2003 research report states explicitly that “the simpler case with one addend fixed” had previously been considered, and the cited reference is Miyano’s 1998 IEICE paper [11]. Read together, these statements do not identify a literal 1996 notation, but they strongly constrain the reconstruction target: Miyano’s object must centrally concern the one-addend-fixed problem, and any mathematically honest reconstruction must contain a low-bit recursive mechanism capable of producing exact probabilities or correlations as functions of the addend. In reverse-engineering language, the missing payload is not the slogan “depends on the addend”; it is the compression step that turns many low-bit cases into a reusable prefix update rule.

G.2 Why the “One Addend Fixed” Case Was a Natural Explosion Point

The phrase “the simpler case with one addend fixed” should not be read as if the problem were mathematically trivial. It is simpler only in comparison with the full two-input addition problem. In the RC5-period setting, this simplification is exactly what makes the object amenable to direct bit-level analysis: one removes one source of randomness while retaining the entire carry mechanism of modular addition. In other words, the primitive

$$f_{\text{Constant}_{\text{Key}}}(X) = X + \text{Constant}_{\text{Key}} \pmod{2^n}$$

keeps the arithmetic source of nonlinearity but freezes one side into a key-dependent bit pattern.

This is why the fixed-addend problem is historically natural. The RC5 papers already force the analyst to confront additions by concrete round subkeys [5, 6]. The later literature then makes two complementary retrospective statements. First, Lipmaa and Moriai explicitly say that Miyano handled the one-addend-fixed case and obtained a linear-time algorithm for the corresponding differential probability [7]. Second, Wallen treats the one-addend-fixed side as an already existing predecessor on both the differential and linear fronts; in the full 2003 report, the linear chapter states explicitly that the simpler case with one addend fixed had already been considered, with Miyano 1998 as the cited source [10, 11]. Taken together, these statements make it historically natural to isolate the fixed-addend case as the minimal fragment in which exact addend dependence could be studied in its own right.

From that viewpoint, one can now state the reconstruction target more sharply. If Miyano really isolated the fixed-addend map and really obtained a linear-time differential algorithm, then the missing mathematics cannot consist only of ad hoc examples. It must contain a bitwise law strong enough to

propagate information from low bits to high bits without enumerating all 2^n inputs. On the differential side, this means a recursion whose state is determined by the pair of carries generated in the two executions. On the linear side, this means a recursion in which the correlation is accumulated through a carry-conditioned partial Walsh sum. Those are exactly the two kernels that the present reconstruction must recover.

G.2.1 What Must Be Reconstructed if the Gap Is Real

If the RC5 papers expose the fixed-addend subproblem from the front, and the later papers cite Miyano from the back, then the historical gap is not arbitrary. It is a constrained gap. Any mathematically honest reconstruction must explain why all three of the following are simultaneously true.

- (H1) In RC5-period cryptanalysis, fixed-key additions already matter locally and can influence both differential propagation and linear bias [5, 6].
- (H2) Miyano is later cited as having treated the one-addend-fixed case itself, rather than merely having made a passing remark about RC5 [7, 10].
- (H3) Subsequent work in the 2001–2004 bridge then supplies algorithmic or formal shells around the same core object under the present normalization: Lipmaa–Moriai expose the carry-difference viewpoint on the differential side, Machado makes the fixed-addend differential recurrence explicit, Wallen develops the carry classification on the linear side, and Lipmaa–Wallen–Dumas show that neighboring additive-probability problems also become finite-state once the right state representation is found [7, 9, 10, 14].

The consequence is severe. The missing mathematics cannot be a loose collection of observations. It must contain enough local structure to support exact low-to-high propagation. On the differential side, that means a carry-pair law strong enough to decide impossibility, propagate prefix survival, and explain why certain addend runs are forced. On the linear side, that means a one-carry law strong enough to propagate conditioned partial Walsh sums and thereby make the correlation depend on the actual addend bits. Once these two kernels are reconstructed, the later literature admits a coherent reading as a family of later algorithmic or formal shells around the same origin problem.

Put differently, the reverse-engineering target is not just a theorem list. It is a workflow:

- (R1) freeze the addend and compute small-word probability tables;
- (R2) sort the table rows by visible low-bit patterns and ask which rows behave the same;
- (R3) discover that the future only cares about a tiny amount of prefix memory;
- (R4) turn that memory into a low-to-high recurrence;
- (R5) only at the end compress the recurrence into what we now call a carry state machine.

If that workflow cannot be recovered, then the phrase “Miyano handled the one-addend-fixed case” remains historically true but mathematically empty. The present draft therefore treats the workflow itself as part of the reconstruction target.

G.3 How the Origin Should Be Read

G.3.1 Why the Origin Should Not Be Written as if 1996 Already Spoke in Modern Automata

A historical warning is still necessary. The later papers do *not* force the claim that a 1996 author already wrote explicit 4×4 or 2×2 transfer matrices in modern notation. What they force is the existence of a local recursive law. If one wants to stay faithful to the size, tone, and likely toolkit of two short 1996 technical reports, the more plausible discovery path is humbler and more mechanical: first fix the addend, then tabulate low-bit behaviour, then notice repeated values such as 0, $1/2$, and other dyadic contributions, then ask what tiny piece of prefix information keeps reappearing in the tables, and only after that compress those regularities into a hidden summary. The exact finite-state forms used later in this draft should therefore be read as mathematically exact normalizations of an older carry-first recursion, not as claims about the literal surface language of the lost reports.

G.3.2 From a Binary Arithmetic Operation to a Fixed-Addend Map

A more explicit bridge is useful here, because the historical object is simpler than its later reconstructions may suggest.

The starting point is the ordinary binary modular addition

$$\text{Add}_N : \mathbb{Z}_{2^N} \times \mathbb{Z}_{2^N} \rightarrow \mathbb{Z}_{2^N}, \quad \text{Add}_N(X, \text{Constant}_{\text{Key}}) := Y = X + \text{Constant}_{\text{Key}} \pmod{2^N}.$$

From now on, every modular addition or subtraction in this draft that has one variable side and one frozen constant side is written with the same constant symbol $\text{Constant}_{\text{Key}}$. The canonical forward and inverse forms are

$$\begin{aligned} f_{\text{Constant}_{\text{Key}}}(X) &= Y = X + \text{Constant}_{\text{Key}} \pmod{2^N}, \\ f_{\text{Constant}_{\text{Key}}}^{-1}(Y) &= X = Y - \text{Constant}_{\text{Key}} \pmod{2^N}. \end{aligned}$$

Nothing new is created when one passes to the fixed-addend viewpoint. One merely freezes the second input and studies the resulting unary map. Thus one obtains the one-parameter family

$$f_{\text{Constant}_{\text{Key}}} : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_{2^n}, \quad f_{\text{Constant}_{\text{Key}}}(x) := x + \text{Constant}_{\text{Key}} \pmod{2^n}.$$

Equivalently, if one wishes to emphasize the family structure rather than the distinguished symbol $\text{Constant}_{\text{Key}}$, one may write

$$\{f_a : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_{2^n}\}_{a \in \mathbb{Z}_{2^n}}, \quad f_a(x) := x + a \pmod{2^n}, \quad f_a^{-1}(y) := y - a \pmod{2^n}.$$

This is the real mathematical move in the fixed-addend viewpoint. One does not replace binary modular addition by a different object. One re-reads the same rule as a parameterized unary function family and asks how a frozen addend acts on the entire space of inputs. So the passage

$$(X, \text{Constant}_{\text{Key}}) \mapsto X + \text{Constant}_{\text{Key}} \quad \rightsquigarrow \quad X \mapsto X + \text{Constant}_{\text{Key}}$$

is not a change of arithmetic object, but only a change of viewpoint: one input is held fixed and the other is allowed to vary.

This change of viewpoint matters because differential and linear cryptanalysis are not interested in one isolated sum. They study the behaviour of the entire map over all inputs. Once the constant is frozen, the relevant questions become properties of the fixed-addend function object and its inverse presentation.

At this stage the reverse-engineering should still resist jumping to carries. The extracted RC5 word object first appears not as a hidden-state system but as a *pair* of unary functions:

$$f_{\text{Constant}_{\text{Key}}}(X) = Y = X + \text{Constant}_{\text{Key}} \pmod{2^N}, \quad f_{\text{Constant}_{\text{Key}}}^{-1}(Y) = X = Y - \text{Constant}_{\text{Key}} \pmod{2^N}.$$

The natural question is therefore not yet “what is the carry recursion?”, but rather “how should these two functions be attacked before any hidden object is introduced?”

Proposition 19 (algebraic pair of forward and inverse translations). *For every fixed $\text{Constant}_{\text{Key}} \in \mathbb{Z}_{2^N}$, the maps $f_{\text{Constant}_{\text{Key}}}$ and $f_{\text{Constant}_{\text{Key}}}^{-1}$ are mutual inverses. More precisely,*

$$f_{\text{Constant}_{\text{Key}}}^{-1}(f_{\text{Constant}_{\text{Key}}}(X)) = X, \quad f_{\text{Constant}_{\text{Key}}}(f_{\text{Constant}_{\text{Key}}}^{-1}(Y)) = Y$$

for all $X, Y \in \mathbb{Z}_{2^N}$. Hence both maps are translations of the cyclic group $\mathbb{Z}_{2^N}$, one by $+\text{Constant}_{\text{Key}}$ and the other by $-\text{Constant}_{\text{Key}}$.

Proof. By definition,

$$f_{\text{Constant}_{\text{Key}}}^{-1}(f_{\text{Constant}_{\text{Key}}}(X)) = (X + \text{Constant}_{\text{Key}}) - \text{Constant}_{\text{Key}} \equiv X \pmod{2^N},$$

and similarly

$$f_{\text{Constant}_{\text{Key}}}(f_{\text{Constant}_{\text{Key}}}^{-1}(Y)) = (Y - \text{Constant}_{\text{Key}}) + \text{Constant}_{\text{Key}} \equiv Y \pmod{2^N}.$$

Thus the two maps are mutual inverses, and both are translations in $\mathbb{Z}_{2^N}$. $\square$

This is the first forced perspective shift. The terminal RC5 step is no longer read as one line inside a round update, but as a forward/inverse pair of 32-bit unary function objects. In reverse-engineering language, this is the point where the raw RC5 formula becomes an analyzable object.

The gain is conceptual but decisive. A single expression $X + \text{Constant}_{\text{Key}}$ is too small an object for later differential and linear measurements. Those measurements ask how the same frozen rule behaves on all inputs and, equally importantly, how it can be read back from all outputs. Once the fixed-addend step has been promoted to a forward/inverse function pair, later cryptanalytic quantities become measurements of one common object rather than unrelated tricks attached to one arithmetic symbol.

From there, at least three further perspective shifts occur before carry becomes unavoidable:

- (S1) **Algebraic shift.** One first studies $f_{\text{Constant}_{\text{Key}}}$ and $f_{\text{Constant}_{\text{Key}}}^{-1}$ as translation objects and permutation objects, without introducing any hidden memory.
- (S2) **Probabilistic shift.** One next treats those maps as sources of events over a uniform finite sample space, asking which event probabilities depend on the fixed constant.
- (S3) **Observable shift.** One then asks which measurable cryptanalytic observables are attached to the graph of the function pair: XOR-differential events and signed linear imbalance.

Only after these three shifts does a deeper question become unavoidable: what hidden quantity explains those observables while surviving low-to-high propagation? The first successful answer to that question is the carry object.

This is why the fixed-addend viewpoint is historically natural in the RC5 period. During a local round step, the round subkey is fixed while the data word varies. The analyst is therefore led not merely to the arithmetic expression $x + \text{Constant}_{\text{Key}}$, but to the behaviour of the one-variable map

$$x \mapsto x + \text{Constant}_{\text{Key}} \pmod{2^n},$$

viewed over all inputs and, equally importantly, to its inverse output-side presentation

$$y \mapsto y - \text{Constant}_{\text{Key}} \pmod{2^n}.$$

In plain language: the object is still the same old modular addition, but once one side is frozen, the real question is no longer “what is this one sum?” but “what pattern does this fixed addition rule impose on every possible input and every possible output description?” The later carry recurrences should therefore be read as a late compression of that object, not as the first move.

G.3.3 A Probability-First View of the Fixed-Addend Function Pair

Before separating differential and linear questions, one may look at the fixed-addend function pair in a more primitive way: as a source of events over a finite probability space.

Fix $\text{Constant}_{\text{Key}} \in \mathbb{Z}_{2^n}$ and define

$$f_{\text{Constant}_{\text{Key}}} : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_{2^n}, \quad f_{\text{Constant}_{\text{Key}}}(x) = x + \text{Constant}_{\text{Key}} \pmod{2^n}.$$

Equally,

$$f_{\text{Constant}_{\text{Key}}}^{-1} : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_{2^n}, \quad f_{\text{Constant}_{\text{Key}}}^{-1}(y) = y - \text{Constant}_{\text{Key}} \pmod{2^n}.$$

Once the input x is taken uniformly from $\mathbb{Z}_{2^n}$, every predicate depending on the input, the output, or both,

$$\mathcal{E}(x, f_{\text{Constant}_{\text{Key}}}(x)) \in \{0, 1\},$$

defines an event probability

$$\Pr_{\text{Constant}_{\text{Key}}}[\mathcal{E}] := 2^{-n} \# \{x \in \mathbb{Z}_{2^n} : \mathcal{E}(x, f_{\text{Constant}_{\text{Key}}}(x)) \text{ holds}\}.$$

Because $f_{\text{Constant}_{\text{Key}}}$ is bijective, the same event can equally be counted from the output side:

$$\Pr_{\text{Constant}_{\text{Key}}}[\mathcal{E}] = 2^{-n} \# \{y \in \mathbb{Z}_{2^n} : \mathcal{E}(f_{\text{Constant}_{\text{Key}}}^{-1}(y), y) \text{ holds}\}.$$

So before one speaks about differentials or linear masks, the fixed-addend map already determines a family of measurable events over a uniform input space. At this stage the map is not yet a differential object and not yet a Walsh object. It is first an event source: a rule that turns the uniform space $\mathbb{Z}_{2^n}$ into a collection of input/output relations whose probabilities can be measured from either side.

Proposition 20 (fixed-addend addition as a permutation source). *For every fixed $\text{Constant}_{\text{Key}} \in \mathbb{Z}_{2^n}$, both $f_{\text{Constant}_{\text{Key}}}$ and $f_{\text{Constant}_{\text{Key}}}^{-1}$ are permutations of $\mathbb{Z}_{2^n}$. In particular, if x is uniform in $\mathbb{Z}_{2^n}$, then $f_{\text{Constant}_{\text{Key}}}(x)$ is also uniform in $\mathbb{Z}_{2^n}$, and if y is uniform in $\mathbb{Z}_{2^n}$, then $f_{\text{Constant}_{\text{Key}}}^{-1}(y)$ is also uniform in $\mathbb{Z}_{2^n}$.*

Proof. Addition by a fixed element in the additive group $\mathbb{Z}_{2^n}$ is a translation, hence bijective. The same is true for subtraction by that fixed element. $\square$

Proposition 20 has an important negative consequence. If one throws away the input and looks only at the marginal law of $Y = f_{\text{Constant}_{\text{Key}}}(X)$, then nothing interesting remains: Y is uniform again. Therefore the addend-dependent structure cannot live in the output distribution alone. The nontrivial structure must lie in *joint* events involving the relation between x and $f_{\text{Constant}_{\text{Key}}}(x)$, or equivalently between $f_{\text{Constant}_{\text{Key}}}^{-1}(y)$ and y . At this stage no hidden carry variable is needed: we are still only looking at the function pair as a measure-preserving source of events.

The two later cryptanalytic specializations are exact refinements of this probability-first viewpoint. They are not arbitrary decorations added later by taste. They are two precise ways of measuring the joint structure already generated by the same fixed-addend event source.

Differential specialization. For fixed (U, V) , the differential probability is the event probability

$$\Pr_x[f_{\text{Constant}_{\text{Key}}}(x \oplus U) \oplus f_{\text{Constant}_{\text{Key}}}(x) = V].$$

Linear specialization. For fixed masks (α, β) , the linear correlation is the signed imbalance between the complementary events

$$\langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle = 0 \quad \text{and} \quad \langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle = 1.$$

Equivalently,

$$\text{CORRELATION}(\alpha \rightarrow \beta) = \Pr_x[\langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle = 0] - \Pr_x[\langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle = 1].$$

Thus differential probabilities and linear correlations should be read as two precise ways of measuring the occurrence, or signed imbalance, of events generated by the same fixed-addend map.

G.3.4 An Observable View of the Function Graph Before Carry

The third forced shift is from “a function acting on points” to “observable structure living on the graph of the function pair”. Define the graph

$$\Gamma_{\text{Constant}_{\text{Key}}} := \{(X, Y) \in \mathbb{Z}_{2^n}^2 : Y = f_{\text{Constant}_{\text{Key}}}(X)\} = \{(f_{\text{Constant}_{\text{Key}}}^{-1}(Y), Y) : Y \in \mathbb{Z}_{2^n}\}.$$

At this level, differential and linear analysis should first be read as measurements on $\Gamma_{\text{Constant}_{\text{Key}}}$, still before any hidden carry summary is introduced. The point is no longer merely that the map generates events. The point is that those events live on one common graph in $\mathbb{Z}_{2^n}^2$, and that the forward-side and inverse-side descriptions are only two coordinate readings of the same object.

Proposition 21 (forward-side and inverse-side observables). *For every fixed $\text{Constant}_{\text{Key}}$, every difference pair $U, V \in \mathbb{Z}_{2^n}$, and every mask pair $\alpha, \beta \in \mathbb{F}_2^n$, one has*

$$\Pr_X[f_{\text{Constant}_{\text{Key}}}(X \oplus U) \oplus f_{\text{Constant}_{\text{Key}}}(X) = V] = \Pr_Y[f_{\text{Constant}_{\text{Key}}}^{-1}(Y \oplus V) \oplus f_{\text{Constant}_{\text{Key}}}^{-1}(Y) = U], \quad (65)$$

and

$$2^{-n} \sum_X (-1)^{\langle \alpha, X \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(X) \rangle} = 2^{-n} \sum_Y (-1)^{\langle \alpha, f_{\text{Constant}_{\text{Key}}}^{-1}(Y) \rangle \oplus \langle \beta, Y \rangle}. \quad (66)$$

Proof. For equation (65), substitute $Y = f_{\text{Constant}_{\text{Key}}}(X)$. Since $f_{\text{Constant}_{\text{Key}}}$ is a permutation, the change of variables is bijective, and

$$f_{\text{Constant}_{\text{Key}}}(X \oplus U) \oplus f_{\text{Constant}_{\text{Key}}}(X) = V$$

holds if and only if

$$X \oplus U = f_{\text{Constant}_{\text{Key}}}^{-1}(Y \oplus V), \quad X = f_{\text{Constant}_{\text{Key}}}^{-1}(Y),$$

that is,

$$f_{\text{Constant}_{\text{Key}}}^{-1}(Y \oplus V) \oplus f_{\text{Constant}_{\text{Key}}}^{-1}(Y) = U.$$

Equation (66) is the same bijective change of variables applied to the Walsh sum. $\square$

Proposition 21 is important for the reverse-engineering story. The forward differential event and the inverse differential event are not two different mechanisms; they are the same observable written in two coordinate systems on $\Gamma_{\text{Constant}_{\text{Key}}}$. The Walsh identity says the same for the signed linear observable. Before carry enters, the analyst can therefore already attack the extracted RC5 word object from two directions: input-side forward translation and output-side inverse translation. Differential and linear questions are first graph observables of the function pair, not yet hidden-state questions.

G.3.5 An Algebraic Decomposition by Simpler Operations

A fourth shift now occurs. Only after the algebraic, probabilistic, and observable views are on the table does it become natural to ask for a hidden correction term that explains those observables with a low-to-high recursive law. This is the point where the carry object finally enters.

The map

$$f_{\text{Constant}_{\text{Key}}}(x) = x + \text{Constant}_{\text{Key}} \pmod{2^n}$$

is more complicated than the coordinatewise XOR rule

$$g_{\text{Constant}_{\text{Key}}}^{(0)}(x) := x \oplus \text{Constant}_{\text{Key}},$$

but the correct historical reading is not that the simple rule *is* the complex map. The point is subtler: one tries to understand the complex map by a hierarchy of simpler surrogates, each of which captures part of the behaviour and each of which fails in a controlled way.

The reason this first surrogate is natural is exact rather than heuristic. If one suppresses carries, binary addition collapses to coordinatewise XOR. So $g_{\text{Constant}_{\text{Key}}}^{(0)}$ is the carry-free affine skeleton of the true map. What remains to be explained is the exact correction term that measures how true modular addition departs from that skeleton.

The first exact identity is

$$f_{\text{Constant}_{\text{Key}}}(x) = x \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x),$$

where $\text{Carry}_{\text{Constant}_{\text{Key}}}(x)$ is the carry word generated by adding x and $\text{Constant}_{\text{Key}}$. Thus the XOR-like rule $g_{\text{Constant}_{\text{Key}}}^{(0)}(x) = x \oplus \text{Constant}_{\text{Key}}$ is the simplest global surrogate, and the entire gap between that surrogate and true modular addition is the carry correction.

Proposition 22 (global surrogate and exact correction). *For every fixed constant $\text{Constant}_{\text{Key}}$ and every input $x \in \mathbb{Z}_{2^n}$,*

$$f_{\text{Constant}_{\text{Key}}}(x) = g_{\text{Constant}_{\text{Key}}}^{(0)}(x) \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x), \quad g_{\text{Constant}_{\text{Key}}}^{(0)}(x) = x \oplus \text{Constant}_{\text{Key}}.$$

Hence a first algebraic approximation to fixed-addend modular addition is obtained by the simple affine map $x \mapsto x \oplus \text{Constant}_{\text{Key}}$, and the exact error term is the carry word $\text{Carry}_{\text{Constant}_{\text{Key}}}(x)$.

Proof. Write

$$x = \sum_{i=0}^{n-1} x_i 2^i, \quad \text{Constant}_{\text{Key}} = \sum_{i=0}^{n-1} a_i 2^i, \quad f_{\text{Constant}_{\text{Key}}}(x) = \sum_{i=0}^{n-1} y_i 2^i,$$

and let c_i denote the carry into bit i , with $c_0 = 0$. Then ordinary binary addition gives the bit recursions

$$y_i = x_i \oplus a_i \oplus c_i, \quad c_{i+1} = \text{Maj}(x_i, a_i, c_i).$$

If the carry word is written in the same indexing convention as

$$\text{Carry}_{\text{Constant}_{\text{Key}}}(x) = \sum_{i=0}^{n-1} c_i 2^i,$$

then assembling the bit equations over all positions yields

$$f_{\text{Constant}_{\text{Key}}}(x) = x \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x).$$

Thus the simple affine rule $x \mapsto x \oplus \text{Constant}_{\text{Key}}$ captures the carry-free skeleton, and the exact discrepancy is the carry word. $\square$

But the stronger point is that no *single* simple surrogate explains the whole map. The complexity is not erased; it is only relocated into the rule selecting which local correction is active. In other words, the nonlinearity of fixed-addend modular addition is concentrated in the switching law

$$x \mapsto \text{Carry}_{\text{Constant}_{\text{Key}}}(x),$$

which decides which affine correction must be applied to the global XOR skeleton. This leads to a hierarchy of approximations.

Definition 2 (carry classes). For each carry word $\gamma \in \{0, 1\}^n$, define the carry class

$$X_\gamma(\text{Constant}_{\text{Key}}) := \{x \in \mathbb{Z}_{2^n} : \text{Carry}_{\text{Constant}_{\text{Key}}}(x) = \gamma\}.$$

Proposition 23 (many simple pieces, not one global simplification). *For fixed $\text{Constant}_{\text{Key}}$, the input space $\mathbb{Z}_{2^n}$ is partitioned by the carry classes $X_\gamma(\text{Constant}_{\text{Key}})$. On each nonempty class one has the exact affine rule*

$$f_{\text{Constant}_{\text{Key}}}(x) = x \oplus \text{Constant}_{\text{Key}} \oplus \gamma \quad \text{for all } x \in X_\gamma(\text{Constant}_{\text{Key}}).$$

Thus the fixed-addend map is not globally replaced by one simple operation. Rather, it is explored by many simple affine pieces, each valid on its own carry class, while the true complexity lies in the switching law $x \mapsto \text{Carry}_{\text{Constant}_{\text{Key}}}(x)$ that decides which piece is active.

Proof. The classes $X_\gamma(\text{Constant}_{\text{Key}})$ form a partition by definition. If $x \in X_\gamma(\text{Constant}_{\text{Key}})$, then $\text{Carry}_{\text{Constant}_{\text{Key}}}(x) = \gamma$, and Proposition 22 gives

$$f_{\text{Constant}_{\text{Key}}}(x) = x \oplus \text{Constant}_{\text{Key}} \oplus \gamma.$$

$\square$

This is the intended algebraic viewpoint. One does not claim that a single simple operation equals the complex modular-addition map. One instead builds a ladder of simpler descriptions—global XOR-like surrogates, carry-conditioned affine pieces, prefix-conditioned low-bit models—and uses them to feel out the structure of the true map from different angles. On each fixed carry class the map is affine; what is hard is not the local rule inside a class, but the law that decides which class an input enters. The object remains complex; the analysis becomes possible because the complexity is encountered through many local simple rules rather than one impossible global simplification.

This is also why the next subsection follows so quickly. Once

$$f_{\text{Constant}_{\text{Key}}}(x) = x \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x)$$

has been isolated, both the differential and linear bridge identities become direct substitutions into two different observables. In that sense the carry-correction identity is the mother formula from which both later specializations descend.

One caution is still needed. The carry word $\text{Carry}_{\text{Constant}_{\text{Key}}}(x)$ is already the correct hidden correction, but as a whole word it is still too large to be the eventual low-to-high state. The next descent is therefore no longer from addition to carry, but from the full carry word to the smallest prefix memory that each observable actually needs while the adder is scanned from low bits upward. Because the differential observable compares two executions, it will inherit two carry histories; because the linear observable weights one execution by signs, it will inherit only one. Section 3.6 writes those two specializations explicitly, and the later core parts then compress them further to a carry pair and a single carry.

G.3.6 Two Exact Bridge Identities: Differential and Linear

The previous subsection gives the exact structural identity

$$f_{\text{Constant}_{\text{Key}}}(x) = x \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x).$$

From this single identity, both the differential and linear problems emerge by exact specialization.

Proposition 24 (exact differential bridge identity). *For every fixed constant $\text{Constant}_{\text{Key}}$ and every $x, U \in \mathbb{Z}_{2^n}$,*

$$f_{\text{Constant}_{\text{Key}}}(x \oplus U) \oplus f_{\text{Constant}_{\text{Key}}}(x) = U \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x \oplus U) \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x).$$

Consequently,

$$\Pr_x[f_{\text{Constant}_{\text{Key}}}(x \oplus U) \oplus f_{\text{Constant}_{\text{Key}}}(x) = V] = \Pr_x[\text{Carry}_{\text{Constant}_{\text{Key}}}(x \oplus U) \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x) = U \oplus V].$$

Proof. Using

$$f_{\text{Constant}_{\text{Key}}}(z) = z \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(z),$$

once for $z = x \oplus U$ and once for $z = x$, one obtains

$$\begin{aligned} f_{\text{Constant}_{\text{Key}}}(x \oplus U) \oplus f_{\text{Constant}_{\text{Key}}}(x) &= (x \oplus U \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x \oplus U)) \oplus (x \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x)) \\ &= U \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x \oplus U) \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x). \end{aligned}$$

The probability identity follows immediately. $\square$

Proposition 24 shows that fixed-addend differential analysis is the study of how two carry words, generated under the same fixed constant $\text{Constant}_{\text{Key}}$, disagree when the input is perturbed by U .

This is already enough to force the shape of the later differential hidden state. The differential observable does not inspect one execution of $f_{\text{Constant}_{\text{Key}}}$, but two parallel executions on the related inputs x and $x \oplus U$. Once the common affine skeleton has cancelled, what remains is not an arbitrary residue of the processed prefix, but exactly the XOR of two carry corrections. So every later low-to-high differential calculus must, in one form or another, follow two carry chains in parallel: one for x and one for $x \oplus U$.

Proposition 25 (exact linear bridge identity). *For every fixed constant $\text{Constant}_{\text{Key}}$, every input $x \in \mathbb{Z}_{2^n}$, and every mask pair $\alpha, \beta \in \{0, 1\}^n$,*

$$\langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle = \langle \alpha \oplus \beta, x \rangle \oplus \langle \beta, \text{Constant}_{\text{Key}} \rangle \oplus \langle \beta, \text{Carry}_{\text{Constant}_{\text{Key}}}(x) \rangle.$$

Consequently,

$$\text{CORRELATION}(\alpha \rightarrow \beta) = (-1)^{\langle \beta, \text{Constant}_{\text{Key}} \rangle} 2^{-n} \sum_x (-1)^{\langle \alpha \oplus \beta, x \rangle \oplus \langle \beta, \text{Carry}_{\text{Constant}_{\text{Key}}}(x) \rangle}.$$

Proof. Substitute

$$f_{\text{Constant}_{\text{Key}}}(x) = x \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x)$$

into the parity expression:

$$\langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle = \langle \alpha, x \rangle \oplus \langle \beta, x \oplus \text{Constant}_{\text{Key}} \oplus \text{Carry}_{\text{Constant}_{\text{Key}}}(x) \rangle$$

$$\begin{aligned}
&= \langle \alpha, x \rangle \oplus \langle \beta, x \rangle \oplus \langle \beta, \text{Constant}_{\text{Key}} \rangle \oplus \langle \beta, \text{Carry}_{\text{Constant}_{\text{Key}}}(x) \rangle \\
&= \langle \alpha \oplus \beta, x \rangle \oplus \langle \beta, \text{Constant}_{\text{Key}} \rangle \oplus \langle \beta, \text{Carry}_{\text{Constant}_{\text{Key}}}(x) \rangle.
\end{aligned}$$

Summing $(-1)^{(\cdot)}$ over x yields the stated formula. $\square$

Proposition 25 shows that fixed-addend linear analysis is controlled by the same hidden core. After peeling off the easiest affine skeleton, the nontrivial signed bias is governed by the parity interaction between the input mask $\alpha \oplus \beta$ and the carry word $\text{Carry}_{\text{Constant}_{\text{Key}}}(x)$.

The linear branch therefore separates from the differential branch for an exact structural reason. Here there is no second execution to compare against the first one. There is only one sampled input x , one carry word $\text{Carry}_{\text{Constant}_{\text{Key}}}(x)$, and one signed observable that tests its interaction with the masks. After the visible affine contribution has been peeled away, the hidden object is therefore not a carry pair but a single carry chain viewed through a sign pattern.

Together, Propositions 24 and 25 close the main historical bridge. Differential and linear analyses are not separate miracles attached to the same addition symbol. They are two exact specializations of one hidden carry-driven correction term inside the same fixed-addend map.

They also explain why the later normal forms have exactly the sizes they do. The differential bridge duplicates the fixed-addend map and therefore duplicates the hidden carry correction, so the natural exact low-bit summary is one carry from each execution. The linear bridge keeps only one execution and weights it by signs, so the natural exact low-bit summary is one carry. Parts II and III simply cash out these two forced compressions: first the carry correction appears as a word, then as one or two evolving carry bits, and only after that as the familiar 4-state and 2-state systems.

There is therefore no historical or mathematical jump left to explain. After Section 3.5, one knows that the hidden correction is the carry word. After Section 3.6, one knows how that correction enters the two observables and why the differential side duplicates it while the linear side does not. The remaining task is purely one of prefix compression: determine how much of the processed carry history still matters once the adder is read from low bits to high bits. The answer will be exactly one carry pair on the differential side and exactly one carry on the linear side.

G.3.7 A 1996-Style Phenomenology of Fixed-Addend Differential Probability

Fix the constant $\text{Constant}_{\text{Key}}$ and consider

$$\Pr_X [(X + \text{Constant}_{\text{Key}}) \oplus ((X \oplus U) + \text{Constant}_{\text{Key}}) = V].$$

If one forces oneself back into a 1996 toolbox, the first artifact is not a theorem and certainly not a polished automaton. The first artifact is a table. One fixes $\text{Constant}_{\text{Key}}$, chooses a very small word size, enumerates all X , and records the exact probability for each visible pair (U, V) . What jumps out from such a table is not elegance, but stubborn structure: the probability is not governed by (U, V) alone. The actual bit pattern of $\text{Constant}_{\text{Key}}$ interferes with the propagation, and it does so already at the lowest positions.

For a concrete toy instance one may freeze $n = 3$ and $a = 001_2$, define

$$f_a(x) := x \boxplus a, \quad \text{DP}_a(u \rightarrow v) = 2^{-n} \# \{x \in \mathbb{F}_2^n : f_a(x \oplus u) \oplus f_a(x) = v\},$$

and rewrite the visible differential relation as

$$f_a(x \oplus u) \oplus f_a(x) = u \oplus \text{Car}_a(x \oplus u) \oplus \text{Car}_a(x), \quad v_i = u_i \oplus c_i \oplus c'_i,$$

where c_i, c'_i are the carries into bit i in the two parallel executions. The standalone verifier from Appendix C prints the full 8×8 table for this toy case:

Even at this width, the first three symptoms are already visible: rigid zero patterns, a tiny dyadic palette, and support that is visibly shaped by the fixed addend. More sharply, one can keep the same visible input difference and change only the addend. For $n = 4$ and $u = 0100_2$, the verifier prints the complete row comparison below:

Table 1: Toy XOR-differential table for $f_a(x) = x \boxplus a$ with $n = 3$ and $a = 001_2$. A dot denotes zero probability.

$u \backslash v$	000	001	010	011	100	101	110	111
000	8/8	.	.	.	.	.	.	.
001	.	.	.	4/8	.	.	.	4/8
010	.	.	4/8	.	.	.	4/8	.
011	.	4/8	.	.	.	4/8	.	.
100	.	.	.	.	8/8	.	.	.
101	.	.	.	4/8	.	.	.	4/8
110	.	.	4/8	.	.	.	4/8	.
111	.	4/8	.	.	.	4/8	.	.

Table 2: Addend-dependency witness: the row for the fixed visible input difference $u = 0100_2$ changes when only the addend changes.

v	$a = 0001_2$	$a = 0011_2$
0000	0/16	0/16
0001	0/16	0/16
0010	0/16	0/16
0011	0/16	0/16
0100	12/16	4/16
0101	0/16	0/16
0110	0/16	0/16
0111	0/16	0/16
1000	0/16	0/16
1001	0/16	0/16
1010	0/16	0/16
1011	0/16	0/16
1100	4/16	12/16
1101	0/16	0/16
1110	0/16	0/16
1111	0/16	0/16

Nothing about the visible difference u changed, yet the mass moves from $v = 0100_2$ to $v = 1100_2$ when one replaces $a = 0001_2$ by $a = 0011_2$. This is the concrete table-level witness that the differential row is a function of the addend, not of u alone.

These toy outputs make the point concrete. This becomes visible in three stages.

- (D1) **Immediate zero patterns.** Some local patterns kill the differential at once. The least-significant-bit condition $u_0 = v_0$ is the simplest example. More generally, certain short visible patterns force impossibility as soon as the next carry is examined.
- (D2) **A sparse local probability palette.** When impossibility does not occur, the local contribution is often very small and highly structured: one repeatedly encounters values such as 1, $1/2$, and more generally dyadic factors 2^{-t} attached to a short block. This is exactly the sort of phenomenon that a short 1996 report could detect by hand from low-bit case tables before any grand formalism existed.
- (D3) **Rigidity along runs.** If the differential survives through a long stretch where $u_i = v_i = 1$, then the addend bits in the corresponding region can no longer vary freely. Survival forces a rigid run condition on the addend. Thus the addend is not a passive parameter; its local shape actively decides whether the differential can continue.

The later Guan–Ding paper is useful here because it exhibits exactly this kind of one-variable, addend-dependent behaviour in a visible form. Their quantity

$$\Delta_{\text{Constant}_{\text{Key}}}(X) := (X + \text{Constant}_{\text{Key}}) \bmod 2^n \oplus X \oplus \text{Constant}_{\text{Key}}$$

is simply the carry word made visible. For special constants such as $\text{Constant}_{\text{Key}} \in A_i = \{2^i, 2^{n-1} + 2^i\}$, its value is not an arbitrary pattern: it is always a contiguous carry block beginning at bit $i + 1$, and the block length is exactly the consecutive-one run length of the suffix of X starting at bit i . The resulting probabilities are therefore dyadic,

$$\frac{1}{2}, \quad \frac{1}{4}, \quad \frac{1}{8}, \quad \dots,$$

up to the top-boundary truncation made explicit later in Theorem 6. That is the same style of phenomenon: fixed addend, carry-governed propagation, and probability determined by local run structure.

What one is naturally led to seek, therefore, is not yet a state machine but a hand-drivable recurrence: a low-to-high update rule that distinguishes impossible local patterns, records the dyadic losses contributed by surviving steps, and explains why certain addend runs are forced. Only after such a recurrence has been isolated does it become reasonable to compress it into a smaller exact formalism. In reverse-engineering terms, the carry state machine is the endpoint of the 1996 differential workflow, not its starting point.

G.3.8 A 1996-Style Phenomenology of Fixed-Addend Linear Probability

The linear side should be read in the same way. For fixed $\text{Constant}_{\text{Key}}$, one studies the correctness probability or correlation of an approximation to

$$f_{\text{Constant}_{\text{Key}}}(X) = Y = X + \text{Constant}_{\text{Key}} \pmod{2^N}.$$

Again, the first discovery is not a polished transfer device. The first discovery is that the quality of a mask pair depends on the actual bit pattern of the fixed addend. A researcher with 1996 tools would not start from Walsh matrices. The natural move is to write down the sign for small word sizes, split on the low bits, and watch where cancellation starts and where it fails.

In modern notation the relevant sign is

$$(-1)^{\langle \alpha, X \rangle \oplus \langle \beta, X + \text{Constant}_{\text{Key}} \rangle}.$$

For a concrete toy instance one may again freeze $n = 3$ and $a = 001_2$, and define the exact Walsh sum

$$W_a(\alpha, \beta) := \sum_{x \in \mathbb{F}_2^n} (-1)^{\langle \alpha, x \rangle \oplus \langle \beta, f_a(x) \rangle} = 2^n \text{CORRELATION}_a(\alpha \rightarrow \beta).$$

The standalone verifier from Appendix C prints the full 8×8 toy table for this case:

Table 3: Toy Walsh table for $f_a(x) = x \boxplus a$ with $n = 3$ and $a = 001_2$. A dot denotes zero Walsh sum, so the only observed values are 0, ± 4 , and ± 8 .

$\alpha \backslash \beta$	000	001	010	011	100	101	110	111
000	8	.	.	.	.	.	.	.
001	.	-8	.	.	.	.	.	.
010	.	.	.	-8	.	.	.	.
011	.	.	8	.	.	.	.	.
100	.	.	.	.	4	-4	-4	-4
101	.	.	.	.	4	-4	4	4
110	.	.	.	.	4	4	4	-4
111	.	.	.	.	-4	-4	4	-4

Even at this width, the linear side already shows the same addend-sensitive rigidity: most mask pairs vanish, the surviving values live on a tiny dyadic palette after normalization by 2^{-n} , and the sign pattern is visibly shaped by the fixed addend. One can make the addend dependence completely explicit by keeping the visible mask pair fixed and changing only the addend. For example,

$$W_{0001_2}(0011_2, 0010_2) = 16, \quad W_{0011_2}(0011_2, 0010_2) = -16.$$

So the same visible mask pair flips from perfect positive correlation to perfect negative correlation when only the addend is changed.

If one examines this sign from the least significant bit upward, then it quickly becomes clear that higher-bit contributions are not independent of lower ones: the low bits create carries, and those carries decide whether later signs reinforce or cancel. In other words, the linear report is not studying a different arithmetic world. It is staring at the same carry chain, but through a signed observable rather than a survival probability.

A historically plausible first reading is therefore the following.

- (L1) one can maintain a very small running summary while scanning bits from low to high;
- (L2) this summary determines whether later signed contributions add or destroy each other;
- (L3) the summary itself depends on the fixed addend through the same carry law that already appears on the differential side.

So the linear phenomenon is parallel to the differential one, but with a different observable. In the differential case one asks whether two related executions survive together. In the linear case one asks whether the signed contributions produced by one execution accumulate or annihilate each other. The addend controls both through the carry chain.

Thus the natural object is not yet an external formalism, but a small carry-driven recurrence for partial signed sums. Once such a recurrence has been recognized, it may be written more compactly in a two-component form. Here too the state machine should be read historically as the compressed output of repeated low-bit bookkeeping.

G.3.9 From Addend-Dependent Probability Tables to Prefix Recurrences

The two 1996 abstracts imply more than mere existence of addend dependence. They strongly suggest that Miyano had some form of *prefix recurrence*. Otherwise the statement “to detect a good differential characteristics” or “to detect a good linear approximation” would be far too weak. One needs a computational mechanism that updates quality while the low bits of the addend are revealed.

This is why the present draft uses the language of *partial probabilities* and *partial correlations*. A plausible 1996 discovery path is:

- (P1) compute the behaviour for very small word sizes;
- (P2) print or tabulate the exact values and group together cases that look locally identical;

- (P3) discover that after one conditions on a tiny prefix summary, the suffix asks the same question again;
- (P4) define that summary variable together with a local multiplier or signed update;
- (P5) iterate this summary from low bits to high bits to obtain a linear-time algorithm.

The verifier-backed toy data make step (P3) unusually explicit. For the surviving toy differential

$$(a, u, v) = (0001_2, 0001_2, 0111_2)$$

at $n = 4$, exactly four values of x survive. Group those four survivors at each level by the current carry pair (c_i, c'_i) :

Table 4: Carry-pair slice counts for the four surviving inputs of the toy differential $(a, u, v) = (0001_2, 0001_2, 0111_2)$ at $n = 4$.

level i	u_i	v_i	$u_i \oplus v_i$	00	01	10	11
0	1	1	0	4	0	0	0
1	0	1	1	0	2	2	0
2	0	1	1	0	2	2	0
3	0	0	0	4	0	0	0

When $u_i \oplus v_i = 0$, the surviving slice lies in $\{00, 11\}$; when $u_i \oplus v_i = 1$, it lies in $\{01, 10\}$. In this example the support is even more concentrated, but the decisive point is already plain: once one conditions on survival up to level i , the detailed low prefix has disappeared and only the current carry-pair slice remains. That is exactly the tiny summary that later gets normalized into the four-state differential recurrence.

The linear toy data show the analogous compression on the signed side. For

$$(n, a, \alpha, \beta) = (4, 0001_2, 0100_2, 0100_2)$$

consider the conditioned partial Walsh sums $L_i(0), L_i(1)$ from equation (31). The verifier prints their exact evolution:

Table 5: Toy linear prefix-state evolution for $(a, \alpha, \beta) = (0001_2, 0100_2, 0100_2)$ at $n = 4$. The two conditioned partial Walsh sums form the exact tiny summary on the linear side.

prefix length i	a_i	μ_i	ν_i	$L_i(0)$	$L_i(1)$	$L_i(0) + L_i(1)$
0	1	0	0	1	0	1
1	0	0	0	1	1	2
2	0	0	1	3	1	4
3	0	0	0	5	-1	4
4	-	-	-	9	-1	8

The point is the same as on the differential side, but with signs instead of survivor counts. Once the processed low prefix has been compressed into the pair $(L_i(0), L_i(1))$, the suffix no longer needs the full low-bit history. It only needs the current signed mass sitting on carry 0 and on carry 1. In the toy example the second component even changes sign after the $(a_i, \mu_i, \nu_i) = (0, 0, 1)$ step, making the future cancellation pressure visibly dependent on the one-bit carry-conditioned state and not on the raw prefix string.

This is the engineering heart of the reverse-engineering claim. A 15-page report is long enough to carry tables, local lemmas, and a recurrence, but short enough that one expects compression pressure everywhere. The modern state-machine language of this draft should therefore be read as the cleaned-up descendant of this recurrence-first viewpoint, not as the literal first language of 1996. Stated bluntly: first came the tables, then the recurrence, and only after that the carry state machine.

G.3.10 A Reconstruction Principle Forced by the Abstract Pair

Let us now formalise the parallel structure. The two 1996 abstracts and the later testimony do not determine a unique 1996 notation, but they do constrain the shape of any mathematically honest

reconstruction. The next remark records that constraint at the phenomenology level rather than immediately in automaton language.

Remark 10 (abstract-level reconstruction principle). For fixed-addend addition

$$f_{\text{Constant}_{\text{Key}}}(X) = Y = X + \text{Constant}_{\text{Key}} \pmod{2^N},$$

any reconstruction of Miyano’s 1996 pair of reports that is consistent with the surviving citations must contain two addend-dependent prefix-evaluation procedures

$$\mathcal{D}_{\text{Constant}_{\text{Key}}}(U, V) \quad \text{and} \quad \mathcal{L}_{\text{Constant}_{\text{Key}}}(\alpha, \beta)$$

with the following properties:

- (R1) $\mathcal{D}_{\text{Constant}_{\text{Key}}}(U, V)$ returns the XOR differential probability of $f_{\text{Constant}_{\text{Key}}}$ and depends explicitly on the low-bit pattern of $\text{Constant}_{\text{Key}}$;
- (R2) $\mathcal{L}_{\text{Constant}_{\text{Key}}}(\alpha, \beta)$ returns the linear approximation probability or correlation of $f_{\text{Constant}_{\text{Key}}}$ and depends explicitly on the low-bit pattern of $\text{Constant}_{\text{Key}}$;
- (R3) both are built from low-to-high prefix updates driven by carries;
- (R4) each update uses only a very small hidden summary, because otherwise the later linear-time and logarithmic algorithms could not plausibly descend from it.

The justification is historical rather than deductive: properties (R1) and (R2) paraphrase the two abstracts, property (R3) reflects the carry recursion of modular addition, and property (R4) is the smallest summary compatible with the later claims of efficient fixed-addend evaluation in [7, 9, 10].

G.4 Reconstructing the Missing DP_K and LC_K Chains

This subsection is intentionally longer and more mathematical than the surviving compressed OCR presentation. The two missing Japanese nodes cited around Miyano’s 1998 letter are not available here, so we do not claim to recover their literal notation. Instead we reconstruct, from first principles, the full fixed-addend chain that those nodes must have compressed: the differential probability

$$\text{DP}_K(u, v) = \Pr_x[F_K(x \oplus u) \oplus F_K(x) = v]$$

and the Walsh correlation

$$\text{LC}_K(\alpha, \beta) = \mathbb{E}_x(-1)^{\langle \alpha, x \rangle \oplus \langle \beta, F_K(x) \rangle}.$$

The goal is not to say “there exists a recurrence.” The goal is to write the recurrence down, derive it, normalize it, and expose exactly where the fixed addend bits enter.

G.4.1 Fixed-addend map, bit convention, and the shared carry law

Let the word size be n , and identify $\mathbb{F}_2^n$ with integers modulo 2^n by little-endian expansion

$$x = \sum_{i=0}^{n-1} x_i 2^i, \quad K = \sum_{i=0}^{n-1} k_i 2^i, \quad x_i, k_i \in \mathbb{F}_2.$$

The fixed-addend map is

$$F_K(x) = x \boxplus K = x + K \pmod{2^n}.$$

Let $c_i = c_i(x)$ be the carry into bit i of the addition $x \boxplus K$. We use the convention

$$c_0 = 0.$$

At every bit position,

$$y_i = (F_K(x))_i = x_i \oplus k_i \oplus c_i, \tag{A1}$$

and

$$c_{i+1} = \text{Maj}(x_i, k_i, c_i), \quad (\text{A2})$$

where

$$\text{Maj}(a, b, c) = 1 \iff a + b + c \geq 2.$$

Equivalently, over $\mathbb{F}_2$,

$$\text{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c). \quad (\text{A3})$$

Equations (A1)–(A3) are the whole arithmetic source. Everything below is a different projection of this same carry law.

G.4.2 The DP_K object and its numerator

For $u, v \in \mathbb{F}_2^n$, define

$$\text{DP}_K(u, v) = 2^{-n} \# \{x \in \mathbb{F}_2^n : F_K(x \oplus u) \oplus F_K(x) = v\}. \quad (\text{D1})$$

It is useful to split off the integer numerator

$$N_K(u, v) := 2^n \text{DP}_K(u, v). \quad (\text{D2})$$

The reconstruction target on the differential side is therefore an exact formula for $N_K(u, v)$.

Run two copies of the adder in parallel:

$$x \mapsto F_K(x), \quad x' = x \oplus u \mapsto F_K(x').$$

Let c_i be the carry in the first copy and c'_i the carry in the second copy. Then

$$(F_K(x))_i = x_i \oplus k_i \oplus c_i, \quad (\text{D3})$$

while

$$(F_K(x \oplus u))_i = x_i \oplus u_i \oplus k_i \oplus c'_i. \quad (\text{D4})$$

Taking the XOR of (D3) and (D4) gives the exact visible difference identity

$$v_i = u_i \oplus c_i \oplus c'_i. \quad (\text{D5})$$

This identity is the first key point. The addend bit k_i cancels out of the visible output-difference test. But it does not vanish from the problem, because the next carries are still

$$c_{i+1} = \text{Maj}(x_i, k_i, c_i), \quad c'_{i+1} = \text{Maj}(x_i \oplus u_i, k_i, c'_i). \quad (\text{D6})$$

Thus the current visible equation only sees the parity $c_i \oplus c'_i$, whereas the suffix still needs the ordered pair (c_i, c'_i) .

Lemma 5 (differential carry-pair sufficiency). *After bits $0, \dots, i-1$ have been processed, the exact distribution of all future differential events from bit i onward depends on the processed prefix only through the carry pair (c_i, c'_i) .*

Proof. For $j \geq i$, equation (D6) updates (c_j, c'_j) to (c_{j+1}, c'_{j+1}) using only the current pair (c_j, c'_j) , the next free input bit x_j , the known difference bit u_j , and the fixed addend bit k_j . The lower prefix bits never appear again once the pair (c_i, c'_i) is known. Since the unprocessed suffix bits remain uniformly distributed after conditioning on a prefix event, the suffix law is fully determined by the current pair and the public suffix data $(u_j, v_j, k_j)_{j \geq i}$. $\square$

G.4.3 Four-state prefix counts for DP_K

Use the state order

$$00, \quad 01, \quad 10, \quad 11. \quad (\text{D7})$$

For $i = 0, \dots, n$ and $r, s \in \{0, 1\}$ define the prefix count

$$D_i(r, s) = \# \left\{ (x_0, \dots, x_{i-1}) : \begin{array}{l} \text{the target output difference holds on bits } 0, \dots, i-1, \\ (c_i, c'_i) = (r, s) \end{array} \right\}. \quad (\text{D8})$$

The initial vector is

$$D_0(0, 0) = 1, \quad D_0(0, 1) = D_0(1, 0) = D_0(1, 1) = 0. \quad (\text{D9})$$

For a fixed bit position i , define the local transition matrix

$$M_{k_i, u_i, v_i}^D[(r, s), (r', s')] = \mathbf{1}_{\{v_i = u_i \oplus r \oplus s\}} \# \left\{ b \in \{0, 1\} : \begin{array}{l} r' = \text{Maj}(b, k_i, r), \\ s' = \text{Maj}(b \oplus u_i, k_i, s) \end{array} \right\}. \quad (\text{D10})$$

Here b is the current input bit x_i . Equation (D10) is not a heuristic. It is the exact enumeration of the two possible values of x_i under the visible constraint (D5) and carry update (D6).

Let

$$\mathbf{D}_i = (D_i(0, 0), D_i(0, 1), D_i(1, 0), D_i(1, 1)) \quad (\text{D11})$$

be a row vector. Then

$$\mathbf{D}_{i+1} = \mathbf{D}_i M_{k_i, u_i, v_i}^D. \quad (\text{D12})$$

Iterating gives

$$\mathbf{D}_n = (1, 0, 0, 0) \prod_{i=0}^{n-1} M_{k_i, u_i, v_i}^D. \quad (\text{D13})$$

Since every surviving prefix of length n is a complete input x , the exact differential numerator is

$$N_K(u, v) = \mathbf{D}_n \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, \quad (\text{D14})$$

and therefore

$$\boxed{\text{DP}_K(u, v) = 2^{-n} (1, 0, 0, 0) \left(\prod_{i=0}^{n-1} M_{k_i, u_i, v_i}^D \right) \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}}. \quad (\text{D15})$$

Derivation of (D12)–(D15). Fix a state (r, s) at level i . A prefix counted by $D_i(r, s)$ can be extended by exactly those $b = x_i \in \{0, 1\}$ for which the current visible difference equation $v_i = u_i \oplus r \oplus s$ holds. If it holds, the extension lands at (r', s') precisely when $r' = \text{Maj}(b, k_i, r)$ and $s' = \text{Maj}(b \oplus u_i, k_i, s)$. Counting these extensions gives (D10). Summing over all old states gives (D12). The product formula (D13) follows by induction on i . At $i = n$, there are no more visible constraints; summing over all final carry-pair states gives the number of inputs satisfying the full differential relation. Division by 2^n gives (D15). $\square$

G.4.4 Explicit DP_K local matrices

For auditability, we now expand (D10). In the state order 00, 01, 10, 11, the eight possible matrices are as follows.

For $k = 0$,

$$M_{0,0,0}^D = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}, \quad M_{0,0,1}^D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad (\text{D16})$$

$$M_{0,1,0}^D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad M_{0,1,1}^D = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}. \quad (\text{D17})$$

For $k = 1$,

$$M_{1,0,0}^D = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}, \quad M_{1,0,1}^D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad (\text{D18})$$

$$M_{1,1,0}^D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad M_{1,1,1}^D = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}. \quad (\text{D19})$$

These matrices are the table form of the missing differential chain. Every zero row is an impossible visible parity slice at that state. Every entry 2 means both values of x_i survive and land in the same next state. Every pair of 1 entries means the two values of x_i split into two next carry-pair states.

G.4.5 Correct look-ahead quotient of the four-state DP_K chain

The four-state matrix product above is the safest raw object. A scalar quotient exists, but it must be stated with the correct timing. The tempting sentence “after processing bits $0, \dots, i-1$, the carry pair at level i is supported only on the slice $c_i \oplus c'_i = u_i \oplus v_i$ ” is false for the raw prefix count. A raw prefix that already satisfies bits $0, \dots, i-1$ has not yet been tested against bit i . The slice $c_i \oplus c'_i = u_i \oplus v_i$ is a look-ahead constraint for the next visible bit.

This is exactly the off-by-one trap in fixed-addend differential derivations. The correct quotient is therefore a look-ahead quotient, not an ordinary projection of the raw vector $\mathbf{D}_i$.

Define

$$\eta_i = u_i \oplus v_i. \quad (\text{D20})$$

For $0 \leq i \leq n-1$, let B_i be the event that

- (B1) the differential relation has already passed on bits $0, \dots, i-1$; and
- (B2) the carry pair before bit i already satisfies the next visible parity test

$$c_i \oplus c'_i = \eta_i. \quad (\text{D21})$$

Thus B_i is a prefix event with one-bit look-ahead. The base case is

$$B_0 \text{ holds} \iff \eta_0 = 0, \quad (\text{D22})$$

because $c_0 = c'_0 = 0$. Hence $u_0 \neq v_0$ immediately gives $\text{DP}_K(u, v) = 0$.

Assume $\eta_0 = 0$. Define

$$A_i = \Pr[B_i], \quad \delta_i = \Pr[c_i = 1 \mid B_i] \quad (A_i > 0), \quad (\text{D23})$$

where A_i is a probability over the already consumed low bits $x_0, \dots, x_{i-1}$. Under B_i , only two ordered carry-pair states remain possible. If $\eta_i = 0$, they are 00 and 11; if $\eta_i = 1$, they are 01 and 10. More explicitly,

$$\Pr[(c_i, c'_i) = (0, \eta_i) \mid B_i] = 1 - \delta_i, \quad \Pr[(c_i, c'_i) = (1, 1 \oplus \eta_i) \mid B_i] = \delta_i. \quad (\text{D24})$$

This is the place where the two-scalar quotient becomes exact: not on the raw prefix vector, but on the look-ahead live slice.

For $1 \leq i \leq n-1$, set

$$S_i = (u_{i-1}, v_{i-1}, \eta_i). \quad (\text{D25})$$

The transition from B_{i-1} to B_i consumes the bit x_{i-1} . The first two coordinates of S_i describe the just-tested visible bit $i-1$, and the third coordinate η_i is the look-ahead carry parity required for bit i . Multiplying the two live carry-pair states in (D24) by the one-bit branch table gives

$$A_i = A_{i-1} \varphi_i, \quad (\text{D26})$$

with

$$\delta_i = \begin{cases} (k_{i-1} + \delta_{i-1})/2, & S_i = 000, \\ 0, & S_i = 001, \\ k_{i-1}, & S_i \in \{010, 100, 110\}, \\ \delta_{i-1}, & S_i \in \{011, 101\}, \\ 1/2, & S_i = 111, \end{cases} \quad (\text{D27})$$

and

$$\varphi_i = \begin{cases} 1, & S_i = 000, \\ 0, & S_i = 001, \\ 1/2, & S_i \in \{010, 011, 100, 101\}, \\ 1 - (k_{i-1} + \delta_{i-1} - 2k_{i-1}\delta_{i-1}), & S_i = 110, \\ k_{i-1} + \delta_{i-1} - 2k_{i-1}\delta_{i-1}, & S_i = 111. \end{cases} \quad (\text{D28})$$

The final bit has a special role. Once B_{n-1} holds, the most significant visible equation

$$v_{n-1} = u_{n-1} \oplus c_{n-1} \oplus c'_{n-1} \quad (\text{D29})$$

has already passed. The last input bit x_{n-1} only affects the discarded carry out of the word and therefore contributes no further survival constraint modulo 2^n . Consequently,

$$\text{DP}_K(u, v) = \begin{cases} 0, & u_0 \neq v_0, \\ \prod_{i=1}^{n-1} \varphi_i, & u_0 = v_0. \end{cases} \quad (\text{D30})$$

For $n = 1$, the empty product is 1.

Equations (D15) and (D30) are equivalent, but they are not the same normalization. Equation (D15) is the raw four-state prefix count. Equation (D30) is the look-ahead parity-slice quotient. Keeping this distinction explicit prevents the scalar recurrence from accidentally repeating an incomplete or off-by-one version of the old hand derivation.

G.4.6 The LC_K object and its relation to LAP_K

For masks $\alpha, \beta \in \mathbb{F}_2^n$, define the fixed-addend Walsh correlation

$$\text{LC}_K(\alpha, \beta) = 2^{-n} \sum_{x \in \mathbb{F}_2^n} (-1)^{\langle \alpha, x \rangle \oplus \langle \beta, F_K(x) \rangle}. \quad (\text{L1})$$

If a historical linear approximation also contains an explicit addend mask ρ on K , then the approximation probability is

$$\text{LAP}_K(\alpha, \rho, \beta) = \frac{1 + (-1)^{\langle \rho, K \rangle} \text{LC}_K(\alpha, \beta)}{2}. \quad (\text{L2})$$

Thus the addend-mask term is a global sign on the Walsh bias. The carry-transfer derivation only needs α and β ; ρ is restored at the end by (L2).

Using (A1),

$$\langle \alpha, x \rangle \oplus \langle \beta, F_K(x) \rangle = \bigoplus_{i=0}^{n-1} ((\alpha_i \oplus \beta_i)x_i \oplus \beta_i k_i \oplus \beta_i c_i). \quad (\text{L3})$$

Set

$$\mu_i = \alpha_i \oplus \beta_i, \quad \nu_i = \beta_i. \quad (\text{L4})$$

Then

$$\text{LC}_K(\alpha, \beta) = 2^{-n} \sum_x (-1)^{\bigoplus_i (\mu_i x_i \oplus \nu_i k_i \oplus \nu_i c_i)}. \quad (\text{L5})$$

This is the linear analogue of the differential reduction. On the differential side, the hidden state is a pair

of carries because two executions are compared. On the linear side, there is one execution, but the sign depends on the carry, so the current carry state must not be merged away.

Lemma 6 (linear carry-state sufficiency). *After bits $0, \dots, i-1$ have been processed, the exact suffix contribution to the Walsh sum depends on the processed prefix only through the current carry c_i .*

Proof. For $j \geq i$, the future carry recursion is $c_{j+1} = \text{Maj}(x_j, k_j, c_j)$. The future phase terms are $\mu_j x_j \oplus \nu_j k_j \oplus \nu_j c_j$. Once c_i is fixed, neither the future carries nor the future signs depend on the detailed lower prefix. Thus the suffix signed sum is determined by the current carry and the public suffix data $(\mu_j, \nu_j, k_j)_{j \geq i}$. $\square$

G.4.7 Two-state signed prefix sums for LC_K

For $i = 0, \dots, n$ and $r \in \{0, 1\}$ define

$$L_i(r) = \sum_{x_0, \dots, x_{i-1}} (-1)^{\bigoplus_{j=0}^{i-1} (\mu_j x_j \oplus \nu_j k_j \oplus \nu_j c_j)} \mathbf{1}_{\{c_i=r\}}. \quad (\text{L6})$$

The initial condition is

$$L_0(0) = 1, \quad L_0(1) = 0. \quad (\text{L7})$$

Define the local signed kernel

$$T_{k_i, \mu_i, \nu_i}^L(r, s) = \sum_{b \in \{0, 1\} : \text{Maj}(b, k_i, r) = s} (-1)^{\mu_i b \oplus \nu_i k_i \oplus \nu_i r}. \quad (\text{L8})$$

Then

$$L_{i+1}(s) = \sum_{r \in \{0, 1\}} L_i(r) T_{k_i, \mu_i, \nu_i}^L(r, s). \quad (\text{L9})$$

In row-vector notation,

$$\mathbf{L}_i = (L_i(0), L_i(1)), \quad \mathbf{L}_{i+1} = \mathbf{L}_i M_{k_i, \mu_i, \nu_i}^L, \quad (\text{L10})$$

where $M_{k, \mu, \nu}^L = (T_{k, \mu, \nu}^L(r, s))_{r, s \in \{0, 1\}}$.

Derivation of (L9). A prefix of length $i+1$ is obtained from a prefix of length i ending in carry r and a new bit $b = x_i$. The new sign factor contributed at bit i is

$$(-1)^{\mu_i b \oplus \nu_i k_i \oplus \nu_i r}.$$

The new carry is $s = \text{Maj}(b, k_i, r)$. Summing the new sign factors over the values of b that land in the same next carry s gives (L8). Multiplying by the old signed mass $L_i(r)$ and summing over r gives (L9). $\square$

At the end,

$$\boxed{\text{LC}_K(\alpha, \beta) = 2^{-n}(L_n(0) + L_n(1))}. \quad (\text{L11})$$

Equivalently,

$$\boxed{\text{LC}_K(\alpha, \beta) = 2^{-n}(1, 0) \left(\prod_{i=0}^{n-1} M_{k_i, \mu_i, \nu_i}^L \right) \begin{pmatrix} 1 \\ 1 \end{pmatrix}}. \quad (\text{L12})$$

G.4.8 Explicit LC_K local matrices and update rules

Expanding (L8) gives two matrix templates. If $k = 0$, then $\text{Maj}(b, 0, 0) = 0$ and $\text{Maj}(b, 0, 1) = b$, so

$$M_{0, \mu, \nu}^L = \begin{pmatrix} 1 + (-1)^\mu & 0 \\ (-1)^\nu & (-1)^{\mu \oplus \nu} \end{pmatrix}. \quad (\text{L13})$$

If $k = 1$, then $\text{Maj}(b, 1, 0) = b$ and $\text{Maj}(b, 1, 1) = 1$, so

$$M_{1,\mu,\nu}^L = \begin{pmatrix} (-1)^\nu & (-1)^{\mu \oplus \nu} \\ 0 & 1 + (-1)^\mu \end{pmatrix}. \quad (\text{L14})$$

For direct implementation, write $\mathbf{L}_i = (P, Q)$. For $k_i = 0$,

$$(P, Q)M_{0,\mu_i,\nu_i}^L = \begin{cases} (2P + Q, Q), & (\mu_i, \nu_i) = (0, 0), \\ (Q, -Q), & (\mu_i, \nu_i) = (1, 0), \\ (2P - Q, -Q), & (\mu_i, \nu_i) = (0, 1), \\ (-Q, Q), & (\mu_i, \nu_i) = (1, 1), \end{cases} \quad (\text{L15})$$

and for $k_i = 1$,

$$(P, Q)M_{1,\mu_i,\nu_i}^L = \begin{cases} (P, P + 2Q), & (\mu_i, \nu_i) = (0, 0), \\ (P, -P), & (\mu_i, \nu_i) = (1, 0), \\ (-P, -P + 2Q), & (\mu_i, \nu_i) = (0, 1), \\ (-P, P), & (\mu_i, \nu_i) = (1, 1). \end{cases} \quad (\text{L16})$$

Equations (L13)–(L16) are the complete local linear chain. Nothing is hidden in an oracle: each update is a two-entry signed integer calculation.

G.4.9 Normalization, probability, and dyadic form

The quantities $D_i(r, s)$ are ordinary counts, while $L_i(r)$ are signed counts. Therefore both final values are dyadic rationals:

$$\text{DP}_K(u, v) = \frac{N_K(u, v)}{2^n}, \quad \text{LC}_K(\alpha, \beta) = \frac{W_K(\alpha, \beta)}{2^n}, \quad (\text{N1})$$

where

$$N_K(u, v) \in \{0, 1, \dots, 2^n\}, \quad W_K(\alpha, \beta) \in \{-2^n, -2^n + 2, \dots, 2^n\}. \quad (\text{N2})$$

The linear approximation probability is then

$$\text{LAP}_K(\alpha, \rho, \beta) = \frac{1 + (-1)^{\langle \rho, K \rangle} W_K(\alpha, \beta) / 2^n}{2}. \quad (\text{N3})$$

This is why the correlation and probability formulations are mathematically equivalent but notationally different. The correlation is the cleaner state-machine object; the probability is the cryptanalytic bias in probability scale.

G.4.10 Why this is the missing $\text{DP}_K / \text{LC}_K$ derivation and not merely a modern restatement

The chain above is forced by three facts.

- (M1) The fixed addend is not random during the attack. Therefore k_i must appear as concrete data in the local update.
- (M2) The differential observable compares two executions. Therefore exact continuation needs the pair (c_i, c'_i) until the visible parity quotient is applied.
- (M3) The linear observable is a signed one-execution sum. Therefore exact continuation needs two signed masses, one conditioned on $c_i = 0$ and one conditioned on $c_i = 1$.

Consequently, the two missing chains have only one possible exact low-to-high shape up to notational normalization:

$$\begin{array}{lcl}
 F_K(x) = x \boxplus K & \implies & c_{i+1} = \text{Maj}(x_i, k_i, c_i) \\
 & \implies & \begin{cases} \text{DP}_K : \text{four-state carry-pair count,} \\ \text{LC}_K : \text{two-state signed carry transfer,} \end{cases} \\
 & \implies & \text{exact } O(n) \text{ evaluators and brute-force-verifiable code.}
 \end{array} \tag{N4}$$

That is the complete mathematical payload that the short historical sources compress. The present paper deliberately writes it out at proof length, because the open-source goal is not only to cite Miyano’s origin but to make the mechanism teachable, auditable, and executable.

G.4.11 Reference implementation contract for DP_K and LC_K

The companion file `fixed_addend_dpk_lck_reference.cpp` is the minimal executable form of equations (D10)–(D15) and (L8)–(L12). It deliberately exposes four functions rather than hiding behind a generic oracle:

- (I1) `dpk_numerator`: the four-state recurrence for DP_K ;
- (I2) `brute_dpk_numerator`: direct enumeration over all x ;
- (I3) `lck_numerator`: the two-state signed recurrence for LC_K ;
- (I4) `brute_lck_numerator`: direct Walsh summation over all x .

The executable checks all addends and all pairs exhaustively for $1 \leq n \leq 8$, then runs deterministic random checks for $n = 12$ and $n = 16$. This is not a replacement for the derivation above. It is the open-source audit hook: anyone can compile the file, break the recurrence, and immediately see the mismatch.

G.5 Miyano 1998 OCR White-Box Reading and Clean Reconstruction

This note belongs in the historical appendix, not in the core proof. Its job is deliberately limited: after the long reconstruction of the missing DP_K and LC_K chains above, it records what the recovered OCR of Miyano’s 1998 IEICE letter directly confirms, and why that confirmation matters.

The OCR confirms four points that are central to this draft.

- (O1) The paper explicitly studies the fixed-addend map

$$Y = X + K \pmod{2^w},$$

with K fixed during the attack, rather than averaged as a fresh random variable.

- (O2) The introductory one-bit example already shows the phenomenon in its smallest possible form. For

$$X_1 \oplus K_1 \oplus Y_1 = 0,$$

the probability is

$$\Pr[X_1 \oplus K_1 \oplus Y_1 = 0] = \begin{cases} 1, & K_0 = 0, \\ 1/2, & K_0 = 1, \end{cases} \quad \text{that is } 1 - \frac{1}{2}K_0.$$

This is the fixed-addend thesis in miniature: a later-bit linear bias is already controlled by a concrete lower addend bit through the carry.

- (O3) The differential object in the OCR is the fixed-addend XOR differential

$$DCP_+(\Delta X, \Delta Y) = \Pr_X[(X + K) \oplus ((X \oplus \Delta X) + K) = \Delta Y],$$

which is exactly $\text{DP}_K(u, v)$ under the renaming $u = \Delta X$ and $v = \Delta Y$.

(O4) The linear object is the fixed-addend linear approximation probability

$$\text{LAP}_+(\Gamma X, \Gamma K, \Gamma Y) = \Pr_X[\langle \Gamma X, X \rangle \oplus \langle \Gamma K, K \rangle \oplus \langle \Gamma Y, X + K \rangle = 0],$$

together with prefix phase notation of the form

$$S(i) = X_i \Gamma X_i \oplus Y_i \Gamma Y_i \oplus K_i \Gamma K_i, \quad T(i) = \bigoplus_{j=0}^i S(j).$$

In the notation of Part II, this is the visible prefix phase whose exact white-box evaluator is the two-state carry-conditioned Walsh transfer.

The OCR theorem stack is compressed: proofs are omitted, the differential section leaves part of the calculation to the reader, and the linear section presents a case partition rather than a modern transfer-matrix evaluator. This draft should therefore not overclaim that Miyano already wrote the 4-state and 2-state machines in the same notation used here. The safer and stronger statement is this: the OCR exposes the same fixed-addend problem, the same addend-dependent probabilities, and the same low-to-high prefix logic; Parts I and II turn that compressed source into explicit state recurrences, matrices, algorithms, and exhaustive verification code.

This is also why the companion implementation is part of the disclosure rather than an optional engineering appendix. The files `fixed_addend_reference_verifier.cpp` and `fixed_addend_dpk_lck_reference.cpp` are the executable white-box form of the reconstruction: they evaluate DP_K through the carry-pair count and LC_K through the signed carry transfer, then compare both against brute force on small widths.

After reading the surrounding RC5 papers, Miyano’s OCR, Lipmaa–Moriai, Machado, and Wallen together, the right historical reaction is not merely that Miyano left a terse note for later authors to expand. The right reaction is stronger: Miyano made a genuinely pioneering contribution by isolating addend dependency itself as a cryptanalytic object before the later literature had standardized the clean carry-state language. The purpose of this paper is to honor that origin in the most open way possible: not by hiding behind a citation, but by turning the compressed insight into a proof-and-code object that other readers can audit, run, and learn from.

G.6 The Missing Middle, Derived by Hand

The real gap between the 1996/1998 titles and the later clean recurrences is not a missing citation count. It is the missing reason why addend dependence must collapse to a tiny prefix memory. This section is where the reverse-engineering report must actually cash out. We are not trying to infer Miyano’s private notation. We are trying to reconstruct the shortest intellectually honest route from a 1996 table-based study to the 2001–2004 recurrences.

That route has one bottleneck question:

after the low bits have already been processed, what information about that processed prefix does the remaining suffix still need?

If the answer were “a lot”, then a short 1996 report could at best list examples. If the answer is “only the carry information”, then a genuine low-to-high theory becomes possible. The next theorem proves exactly that.

Theorem 12 (forced prefix state for the fixed-addend problem). *Let*

$$f_{\text{Constant}_{\text{Key}}}(X) = Y = X + \text{Constant}_{\text{Key}} \pmod{2^N}, \quad f_{\text{Constant}_{\text{Key}}}^{-1}(Y) = X = Y - \text{Constant}_{\text{Key}} \pmod{2^N},$$

and, for a concrete word size n , write

$$\text{Constant}_{\text{Key}} = \sum_{i=0}^{n-1} a_i 2^i.$$

Then the following statements hold.

- (a) **Differential side.** Fix visible differences (u, v) . When one scans the two executions

$$x \mapsto f_{\text{Constant}_{\text{Key}}}(x), \quad x \oplus u \mapsto f_{\text{Constant}_{\text{Key}}}(x \oplus u)$$

from low bits to high bits, the exact continuation of the suffix probability from level i onward depends on the already processed prefix bits only through the current carry pair $(c_i, c'_i) \in \{0, 1\}^2$.

- (b) **Parity slice reduction.** On every live prefix at level i , the visible differential bit enforces

$$v_i = u_i \oplus c_i \oplus c'_i.$$

Therefore among the four exact carry-pair states only two can survive at any fixed level i : either $\{00, 11\}$ or $\{01, 10\}$.

- (c) **Linear side.** Fix masks (α, β) . When one scans the Walsh phase

$$(-1)^{\langle \alpha, x \rangle \oplus \langle \beta, f_{\text{Constant}_{\text{Key}}}(x) \rangle}$$

from low bits to high bits, the exact continuation of the suffix signed sum from level i onward depends on the already processed prefix bits only through the single carry bit $c_i \in \{0, 1\}$.

- (d) **Consequence.** Any exact low-to-high evaluation of the fixed-addend differential problem is equivalent to a 4-state prefix system, and any exact low-to-high evaluation of the fixed-addend linear problem is equivalent to a 2-state prefix system. The 2001–2004 recurrences, matrix products, and finite-state formulations are therefore different surfaces of the same forced prefix law rather than different hidden phenomena. Later modern engineering encodings may still be useful, but only as downstream consumers of that same law.

Remark 11 (how this theorem should be read historically). The theorem should not be overread as a claim that Miyano must already have written a polished 4-state or 2-state diagram in 1996. The more plausible order is the reverse: first one computes low-bit tables, then one notices that the suffix only asks for the current carry pair or current carry bit, and only after that does one compress the observation into a tiny state system. In that historical reading, the carry state machine is not an imported modern idea; it is the final compressed form of a table-driven 1996 workflow.

Proof. We prove the differential and linear claims separately.

Differential side. Fix a level i . Split the input into a processed prefix and an unprocessed suffix:

$$x = x_{<i} + 2^i x_{\geq i},$$

where $x_{<i}$ contains bits $0, \dots, i-1$ and $x_{\geq i}$ contains bits $i, \dots, n-1$. The bits of $x_{\geq i}$ remain uniform and independent after conditioning on any event that depends only on $x_{<i}$.

Now write the two carry recursions

$$c_{j+1} = \text{Maj}(x_j, a_j, c_j), \quad c'_{j+1} = \text{Maj}(x_j \oplus u_j, a_j, c'_j).$$

For every $j \geq i$, the next pair (c_{j+1}, c'_{j+1}) depends only on the current pair (c_j, c'_j) , on the suffix bit x_j , on the visible input bit u_j , and on the constant bit a_j . It does *not* depend on the detailed shape of the lower prefix once (c_j, c'_j) has been fixed. Therefore the exact law of every future suffix event is determined by the current carry pair and the remaining suffix data. This proves part (a).

The visible output relation at level i is

$$v_i = u_i \oplus c_i \oplus c'_i.$$

Hence the parity $c_i \oplus c'_i$ is fixed by (u_i, v_i) . If $u_i = v_i$, only the equal-parity states $\{00, 11\}$ can survive; if $u_i \neq v_i$, only the odd-parity states $\{01, 10\}$ can survive. This proves part (b).

Linear side. Again fix a level i and split the input as above. The output bits at positions $j \geq i$ are generated by the recursion

$$y_j = x_j \oplus a_j \oplus c_j, \quad c_{j+1} = \text{Maj}(x_j, a_j, c_j).$$

Once the current carry c_i is fixed, every later output bit and every later sign contribution depends only on the suffix bits $x_i, x_{i+1}, \dots$ and on the suffix bits of the constant. The processed low prefix influences the future only through the single bit c_i .

Therefore the full Walsh sum can be decomposed into two conditioned partial sums according to the two possible values of c_i . No larger hidden memory is required for exact continuation. This proves part (c).

Part (d) is now immediate. The differential side has exactly four exact carry-pair states before parity reduction, while the linear side has exactly two exact carry states. Any later exact reformulation may quotient, compress, or compile these states, but it cannot replace them by a different underlying mechanism without changing the problem itself. $\square$

Corollary 9 (why the 1996/1998 to 2004 chain contains no missing miracle). *The passage*

1996/1998 addend-dependent phenomenology $\longrightarrow$ 2001–2004 explicit recurrences and finite-state forms $\longrightarrow$ later down

is mathematically continuous. The first layer observes that the probability depends on the actual fixed constant. The second layer writes the exact 4-state or 2-state prefix law. The third layer, when present, merely packages the same law for implementation or search purposes.

Proof. Theorem 12 shows that the tiny prefix state is forced by the adder itself. Therefore the later surfaces do not introduce a new hidden mechanism; they merely choose different exact or approximate coordinate systems for the same carry-governed prefix law. $\square$

In plain language: the lost middle is not a mysterious extra theorem between Miyano and the later literature. The historical gap is the manual observation that once one operand is frozen, the adder still remembers the past, but it remembers it only through the carry. Two executions give a carry pair; one Walsh scan gives one carry. The 2001–2004 line is then the first clean rewriting of that fact. Everything later is secondary packaging.

G.7 What Makes a Good Differential Characteristic in the Fixed-Addend Setting

We now make the differential search problem explicit.

The point here is not merely to repeat the slogan that the probability depends on the addend. The stronger claim is that the fixed-addend differential object already carries an *exact local transfer law*. In the modern normalization proved later in Part I, that law appears in two equivalent forms: a scalar prefix recurrence through (δ_i, ϕ_i) and an exact four-state transition system through the operators M_{a_i, u_i, v_i}^D .

For a fixed constant $\text{Constant}_{\text{Key}}$, a differential characteristic is a pair $(u, v) \in \{0, 1\}^n \times \{0, 1\}^n$, and by theorem 1 its exact probability is

$$\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) = \prod_{i=0}^{n-1} \phi_i, \quad (67)$$

where each local factor ϕ_i is determined by the local parity type

$$S_i = (u_{i-1}, v_{i-1}, u_i \oplus v_i)$$

together with the current addend bit a_{i-1} and the current carry parameter δ_{i-1} . Equivalently, by theorem 3, one may write

$$\text{DP}_{\text{Constant}_{\text{Key}}}^{\boxplus}(u \rightarrow v) = 2^{-n} e_{00} \left(\prod_{i=0}^{n-1} M_{a_i, u_i, v_i}^D \right) \mathbf{1}. \quad (68)$$

Accordingly, to find a good differential characteristic means to choose the visible pair (u, v) so that $\text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v)$ is as large as possible under the relevant boundary conditions.

At the prefix-recursive level, the search problem is governed by three exact mechanisms.

- (D1) **Forbidden local patterns.** If $S_i = (0, 0, 1)$ for some $i \geq 1$, then $\phi_i = 0$ and the whole characteristic dies.
- (D2) **Half steps.** If $S_i \in \{010, 011, 100, 101\}$, then $\phi_i = 1/2$, so every such local step costs one exact factor of 2^{-1} .
- (D3) **Run-compatible blocks.** If a long block satisfies $u_i = v_i = 1$, then nonzero probability forces the addend bits inside the block to satisfy theorem 2.

Thus the fixed-addend differential search problem is not merely a count of low-weight differences. It is a problem of steering the visible pattern (u, v) so that the induced local types avoid impossibility, minimize the number of half steps, and exploit the run-compatible regions of the actual addend.

Proposition 26 (exact dynamic form of the differential search problem). *For any exact carry-count row vector $z \in \mathbb{R}_{\geq 0}^4$, define the suffix value function recursively by*

$$\Delta_n(z) := 2^{-n} z \mathbf{1},$$

and for $0 \leq i < n$,

$$\Delta_i(z) := \max_{(u_i, v_i) \in \{0,1\}^2} \Delta_{i+1}(z M_{a_i, u_i, v_i}^D).$$

Then

$$\max_{u, v \in \{0,1\}^n} \text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) = \Delta_0(e_{00}).$$

Equivalently, a best characteristic is obtained by a sequence of local choices (u_i, v_i) maximizing

$$2^{-n} e_{00} \left(\prod_{i=0}^{n-1} M_{a_i, u_i, v_i}^D \right) \mathbf{1}.$$

Proof. Equation (68) shows that every visible pair (u, v) determines a unique sequence of local matrices M_{a_i, u_i, v_i}^D . Conversely, every such sequence determines a unique visible pair. Hence maximizing over characteristics is exactly the same as maximizing over the local visible choices bit by bit.

Now fix a row-vector prefix state z at level i . The best differential probability obtainable from the suffix $i, i+1, \dots, n-1$ is the best value obtainable after one legal local visible choice (u_i, v_i) , followed by an optimal continuation on the remaining suffix. This gives the backward recurrence for Δ_i . At the terminal level no bits remain, so the exact probability represented by the terminal count vector is $2^{-n} z \mathbf{1}$. Starting from the initial state e_{00} yields the claim. $\square$

Proposition 27 (exact local decomposition of the differential search objective). *Let*

$$\begin{aligned} \mathcal{Z}(u, v) &:= \#\{i \in \{1, \dots, n-1\} : S_i = 001\}, \\ \mathcal{H}(u, v) &:= \#\{i \in \{1, \dots, n-1\} : S_i \in \{010, 011, 100, 101\}\}. \end{aligned}$$

Then the exact fixed-addend differential probability admits the factorization

$$\begin{aligned} \text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) &= \mathbf{1}_{\{u_0=v_0\}} \mathbf{1}_{\{\mathcal{Z}(u,v)=0\}} 2^{-\mathcal{H}(u,v)} \\ &\quad \times \prod_{\substack{1 \leq i \leq n-1 \\ S_i=110}} (1 - (a_{i-1} + \delta_{i-1} - 2a_{i-1}\delta_{i-1})) \prod_{\substack{1 \leq i \leq n-1 \\ S_i=111}} (a_{i-1} + \delta_{i-1} - 2a_{i-1}\delta_{i-1}), \end{aligned} \tag{69}$$

where the parameters δ_{i-1} are generated recursively by equation (19). In particular,

$$\text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) = 0 \iff u_0 \neq v_0 \text{ or } \mathcal{Z}(u, v) > 0 \text{ or one of the factors in (69) vanishes.} \tag{70}$$

Proof. This is a direct regrouping of the product formula (18). The bit-0 term is $\phi_0 = \mathbf{1}_{\{u_0=v_0\}}$, because $S_0 = (0, 0, u_0 \oplus v_0)$. For $i \geq 1$, the factors are then grouped according to the six classes appearing in equation (20). Type 001 contributes the zero factor, the four half-step types contribute the exact factor 2^{-1} each, type 110 contributes the first nontrivial carry-dependent factor, and type 111 contributes the complementary carry-dependent factor. $\square$

Corollary 10 (necessary conditions for nonzero and high-probability characteristics). *Let (u, v) be a fixed-addend differential characteristic.*

(D1) *If $u_0 \neq v_0$, then $\text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) = 0$.*

(D2) *If $S_i = 001$ for some $i \geq 1$, then $\text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) = 0$.*

(D3) *If $\mathcal{H}(u, v) = h$, then*

$$\text{DP}_{\text{ConstantKey}}^{\boxplus}(u \rightarrow v) \leq 2^{-h},$$

with equality if and only if $u_0 = v_0$, $\mathcal{Z}(u, v) = 0$, and every type-110 and type-111 factor in equation (69) equals 1.

(D4) *If $u_i = v_i = 1$ on a nontrivial block, then theorem 2 imposes the corresponding run-compatibility constraints on the addend bits.*

Proof. Part (1) is the bit-0 constraint of the fixed-addend differential relation. Part (2) is corollary 2. Part (3) follows immediately from proposition 27, since the two indicator factors and every remaining multiplicative factor are at most 1. Part (4) is exactly theorem 2. $\square$

In practical terms, a good fixed-addend differential characteristic is therefore one that satisfies all low-bit consistency conditions, uses as few half steps as possible, and aligns its visible runs with the carry-compatible block structure of the concrete addend.

G.8 What Makes a Good Linear Approximation in the Fixed-Addend Setting

We now make the search problem exact.

The transition from the previous section is parallel rather than abrupt. There the question was how to steer a visible differential pattern through the exact carry-dependent survival law. Here the question is how to steer a mask pair through the exact carry-dependent signed transfer law. The fixed addend is the same; only the observable has changed from survival probability to signed accumulation.

Again, the claim is not merely that the bias depends on the addend. The stronger claim is that the fixed-addend linear object already carries an *exact signed one-bit transfer law*. In the modern normalization proved later in Part II, that law appears as a two-state carry-conditioned Walsh recurrence, equivalently written through the local kernels M_{a_i, μ_i, ν_i}^L .

For a fixed constant ConstantKey , a linear approximation is a mask pair $(\alpha, \beta) \in \{0, 1\}^n \times \{0, 1\}^n$, and by theorem 4 its exact correlation is

$$\text{CORRELATION}(\alpha \rightarrow \beta) = 2^{-n} e_0^\top \left(\prod_{i=0}^{n-1} M_{a_i, \mu_i, \nu_i}^L \right) \mathbf{1}^\top, \quad \mu_i = \alpha_i \oplus \beta_i, \quad \nu_i = \beta_i. \quad (71)$$

Accordingly, to find a good linear approximation means to choose the masks so that

$$|\text{CORRELATION}(\alpha \rightarrow \beta)|$$

is as large as possible.

Since the bitwise change of variables

$$(\alpha_i, \beta_i) \longleftrightarrow (\mu_i, \nu_i) \quad \text{via} \quad \beta_i = \nu_i, \quad \alpha_i = \mu_i \oplus \nu_i$$

is bijective, this is equivalently a search over the $2n$ local control bits

$$(\mu_0, \nu_0), (\mu_1, \nu_1), \dots, (\mu_{n-1}, \nu_{n-1}).$$

Thus the fixed-addend linear search problem is not merely a matter of counting local mask types. It is a problem of steering the signed carry-conditioned prefix Walsh vector

$$L_i = (L_i(0), L_i(1))$$

through the recurrence

$$L_0 = e_0^\top, \quad L_{i+1} = L_i M_{a_i, \mu_i, \nu_i}^L, \quad \text{CORRELATION}(\alpha \rightarrow \beta) = 2^{-n} L_n \mathbf{1}^\top.$$

Proposition 28 (exact dynamic form of the search problem). *For any row vector $z \in \mathbb{R}^2$, define the suffix value function recursively by*

$$\Gamma_n(z) := |z \mathbf{1}^\top|,$$

and for $0 \leq i < n$,

$$\Gamma_i(z) := \max_{(\mu, \nu) \in \{0,1\}^2} \Gamma_{i+1}(z M_{a_i, \mu, \nu}^L).$$

Then

$$\max_{\alpha, \beta \in \{0,1\}^n} |\text{CORRELATION}(\alpha \rightarrow \beta)| = 2^{-n} \Gamma_0(e_0^\top).$$

Equivalently, a best fixed-addend linear approximation is obtained by a sequence of local choices (μ_i, ν_i) maximizing

$$\left| e_0^\top \left(\prod_{i=0}^{n-1} M_{a_i, \mu_i, \nu_i}^L \right) \mathbf{1}^\top \right|.$$

Proof. Equation (71) shows that every mask pair (α, β) determines a unique sequence of local kernels M_{a_i, μ_i, ν_i}^L . Conversely, because

$$\beta_i = \nu_i, \quad \alpha_i = \mu_i \oplus \nu_i,$$

every sequence (μ_i, ν_i) determines a unique mask pair (α, β) . Thus maximizing over masks is exactly the same as maximizing over the local control sequence.

Now fix a row-vector prefix state z at level i . The best correlation obtainable from the suffix $i, i+1, \dots, n-1$ is the best value obtainable after one legal local choice (μ, ν) at bit i , followed by an optimal continuation on the remaining suffix. This gives the backward recurrence

$$\Gamma_i(z) = \max_{(\mu, \nu) \in \{0,1\}^2} \Gamma_{i+1}(z M_{a_i, \mu, \nu}^L).$$

At the terminal level, no bits remain, so the observed quantity is the absolute final Walsh sum

$$\Gamma_n(z) = |z \mathbf{1}^\top|.$$

Starting from the initial prefix state $L_0 = e_0^\top$, the claim follows. $\square$

Proposition 28 shows the main structural difference from the differential side. In the fixed-addend differential problem, much of the local behaviour can be organized by a static case split in terms of the visible pattern $(u_{i-1}, v_{i-1}, u_i \oplus v_i)$. In the linear problem, by contrast, the usefulness of a local choice (μ_i, ν_i) depends on the current signed prefix state $L_i = (L_i(0), L_i(1))$. The search problem is therefore inherently dynamic.

We now write the one-bit actions explicitly.

Proposition 29 (explicit one-bit actions on the prefix Walsh vector). *Let $z = (z_0, z_1)$ be any row vector, and write*

$$\sigma := (-1)^\nu.$$

Then the four one-bit updates are:

$$z M_{0,0,\nu}^L = (2z_0 + \sigma z_1, \sigma z_1), \tag{72}$$

$$z M_{0,1,\nu}^L = (\sigma z_1, -\sigma z_1), \tag{73}$$

$$z M_{1,0,\nu}^L = (\sigma z_0, \sigma z_0 + 2z_1), \tag{74}$$

$$z M_{1,1,\nu}^L = (\sigma z_0, -\sigma z_0). \quad (75)$$

Proof. From the explicit kernels in Appendix B, one has

$$M_{0,0,\nu}^L = \begin{pmatrix} 2 & 0 \\ \sigma & \sigma \end{pmatrix}, \quad M_{0,1,\nu}^L = \begin{pmatrix} 0 & 0 \\ \sigma & -\sigma \end{pmatrix},$$

and

$$M_{1,0,\nu}^L = \begin{pmatrix} \sigma & \sigma \\ 0 & 2 \end{pmatrix}, \quad M_{1,1,\nu}^L = \begin{pmatrix} \sigma & -\sigma \\ 0 & 0 \end{pmatrix}.$$

A direct row-vector multiplication gives equations (72)–(75). $\square$

These formulas give the exact local meaning of the search problem.

Corollary 11 (local interpretation). *At bit i , the local choice (μ_i, ν_i) has the following effect.*

- (L1) *If $\mu_i = 0$, then the carry component indexed by a_i is reinforced by a factor 2 and remains on the same carry state. The opposite carry component is copied forward with a sign depending on ν_i .*
- (L2) *If $\mu_i = 1$, then the carry component indexed by a_i is cancelled exactly. The opposite carry component survives, but it is mapped to an antisymmetric pair of the form $(t, -t)$.*
- (L3) *The bit ν_i changes signs only. It does not alter which component is cancelled, reinforced, or copied; rather, it determines whether surviving contributions later interfere constructively or destructively.*

Proof. This is an immediate reading of equations (72)–(75). $\square$

Corollary 11 explains why the fixed-addend linear problem cannot be reduced to a purely static count of local bit patterns. The choice of μ_i decides whether the currently relevant signed mass is amplified or filtered, but whether this is beneficial depends on where that mass is located in the present prefix state L_i .

There is also an immediate terminal consequence.

Proposition 30 (last-bit necessary condition for nonzero correlation). *If*

$$\mu_{n-1} = 1,$$

then

$$\text{CORRELATION}(\alpha \rightarrow \beta) = 0.$$

Equivalently, every nonzero fixed-addend linear approximation must satisfy

$$\alpha_{n-1} = \beta_{n-1}.$$

Proof. If $\mu_{n-1} = 1$, then by proposition 29 the final update produces a vector of the form

$$(t, -t)$$

for some real number t . Therefore

$$L_n \mathbf{1}^\top = t + (-t) = 0,$$

and hence

$$\text{CORRELATION}(\alpha \rightarrow \beta) = 2^{-n} L_n \mathbf{1}^\top = 0.$$

Since $\mu_{n-1} = \alpha_{n-1} \oplus \beta_{n-1}$, this is equivalent to

$$\alpha_{n-1} = \beta_{n-1}.$$

$\square$

G.8.1 Equivalent Low-State Normalization of the Same Origin

Once the phenomenology layer is accepted, the modern normalization becomes unavoidable. If a one-variable fixed-addend differential is evaluated by scanning two executions of the adder in parallel, then the hidden summary is equivalent to a pair of carries; if a fixed-addend linear correlation is evaluated by scanning one execution in the Walsh domain, then the hidden summary is equivalent to a single carry. This yields the clean state counts that later work makes explicit.

Remark 12 (modern equivalent skeleton). A mathematically faithful modern normalization of Miyano’s fixed-addend line consists of the following equivalent hidden-state objects.

- (M1) On the differential side, the minimal exact hidden object is a 4-state carry-pair process, because two executions are compared and each execution has one carry.
- (M2) On the linear side, the minimal exact hidden object is a 2-state carry process, because one execution is approximated and only one carry survives as hidden memory.
- (M3) Downstream papers preserve this backbone in different exact senses: Machado gives an exact quotient of the 4-state differential machine, Azimi gives an exact Boolean encoding of validity together with an approximate weight compilation, and Wallen gives the full two-input linear carry classification whose one-carry mechanism collapses to the fixed-addend transfer theory after one input is frozen.

The first two items are exact state-count claims: the differential problem compares two carry strings and therefore has four exact carry-pair states, while the linear problem tracks one carry and therefore has two exact states. The third item is a bridge claim whose exact content differs by paper: quotient for Machado, compilation for Azimi, and frozen-input specialization of Wallen’s two-input carry framework on the linear side. It is therefore not a claim that all three papers study the same probability space or use the same surface notation.

G.8.2 Why the Later Papers Fit the Three-Layer Order

With the three-layer structure in place, the later papers fall into a clean order.

First layer: fixed-addend phenomenology. The missing Miyano reports belong here. This is the layer of what one can directly *see*: tables, local impossibility patterns, dyadic losses, run rigidity, and addend dependence that refuses to average away. Guan–Ding, although much later, also belongs here mathematically because it studies another one-variable fixed-addend observable whose distribution is governed by carry runs: for spike constants in the family $A_i = \{2^i, 2^{n-1} + 2^i\}$, the visible carry word is an exact contiguous block whose length follows a truncated geometric law, and this law is then turned into an optimal probability-ordered search procedure.

Second layer: the 2001–2004 normalization of the fixed-addend object. This is the real reverse-engineering target. Lipmaa–Moriai expose the carry-difference viewpoint on the differential side. Machado makes the fixed-addend differential recurrence explicit and algorithmic. Wallen gives the linear carry classification, and the full report also serves as a historical bridge: its linear chapter explicitly points back to Miyano 1998 as the predecessor on the one-addend-fixed side. Lipmaa–Wallen–Dumas show that neighboring additive-probability problems also admit finite-state treatment once the right state is chosen. In engineering language, this is the first external dump of the compressed logic that the 1996 reports must already have been feeling toward.

Third layer: later generalization and outer shells. The papers after that stage are not the primary missing link. They are downstream consumers, reformulations, or stress tests. Schulte–Geers, Li, and Lipmaa’s PHT paper lift the same carry logic into CCZ-equivalent, Boolean, or block-matrix shells. They matter in this draft for one reason only: once our reverse-engineered core is on the table, these later papers let us hit it from several directions and check whether the same skeleton survives. They are verification artillery, not the bridge itself.

G.8.3 Exact Meanings of the Bridge Claims

The phrase “the same skeleton in another form” can easily be overread. In this draft it is always meant in one of the exact senses listed in Table 6; nothing stronger should be inferred.

Table 6: Exact meanings of the bridge claims used in this draft

Relation type	What is preserved exactly	What changes / representative use in this draft
freeze of one input	The local majority-carry law of ordinary addition.	One passes from the binary map $(x, y) \mapsto x + y$ to the unary family $f_{\text{Constant}_{\text{Key}}}(x) = x + \text{Constant}_{\text{Key}}$. This is exact at the function-family level, but not at the averaged two-input probability level. This is the sense in which RC5 local subkey additions and the fixed-addend object are connected, and the sense in which Lipmaa–Moriai and Wallen sit downstream of the one-addend-fixed problem.
quotient / normalization	The exact hidden-state dynamics on the live prefixes.	A larger exact state is compressed into a smaller summary without loss of information on the relevant slice. This is the relation between the 4-state differential automaton and Machado’s scalar (ϕ_i, δ_i) recurrence, made explicit later in Proposition 13.
bit-vector compilation	The obstruction pattern and the local weight ingredients.	The carry logic is rewritten as solver-friendly masks and arithmetic on bit-vectors. For Azimi, validity is compiled exactly, while the weight is compiled approximately with a proved one-sided error bound.
state elimination	The same word-level function.	The carry is not discarded; it is absorbed into another exact description. Li’s carry-free Boolean recurrence is of this type.
graph transport	The full two-input addition graph up to an invertible linear transport of graphs.	Schulte-Geers moves from addition to a CCZ-equivalent quadratic shadow. This is an exact reformulation of the full two-input graph, not evidence that Miyano already used CCZ language in 1996.
coupled lift	The one-coordinate addition law in each coordinate.	One stacks several addition/subtraction coordinates and studies their compatibility by a block-matrix criterion. Lipmaa’s Quasi-Hadamard / PHT viewpoint is of this type.
observable projection	The carry object itself.	The hidden carry word is promoted to the visible quantity being measured. Guan–Ding’s $\Delta_{\text{Constant}_{\text{Key}}}(x)$ is exactly this type of bridge.
conditioning consumer	The same local carry law on a restricted subset of the sample space.	Partitioning and the Chaskey line do not redefine addition; they condition on low-bit context so that the existing local law becomes more deterministic on the retained subset.

Thus every bridge claim in this draft is a claim about persistence of the carry-governed local law under one of these exact relations. It is never a claim that all cited papers analyze the same probability space, solve the same optimization problem, or speak in the same historical language.

At this point the historical appendix has finished the task it was meant to perform. The fixed-addend object has been isolated, the hidden memory has been forced down to carry-based state, and the exact senses in which later papers preserve the same skeleton have been fixed. The remaining work in the main text is no longer historical persuasion but mathematical cash-out: first the differential branch, where two

related executions force a carry pair, and then the linear branch, where one signed execution leaves only a single surviving carry.

References

- [1] H. Miyano. *Addend Dependency of Differential/Linear Probability of Addition*. IEICE Transactions on Fundamentals, E81-A(1):106–109, 1998.
- [2] S. Moriai, K. Aoki, and K. Ohta. *On the linear probability of RC5*. Proceedings of the 1996 Symposium on Cryptography and Information Security, SCIS96-12C, 1996. In Japanese.
- [3] H. Miyano. *Differential/linear probability of addition upon the addend dependency*. Workshop on Design and Evaluation of Cryptographic Algorithm, November 1996. In Japanese.
- [4] Twilight-Dream-Of-Magic, “Reference code for fixed-addend modular addition recurrences,” GitHub repository. <https://github.com/Twilight-Dream-Of-Magic/exact-fixed-addend-theory> Accessed: 2026-03-29.
- [5] B. S. Kaliski Jr. and Y. L. Yin. *On Differential and Linear Cryptanalysis of the RC5 Encryption Algorithm*. CRYPTO 1995.
- [6] L. R. Knudsen and W. Meier. *Improved Differential Attacks on RC5*. CRYPTO 1996.
- [7] H. Lipmaa and S. Moriai. *Efficient Algorithms for Computing Differential Properties of Addition*. FSE 2001.
- [8] H. Lipmaa. *On Differential Properties of Pseudo-Hadamard Transform and Related Mappings*. INDOCRYPT 2002.
- [9] A. W. Machado. *Differential Probability of Modular Addition with a Constant Operand*. IACR ePrint 2001/052.
- [10] J. Wallen. *Linear Approximations of Addition Modulo 2^n* . FSE 2003.
- [11] J. Wallen. *On the Differential and Linear Properties of Addition*. HUT-TCS-A84, Helsinki University of Technology, 2003.
- [12] S. A. Azimi et al. *A Bit-Vector Differential Model for the Modular Addition by a Constant*. ASIACRYPT 2020.
- [13] S. A. Azimi et al. *A bit-vector differential model for the modular addition by a constant and its applications to differential and impossible-differential cryptanalysis*. Designs, Codes and Cryptography, 2022.
- [14] H. Lipmaa, J. Wallen, and P. Dumas. *On the Additive Differential Probability of Exclusive-Or*. FSE 2004.
- [15] N. Mouha et al. *Maximums of the Additive Differential Probability of Exclusive-Or*. 2021.
- [16] J. Guan and L. Ding. *Probability Analysis and Application of XOR Difference Between $(X+K) \bmod 2^n$ and $X \oplus K$* . IEEE, 2010.
- [17] Z. Li et al. *Analysis of Addition Modulo 2^n on Boolean Function*. IEEE, 2011.
- [18] E. Schulte-Geers. *On CCZ-equivalence of Addition mod 2^n* . WCC 2011.
- [19] G. Leurent. *Differential and Linear Cryptanalysis of ARX with Partitioning – Application to FEAL and Chaskey*. 2015.
- [20] G. Leurent. *Improved Differential-Linear Cryptanalysis of 7-Round Chaskey with Partitioning*. 2016.
- [21] S. Dey et al. *Revamped Differential-Linear Cryptanalysis on Reduced Round ChaCha*. 2022.
- [22] C. Dey and S. Sarkar. *A New Distinguishing Attack on Reduced Round ChaCha Permutation*. 2023.